

Subject : **ARTIFICIAL INTELLIGENCE**
Sub code : IS52
Credits : 2:0:1
Faculty : Dr. Anitha P

Text Book:

1. Stuart Russel, Peter Norvig: Artificial Intelligence - A Modern Approach, 3rd Edition, Pearson Education, 2012.
2. Elaine Rich, Kevin Knight, Shivashankar B Nair: Artificial Intelligence, 3rd Edition, Tata McGraw Hill, 2011

References:

1. Peter Jackson, “Introduction to Expert Systems”, 3rd Edition, Pearson Education, 2007.
2. Deepak Khemani “Artificial Intelligence”, Tata Mc Graw Hill Education 2013.
3. <http://nptel.ac.in>

UNIT-I

Introduction: Definition of AI. Foundation of Artificial Intelligence. **Intelligent Agents:** Agents and Environments, Rationality, The Nature of Environments, The Structure of Agents.

Problem-solving by search: Problem-Solving Agents, Uninformed Search Strategies: Bidirectional Search. Informed Search Strategies: A* Search, Heuristic Functions.

UNIT II

Adversarial Search: Games, Optimal Decisions in Games, Alpha Beta Pruning, Imperfect Real-Time Decision. **Logical Agents:** Knowledge-Based Agents, The Wumpus World, Logic.

Propositional Logic: A very simple logic, Effective Propositional Model Checking, Agents Based on Propositional Logic. **First Order Logic:** Wumpus World representation, Knowledge Engineering in First-Order Logic.

UNIT III

Interference in First-order Logic: Propositional vs. First-Order Inference, Unification and Lifting, Forward chaining, Backward Chaining. Resolution. **Classical Planning:** Definition, Algorithms for Planning as State-Space Search, Planning Graphs, Other Planning Approaches.

UNIT IV

Knowledge Representation: ontological engineering, categories and objects, Events, mental objects and modal logic, Reasoning systems for categories. **Uncertainty:** Acting under Uncertainty, Inference using Full Joint Distributions, Independence.

UNIT V

Uncertainty: The Wumpus World Revisited, Learning from Examples: Forms of Learning. **Robotics:** Introduction, Hardware, Perception, Planning to Move, Planning Uncertain Movement, Moving, Robotic Software Architecture, Application Domains.

COURSE OUTCOMES:

1. Identify the fundamental characteristics and challenging issues of Artificial Intelligence (AI) systems (PO-1,2,3,4,12, PSO-2)
2. Apply various general purpose search algorithm as solutions for various problem-solving agents (PO -1,2,3,4,12, PSO-2)
3. Apply various symbolic knowledge representation to specify domains and reasoning tasks of a situated intelligent agent. (PO 4,5,9,12, PSO-2)
4. Apply algorithmic approach for planning and solving AI solutions that require problem solving, inference, perception, knowledge representation, and learning. (PO-1,4,5,6,7,PSO-2)
5. Extract conclusions on learning and quantify the uncertainty in the conclusions obtained from uncertain knowledge. (PO-5,9, PSO-2)

Introduction to AI

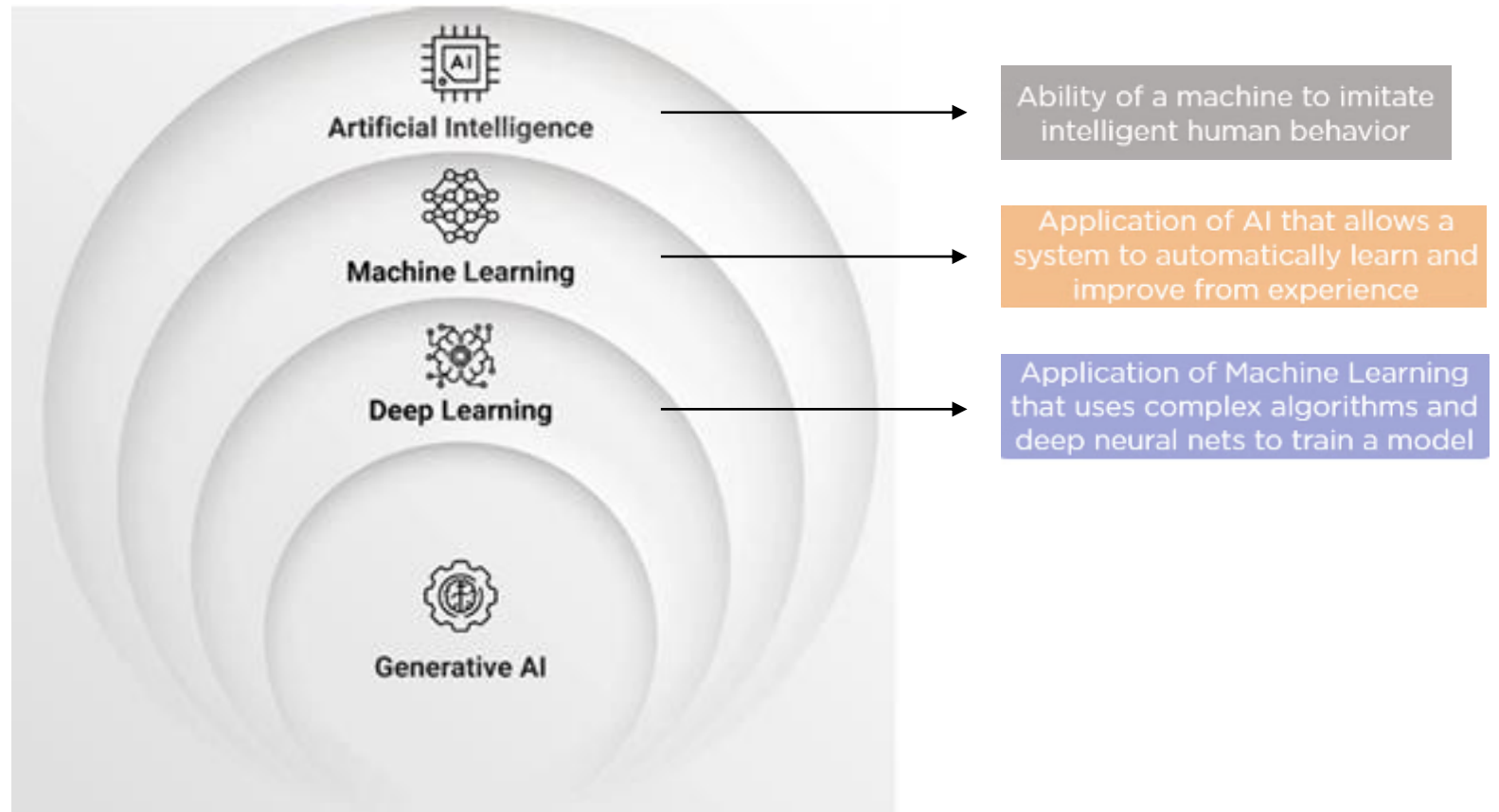
AI is a field of computer science that aims to create machines capable of emulating human intelligence, enabling them to perform tasks like problem-solving and learning from data.

Gen AI:

- Generative AI, a branch of artificial intelligence and a subset of Deep Learning, focuses on creating models capable of generating new content that resemble existing data. It aims to generate content that is indistinguishable from what might be created by humans. Generative AI models that use deep neural networks to generate realistic content such as images, text, or even music.

Large Language Models (LLMs)

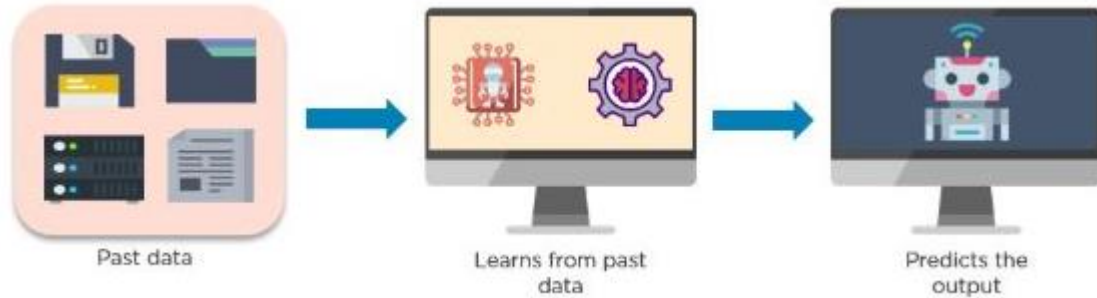
Type of AI model designed to understand, generate, and manipulate human language. These models are trained on massive datasets of text and use sophisticated algorithms, particularly **deep learning**, to predict and generate human-like responses to various language tasks.



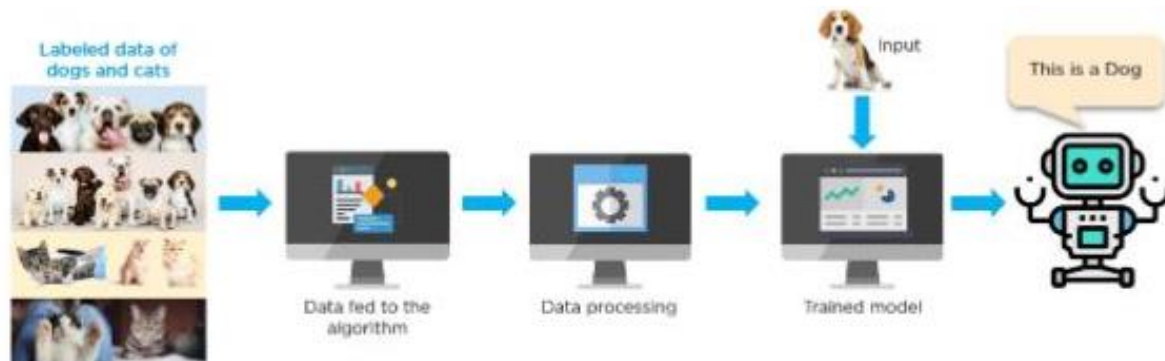


The person wants to know the current temperature in Chicago.
The person's voice is first converted into a machine-readable format.
The formatted data is then fed into the Amazon Alexa system for processing and analyzing.
Finally, Alexa returns the desired voice output via Amazon Echo

Machine learning accesses vast amounts of data (both structured and unstructured) and learns from it to predict the future. It learns from the data by using multiple algorithms and techniques. Below is a diagram that shows how a machine learns from data.



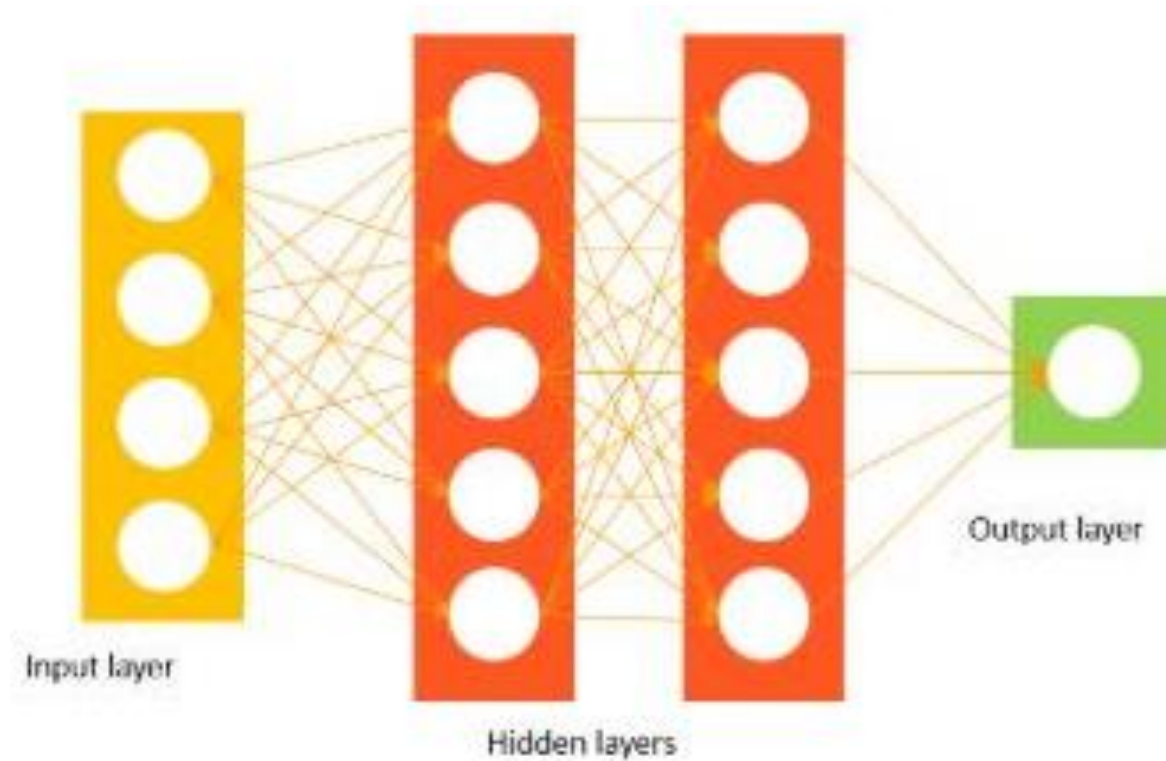
Supervised Learning



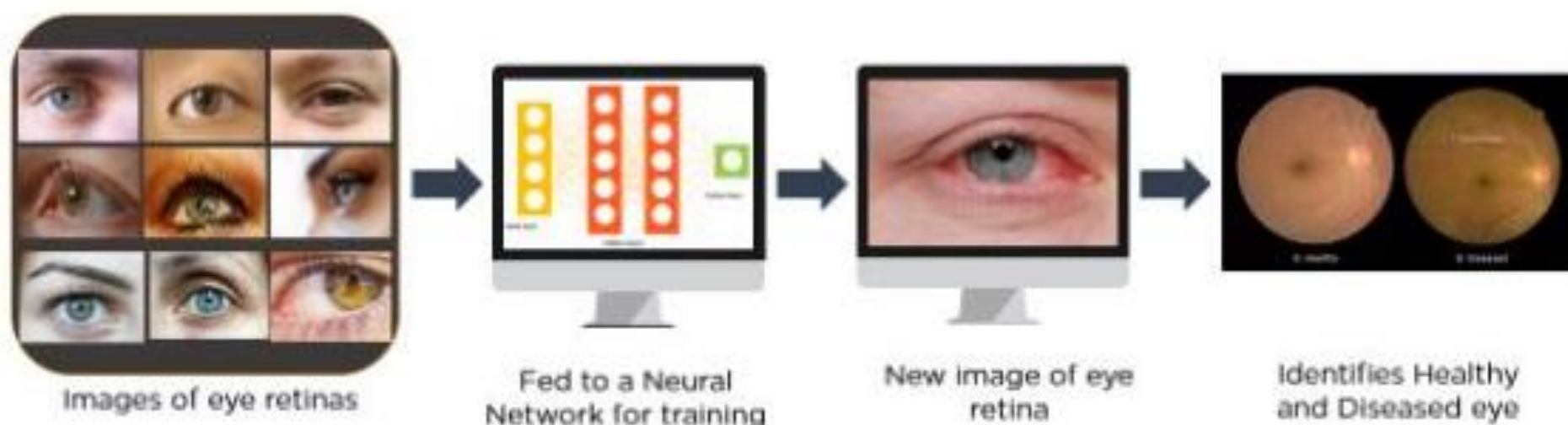
Unsupervised Learning



- Deep learning is a subset of machine learning that deals with algorithms inspired by the structure and function of the human brain. Deep learning can work with an enormous amount of both structured and unstructured data. Deep learning's core concept lies in artificial neural networks, which enable machines to make decisions.
- Machine learning algorithms usually require structured data, whereas deep learning networks work on multiple layers of artificial neural networks.



Here is an example of a neural network that uses large sets of unlabeled data of eye retinas. The network model is trained on this data to find out whether or not a person has diabetic retinopathy.



Common Examples of Large Language Models:

- GPT Series (GPT-3, GPT-4)
- BERT (Bidirectional Encoder Representations from Transformers) –by google
- T5 (Text-To-Text Transfer Transformer)- by google

Applications of LLM

- Text Generation
- Chatbots and Conversational AI
- Translation
- Question Answering
- Summarization
- Sentiment Analysis
- Content Creation
- Coding Assistance

Applications of AI

• Medical Diagnosis • Drug Discovery • Personalized Medicine • Robotic Surgery • Virtual Health Assistants • Fraud Detection • Algorithmic Trading • Credit Scoring • Customer Service • Financial Forecasting • Recommendation Engines • Inventory Management	• Customer Service • Dynamic Pricing • Virtual Try-Ons • Self-Driving Cars • Driver Assistance • Traffic Management • Industrial Automation • Predictive Maintenance • Quality Control • Content Recommendation • Content Creation • Video Game AI - - - -
--	--

Prompt engineering:

- Practice of crafting and refining prompts (input instructions or queries) to produce the most effective or desired response from AI models, particularly in natural language processing (NLP) systems like OpenAI's GPT models.
- Since AI language models generate outputs based on the input they receive, prompt engineering is essential for guiding these models to provide accurate, relevant, or creative results.

Basic prompt:

What is the Bengaluru weather today?

The model may provide a generic answer or a response without much context.

Engineered Prompt:

“Provide a brief, 3-sentence explanation of today's weather in bengaluru, including temperature, chance of rain, and wind speed”.

More specific

LangChain is a framework designed to help developers build applications that integrate **LLMs** with external data sources, logic, and workflows. LangChain is especially useful for developing chatbots, intelligent agents, and knowledge-based systems that go beyond simple text generation.

UNIT I

Artificial Intelligence:

- The name Artificial Intelligence was coined in 1956 by John McCarthy after world war II.
- The field of **artificial intelligence** not only understands but also used to *build* intelligent entities.
- AI currently encompasses a huge variety of subfields, such as playing chess, proving mathematical theorems, writing poetry, driving a car on a crowded street, and diagnosing diseases.
- AI is relevant to any intellectual task; it is truly a universal field.

WHAT IS AI?

- we see eight definitions of AI, laid out along two dimensions. The definitions on top are concerned with *thought processes* and *reasoning*, whereas the ones on the bottom address *behavior*.
- The definitions on the left measure success in terms of fidelity to *human* performance whereas the ones on the right measure against an *ideal* performance measure, called **rationality**. A system is rational if it does the “right thing,” given what it knows.

All four approaches to AI have been followed, each by different people with different methods.

Thinking Humanly “The exciting new effort to make computers think . . . <i>machines with minds</i>, in the full and literal sense.” (Haugeland, 1985) “[The automation of] activities that we associate with human thinking, activities such as decision-making, problem solving, learning . . .” (Bellman, 1978)	Thinking Rationally “The study of mental faculties through the use of computational models.” (Charniak and McDermott, 1985) “The study of the computations that make it possible to perceive, reason, and act.” (Winston, 1992)
Acting Humanly “The art of creating machines that perform functions that require intelligence when performed by people.” (Kurzweil, 1990) “The study of how to make computers do things at which, at the moment, people are better.” (Rich and Knight, 1991)	Acting Rationally “Computational Intelligence is the study of the design of intelligent agents.” (Poole <i>et al.</i>, 1998) “AI . . . is concerned with intelligent behavior in artifacts.” (Nilsson, 1998)

Figure 1.1 Some definitions of artificial intelligence, organized into four categories.

I Acting humanly: The Turing Test approach

- The Turing Test, proposed by Alan Turing (1950), was designed to provide a satisfactory operational definition of intelligence. A computer passes the test if a human interrogator, after posing some written questions, cannot tell whether the written responses come from a person or from a computer.

The computer would need to possess the following capabilities:

- **natural language processing** to enable it to communicate successfully in English
- **knowledge representation** to store what it knows or hears
- **automated reasoning** to use the stored information to answer questions and to draw new conclusions;
- **machine learning** to adapt to new circumstances and to detect and extrapolate patterns

Total Turing Test: includes a video signal so that the interrogator can test the subject's perceptual abilities, as well as the opportunity for the interrogator to - pass physical objects "through the hatch." To pass the total Turing Test, the computer will need

- **computer Vision** to perceive objects, and
- **Robotics** to manipulate objects and move about.
- Above six disciplines compose most of AI and Turing deserves credit for designing a test that remains relevant 60 years later.

II Thinking humanly: The cognitive modeling approach

Determining how humans think and need to get *inside* the actual workings of human minds. Following are the 3 ways to do this:

- Through introspection—trying to catch our own thoughts as they go by;
- Through psychological experiments—observing a person in action;
- Through brain imaging—observing the brain in action.

Example, Allen Newell and Herbert Simon, who developed GPS, the “General Problem Solver”.

- Cognitive science is a fascinating field in itself, We will occasionally comment on similarities or differences between AI techniques and human cognition

III Thinking rationally: The “laws of thought” approach:

- The Greek philosopher Aristotle was one of the first to attempt to codify “right thinking,” that is, irrefutable reasoning processes.
- His **syllogisms** provided patterns for argument structures that always yielded correct conclusions when given correct premises—for example, “Socrates is a man; all men are mortal; therefore, Socrates is mortal.”
- These laws of thought were supposed to govern the operation of the mind; their study initiated the field called **logic**.
- Logicians in the 19th century developed a precise notation for statements about all kinds of objects in the world and the relations among them

- By 1965, programs existed that could, in principle, solve *any* solvable problem described in logical notation.
- The so-called **logicist** tradition within artificial intelligence hopes to build on such programs to create intelligent systems.

Two main obstacles to this approach:

- It is not easy to take informal knowledge and state it in the formal terms required by logical notation, particularly when the knowledge is less than 100% certain.
- There is a big difference between solving a problem “in principle” and solving it in practice.

IV Acting rationally: The rational agent approach:

- An agent is just something that acts.
- All computer programs do something, but computer agents are expected to do more: operate autonomously, perceive their environment, persist over a prolonged time period, adapt to change, and create and pursue goals.
- A **rational agent** is one that acts so as to achieve the best outcome or, when there is uncertainty, the best-expected outcome.
- In the “**laws of thought**” approach to AI, the emphasis was on correct inferences.
- Making correct inferences is sometimes *part* of being a rational agent because one way to act rationally is to reason logically to the conclusion that a given action will achieve one’s goals and then act on that conclusion.

The rational-agent approach has two advantages over the other approaches.

- First, it is more general than the “laws of thought” approach because correct inference is just one of several possible mechanisms for achieving rationality.
- Second, it is more amenable to scientific development than are approaches based on human behavior or human thought.

1.2 The foundations of AI

1.2.1 Philosophy

- Can formal rules be used to draw valid conclusions?
- How does the mind arise from a physical brain?
- Where does knowledge come from?
- How does knowledge lead to action?

- Aristotle developed an informal system of syllogisms for proper reasoning, which in principle allowed one to generate conclusions mechanically, given initial premises.
- Followed by Ramon Lull - useful reasoning could actually be carried out by a mechanical artifact.
- Thomas Hobbes- reasoning was like numerical computation, that “we add and subtract in our silent thoughts.”
- Leonardo da Vinci- designed but did not build a mechanical calculator;

1.2.2 Mathematics:

- What are the formal rules to draw valid conclusions?
- What can be computed?
- How do we reason with uncertain information?

- The idea of formal logic can be traced back to the philosophers of ancient Greece, but its mathematical development really began with the work of George Boole, who worked out the details of propositional, or Boolean logic.
- Gottlob Frege extended Boole's logic to include objects and relations.
- Alfred Tarski (1902–1983) introduced a theory of reference that shows how to relate the objects in a logic to objects in the real world.
- Alfred Tarski (1902–1983) introduced a theory of reference that shows how to relate the objects in a logic to objects in the real world.
 - The first nontrivial **algorithm** is thought to be Euclid's algorithm for computing greatest common divisors.
 - Kurt Godel (1906–1978) showed that there exists an effective procedure to prove any true statement in the first-order logic of Frege and Russell, but that first-order logic could not capture the principle of mathematical induction needed to characterize the natural numbers.
 - His **incompleteness theorem** showed that in any formal theory as strong as Peano arithmetic (the elementary theory of natural numbers), there are true statements that are undecidable in the sense that they have no proof within the theory.

- Alan Turing- try to characterize exactly which functions *are* **computable**—capable of being computed.
- Although decidability and computability are important to an understanding of computation, the notion of **tractability** has had an even greater impact.
- The theory of **NP-completeness** showed the existence of large classes of canonical combinatorial search and reasoning problems that are NP-complete.
- Besides logic and computation, the third great contribution of mathematics to AI is the theory of **probability**.

1.2.3 Economics

- How should we make decisions so as to maximize payoff?
- How should we do this when others may not go along?
- How should we do this when the payoff may be far in the future?

1.2.4 Neuroscience

How do brains process information?

- **Neuroscience** is the study of the nervous system, particularly the brain.
- Although the exact way in which the brain enables thought is one of the great mysteries of science.
- Aristotle wrote, “Of all the animals, man has the largest brain in proportion to his size.”
- Paul Broca’s study of aphasia (speech deficit) in brain-damaged patients in 1861 demonstrated the existence of localized areas of the brain responsible for specific cognitive functions.
- he showed that speech production was localized to the portion of the left hemisphere now called Broca’s area.
- Nicolas Rashevsky was the first to apply mathematical models to the study of the nervous system.

Nerve cell or Neurons:

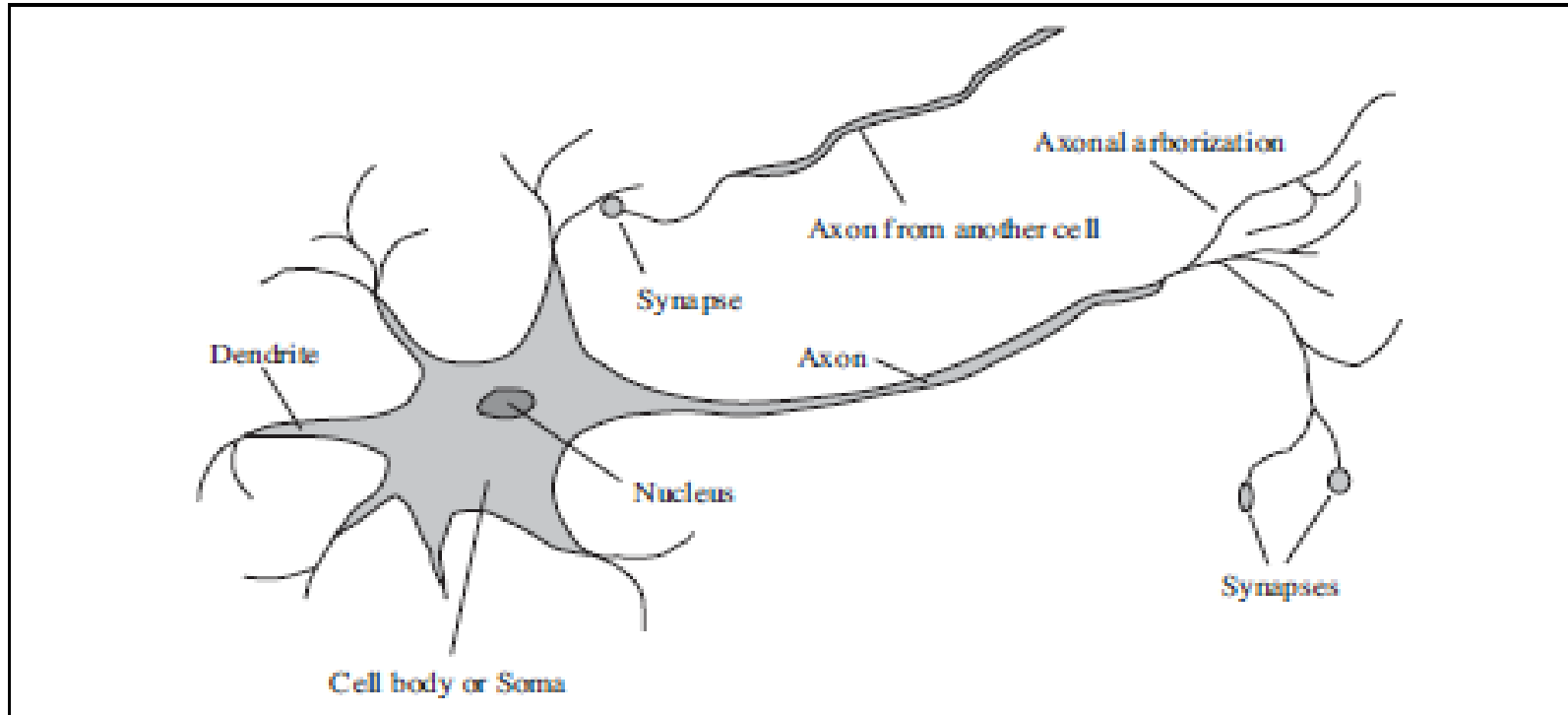


Figure 1.2 The parts of a nerve cell or neuron. Each neuron consists of a cell body, or soma, that contains a cell nucleus. Branching out from the cell body are a number of fibers called dendrites and a single long fiber called the axon. The axon stretches out for a long distance, much longer than the scale in this diagram indicates. Typically, an axon is 1 cm long (100 times the diameter of the cell body), but can reach up to 1 meter. A neuron makes connections with 10 to 100,000 other neurons at junctions called synapses. Signals are propagated from neuron to neuron by a complicated electrochemical reaction. The signals control brain activity in the short term and also enable long-term changes in the connectivity of neurons. These mechanisms are thought to form the basis for learning in the brain. Most information processing goes on in the cerebral cortex, the outer layer of the brain. The basic organizational unit appears to be a column of tissue about 0.5 mm in diameter, containing about 20,000 neurons and extending the full depth of the cortex about 4 mm in humans).

- The measurement of intact brain activity began in 1929 with the invention by Hans Berger of the electroencephalo graph (EEG).
- The recent development of functional magnetic resonance imaging (fMRI) is giving neuroscientists unprecedentedly detailed images of brain activity, enabling measurements that correspond in interesting ways to ongoing cognitive processes. These are augmented by advances in single-cell recording of neuron activity.
- Individual neurons can be stimulated electrically, chemically, or even optically allowing neuronal input– output relationships to be mapped. Despite these advances, we are still a long way from understanding how cognitive processes actually work.

Comparative analysis of Supercomputer, PC and Human Brain

	Supercomputer	Personal Computer	Human Brain
Computational units	10^4 CPUs, 10^{12} transistors	4 CPUs, 10^9 transistors	10^{11} neurons
Storage units	10^{14} bits RAM 10^{15} bits disk	10^{11} bits RAM 10^{13} bits disk	10^{11} neurons 10^{14} synapses
Cycle time	10^{-9} sec	10^{-9} sec	10^{-3} sec
Operations/sec	10^{15}	10^{10}	10^{17}
Memory updates/sec	10^{14}	10^{10}	10^{14}
Figure 1.3 A crude comparison of the raw computational resources available to the IBM BLUE GENE supercomputer, a typical personal computer of 2008, and the human brain. The brain's numbers are essentially fixed, whereas the supercomputer's numbers have been increasing by a factor of 10 every 5 years or so, allowing it to achieve rough parity with the brain. The personal computer lags behind on all metrics except cycle time.			

For discussion

1. To what extent are the following computer systems instances of artificial intelligence:

- supermarket bar code scanners.
- Web search engines.
- Voice-activated telephone menus.
- Internet routing algorithms that respond dynamically to the state of the network.

2. Is AI a science, or is it engineering? Or neither or both? Explain.

3. “Surely computers cannot be intelligent—they can do only what their programmers tell them.” Is the latter statement true, and does it imply the former?

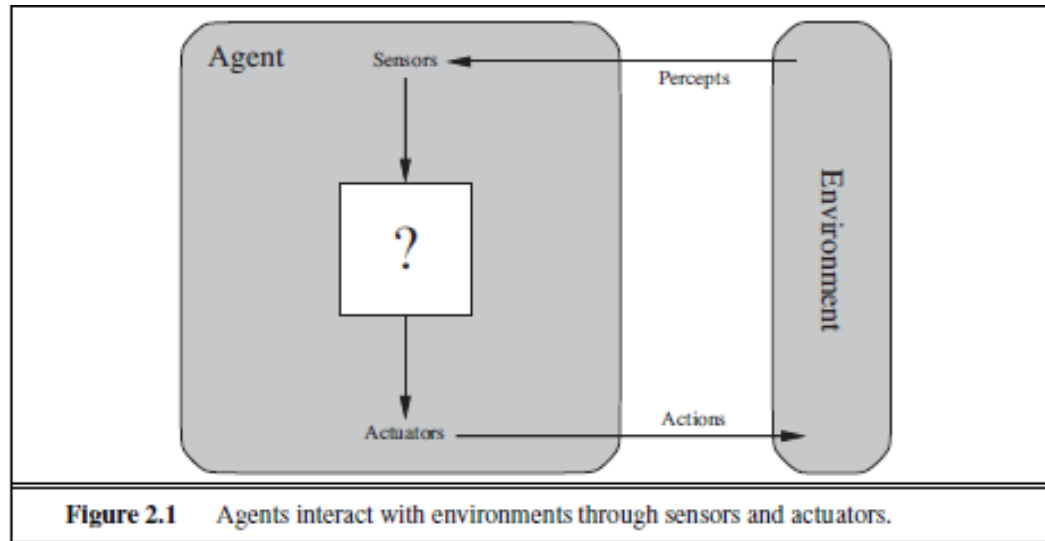
4. Examine the AI literature to discover whether the following tasks can currently be solved by computers:

- a. Playing a decent game of table tennis (Ping-Pong).
- b. Driving in the center of Cairo, Egypt.
- c. Driving in Victorville, California.
- d. Buying a week's worth of groceries at the market.
- e. Buying a week's worth of groceries on the Web.
- f. Playing a decent game of bridge at a competitive level.
- g. Discovering and proving new mathematical theorems.
- h. Writing an intentionally funny story.
- i. Giving competent legal advice in a specialized area of law.
- j. Translating spoken English into spoken Swedish in real time.
- k. Performing a complex surgical operation.

Intelligent Agents:

Agents and Environment

Agent: An agent is anything that can be viewed as perceiving its environment through sensors and acting upon that environment through actuators.



Ex: human agent, robotic agent, s/w agent.....

Percept: refer to the agent's perceptual inputs at any given instant.

- An agent's **percept sequence** is the complete history of everything the agent has ever perceived.
- In general, an agent's choice of action at any given instant can depend on the entire percept sequence observed to date, but not on anything it hasn't perceived.
- By specifying the agent's choice of action for every possible percept sequence, we have said more or less everything there is to say about the agent.
- Mathematically speaking, we say that an agent's behavior is described by the **agent function** that maps any given percept sequence to an action.
- imagine ***tabulating*** the agent function that describes any given agent;

Tabulating an agent function:

- for most agents, this would be a very large table—infinite, in fact, unless we place a bound on the length of percept sequences we want to consider.
- Given an agent to experiment with, we can, in principle, construct this table by trying out all possible percept sequences and recording which actions the agent does in response.
- *It is an **external** characterization* of the agent.
- **Agent Program: *Internally***, the agent function for an artificial agent will be implemented by an **agent program**.
- The agent function is an abstract mathematical description; the agent program is a concrete implementation, running within some physical system.

A very simple example—the **vacuum-cleaner** world shown in Figure 2.2

- This particular world has just two locations: squares A and B. The vacuum agent perceives which square it is in and whether there is dirt in the square. It can choose to move left, move right, suck up the dirt, or do nothing.
- One very simple agent function is the following: if the current square is dirty, then suck; otherwise, move to the other square.
- A partial tabulation of this agent function is shown in Figure 2.3 and an agent program that implements it.

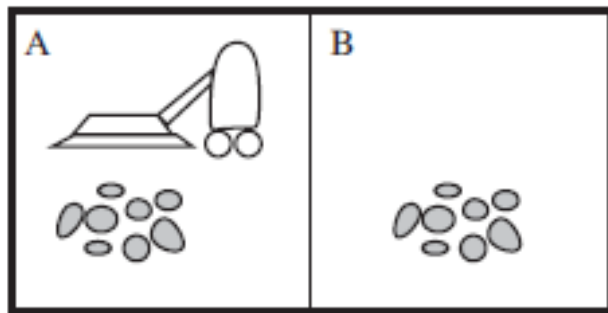


Figure 2.2 A vacuum-cleaner world with just two locations.

Percept sequence	Action
$[A, \text{Clean}]$	<i>Right</i>
$[A, \text{Dirty}]$	<i>Suck</i>
$[B, \text{Clean}]$	<i>Left</i>
$[B, \text{Dirty}]$	<i>Suck</i>
$[A, \text{Clean}], [A, \text{Clean}]$	<i>Right</i>
$[A, \text{Clean}], [A, \text{Dirty}]$	<i>Suck</i>
$\vdots$	$\vdots$
$[A, \text{Clean}], [A, \text{Clean}], [A, \text{Clean}]$	<i>Right</i>
$[A, \text{Clean}], [A, \text{Clean}], [A, \text{Dirty}]$	<i>Suck</i>
$\vdots$	$\vdots$

Figure 2.3 Partial tabulation of a simple agent function for the vacuum-cleaner world shown in Figure 2.2.

function REFLEX-VACUUM-AGENT($[location, status]$) **returns** an action

if $status = \text{Dirty}$ **then return** *Suck*
else if $location = A$ **then return** *Right*
else if $location = B$ **then return** *Left*

Figure 2.8 The agent program for a simple reflex agent in the two-state vacuum environment. This program implements the agent function tabulated in Figure 2.3.

Good Behavior: The concept of Rationality

- A **rational agent** is one that does the right thing – for ex, every entry in the table for the agent function is filled out correctly.
- Doing the right thing is better than doing the wrong thing, but what does it mean to do the right thing?
- When an agent is plunked down in an environment, it generates a sequence of actions according to the percepts it receives.
- This sequence of actions causes the environment to go through a sequence of states. If the sequence is desirable, then the agent has performed well.
- This notion of desirability is captured by a **performance measure** that evaluates any given sequence of environment states.
- **Note: Measuring performance is not easy:** For ex, we might propose to measure performance by the amount of dirt cleaned up in a single eight-hour shift

- As a general rule, it is better to design performance measures according to what one actually wants in the environment, rather than according to how one thinks the agent should behave.

2.2.1 Rationality

What is rational at any given time depends on four things:

- The performance measure that defines the criterion of success.
- The agent's prior knowledge of the environment.
- The actions that the agent can perform.
- The agent's percept sequence to date.

Above leads to a **definition of a Rational agent** - For each possible percept sequence, a rational agent should select an action that is expected to maximize its performance measure, given the evidence provided by the percept sequence and whatever built-in knowledge the agent has.

Consider the simple vacuum-cleaner agent that cleans a square if it is dirty and moves to the other square if not; this is the agent function tabulated in Figure 2.3. Is this a rational agent? That depends! First, we need to say what the performance measure is, what is known about the environment, and what sensors and actuators the agent has. Let us assume the following:

- The performance measure awards one point for each clean square at each time step, over a “lifetime” of 1000 time steps.
- The “geography” of the environment is known *a priori* (Figure 2.2) but the dirt distribution and the initial location of the agent are not. Clean squares stay clean and sucking cleans the current square. The Left and Right actions move the agent left and right except when this would take the agent outside the environment, in which case the agent remains where it is.
- The only available actions are Left , Right, and Suck.
- The agent correctly perceives its location and whether that location contains dirt.

We claim that *under these circumstances* the agent is indeed rational; its expected performance is at least as high as any other agent’s.

- For example, once all the dirt is cleaned up, the agent will oscillate needlessly back and forth; if the performance measure includes a penalty of one point for each movement left or right, the agent will fare poorly.
- A better agent for this case would do nothing once it is sure that all the squares are clean. If clean squares can become dirty again, the agent should occasionally check and re-clean them if needed. If the geography of the environment is unknown, the agent will need to explore it rather than stick to squares A and B.

Omniscience, learning, and autonomy:

- An omniscient agent knows the *actual* outcome of its actions and can act accordingly; but omniscience is impossible in reality.
- This example shows that rationality is not the same as perfection. Rationality maximizes *expected* performance, while perfection maximizes *actual* performance.
- **Learning:** Our definition requires a rational agent not only to gather information but also to **learn** as much as possible from what it perceives.

- To the extent that an agent relies on the prior knowledge of its designer rather than on its own percepts, we say that the agent lacks **autonomy**. A rational agent should be autonomous—it should learn what it can to compensate for partial or incorrect prior knowledge.

Specifying the task environment:

- The rationality of the simple vacuum-cleaner agent, we had to specify the performance measure, the environment, and the agent's actuators and sensors. We group all these under the heading of the **task environment**. We call this as PEAS (**P**erformance, **E**nvironment, **A**ctuators, **S**ensors).
- The vacuum world was a simple example; let us consider a more complex problem: an automated taxi driver.
- Figure 2.4 summarizes the PEAS description for the taxi's task environment.

Task environment for Taxi

Agent Type	Performance Measure	Environment	Actuators	Sensors
Taxi driver	Safe, fast, legal, comfortable trip, maximize profits	Roads, other traffic, pedestrians, customers	Steering, accelerator, brake, signal, horn, display	Cameras, sonar, speedometer, GPS, odometer, accelerometer, engine sensors, keyboard

Figure 2.4 PEAS description of the task environment for an automated taxi.

Ex:

Agent Type	Performance Measure	Environment	Actuators	Sensors
Medical diagnosis system	Healthy patient, reduced costs	Patient, hospital, staff	Display of questions, tests, diagnoses, treatments, referrals	Keyboard entry of symptoms, findings, patient's answers
Satellite image analysis system	Correct image categorization	Downlink from orbiting satellite	Display of scene categorization	Color pixel arrays
Part-picking robot	Percentage of parts in correct bins	Conveyor belt with parts; bins	Jointed arm and hand	Camera, joint angle sensors
Refinery controller	Purity, yield, safety	Refinery, operators	Valves, pumps, heaters, displays	Temperature, pressure, chemical sensors
Interactive English tutor	Student's score on test	Set of students, testing agency	Display of exercises, suggestions, corrections	Keyboard entry

Figure 2.5 Examples of agent types and their PEAS descriptions.

2.3.2 Properties of task environments

- Fully observable vs. partially observable
- Single agent vs. multiagent
- Deterministic vs Stochastic
- Episodic vs. sequential
- Static vs Dynamic
- Discrete vs Continuous
- Known vs Unknown

If an agent's sensors give it access to the complete state of the environment at each point in time, then we say that the task environment is fully observable.

Examples:

Task Environment	Observable	Agents	Deterministic	Episodic	Static	Discrete
Crossword puzzle	Fully	Single	Deterministic	Sequential	Static	Discrete
Chess with a clock	Fully	Multi	Deterministic	Sequential	Semi	Discrete
Poker	Partially	Multi	Stochastic	Sequential	Static	Discrete
Backgammon	Fully	Multi	Stochastic	Sequential	Static	Discrete
Taxi driving	Partially	Multi	Stochastic	Sequential	Dynamic	Continuous
Medical diagnosis	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Image analysis	Fully	Single	Deterministic	Episodic	Semi	Continuous
Part-picking robot	Partially	Single	Stochastic	Episodic	Dynamic	Continuous
Refinery controller	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Interactive English tutor	Partially	Multi	Stochastic	Sequential	Dynamic	Discrete

Figure 2.6 Examples of task environments and their characteristics.

2.4 The Structure of Agents:

- The job of AI is to design an **agent program** that implements the agent function—the mapping from percepts to actions.
- We assume this program will run on some sort of computing device with physical sensors and actuators—called the **architecture**:
 - *agent = architecture + program*
- **Agent programs:** The agent program takes the current percept as input from the sensors and returns an action to the actuators. The difference between the agent program, which takes the current percept as input, and the agent function, which takes the entire percept history

```
function TABLE-DRIVEN-AGENT(percept) returns an action
  persistent: percepts, a sequence, initially empty
               table, a table of actions, indexed by percept sequences, initially empty

  append percept to the end of percepts
  action ← LOOKUP(percepts, table)
  return action
```

Figure 2.7 The TABLE-DRIVEN-AGENT program is invoked for each percept and returns an action each time. It retains the complete percept sequence in

Intelligent Agent: An intelligent agent is an autonomous entity which act upon an environment using sensors and actuators for achieving goals.

A **Rational Agent** is an Intelligent Agent that makes decisions based on logical reasoning and optimizes its behavior to achieve a specific goal.

Four basic kinds of agent programs that embody the principles underlying almost all intelligent systems:

- Simple reflex agents;
- Model-based reflex agents;
- Goal-based agents; and
- Utility-based agents.



- **Simple reflex agents:** These agents select actions on the basis of the *current* percept, ignoring the rest of the percept history. For example, the vacuum agent, because its decision is based only on the current location and on whether that location contains dirt. Agent pgm & schematic diagram is shown below:

```
function REFLEX-VACUUM-AGENT([location,status]) returns an action
  if status = Dirty then return Suck
  else if location = A then return Right
  else if location = B then return Left
```

Figure 2.8 The agent program for a simple reflex agent in the two-state vacuum environment. This program implements the agent function tabulated in Figure 2.3.

Problems with Simple reflex agents are :

- Very limited intelligence.
- No knowledge of non-perceptual parts of the state.
- Usually too big to generate and store.
- If there is any change in the environment, then the collection is updated.

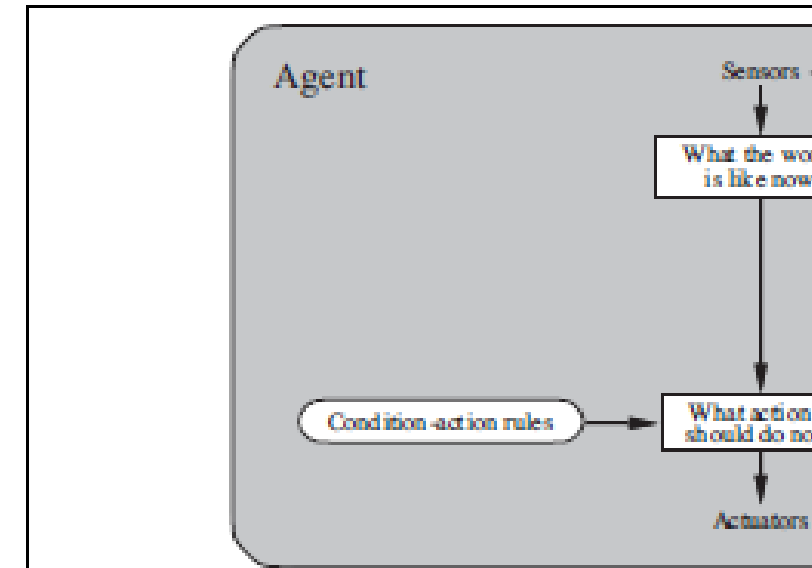


Figure 2.9 Schematic diagram of a simple reflex agent

```
function SIMPLE-REFLEX-AGENT(percept) returns an action
  persistent: rules, a set of condition-action rules

  state ← INTERPRET-INPUT(percept)
  rule ← RULE-MATCH(state, rules)
  action ← rule.ACTION
  return action
```

- The reflex agent will work only if the correct decision can be made on the basis of only the current percept—that is, only if the environment is fully observable.
- Suppose that a simple reflex vacuum agent is deprived of its location sensor and has only a dirt sensor. Such an agent has just two possible percepts: [Dirty] and [Clean]. It can Suck in response to [Dirty];
- what should it do in response to [Clean]? Moving Left fails (forever) if it happens to start in a square A, and moving Right fails (forever) if it happens to start in square B. Infinite loops are often unavoidable for simple reflex agents operating in partially observable environments.

Model-based reflex agents:

- The most effective way to handle partial observability is for the agent to keep track of the part of the world it can't see now.
- the agent should maintain some sort of **internal state** that depends on the percept history and thereby reflects at least some of the unobserved aspects of the current state.

- when the agent turns the steering wheel clockwise, the car turns to the right, or after driving for five minutes northbound on the freeway, one is usually about five miles north of where one was five minutes ago.
- This knowledge about “how the world works”—whether implemented in simple Boolean circuits or in complete scientific theories—is called a **model** of the world. An agent that uses such a model is called a model-based reflex agent.

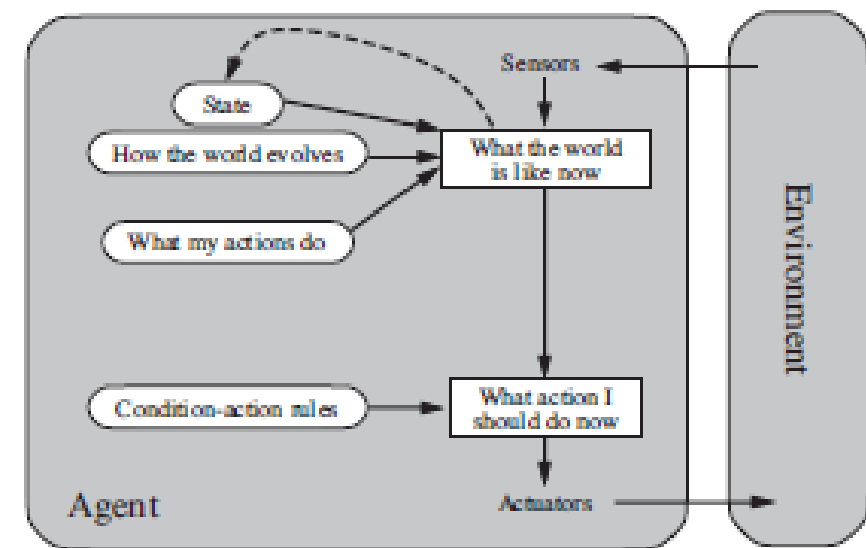


Figure 2.11 A model-based reflex agent.

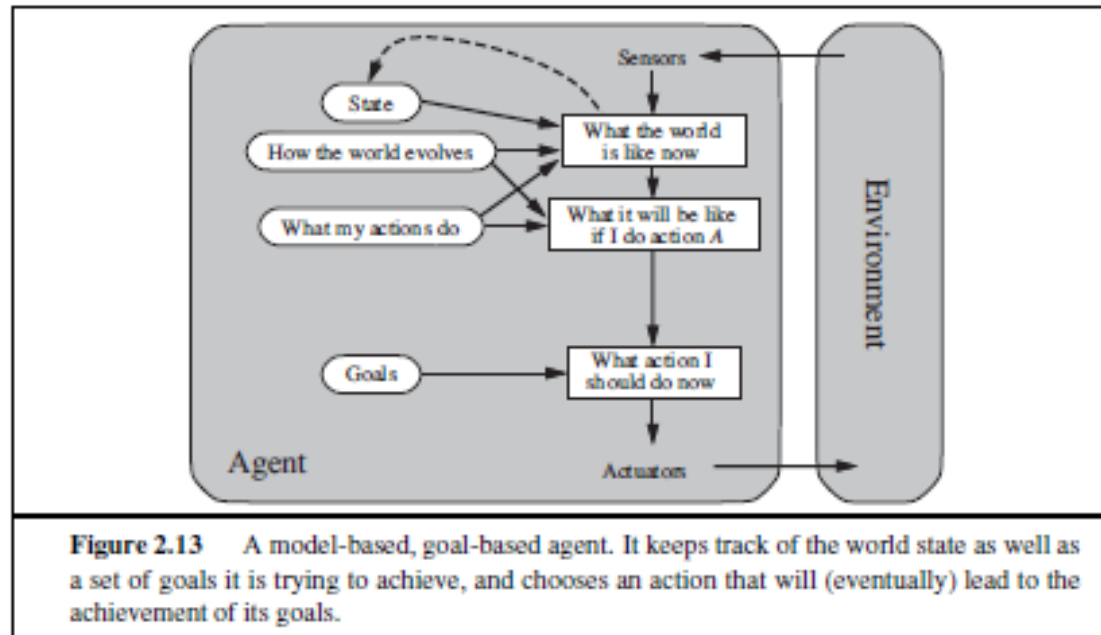
```

function MODEL-BASED-REFLEX-AGENT(percept) returns an action
  persistent: state, the agent's current conception of the world state
               model, a description of how the next state depends on current state and action
               rules, a set of condition-action rules
               action, the most recent action, initially none

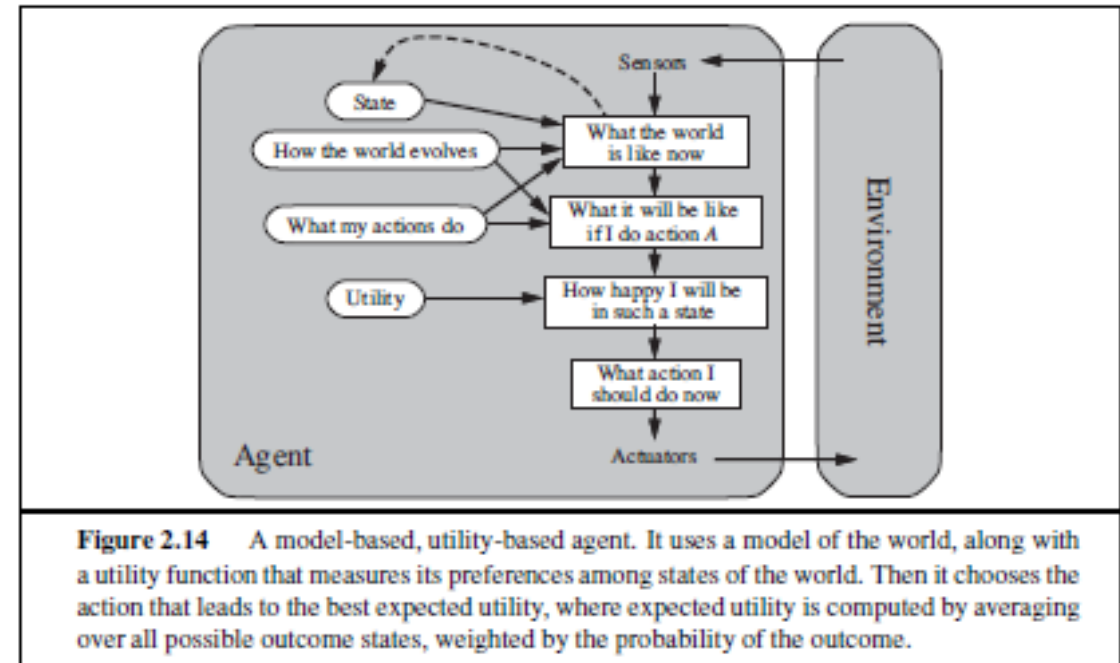
  state ← UPDATE-STATE(state, action, percept, model)
  rule ← RULE-MATCH(state, rules)

```

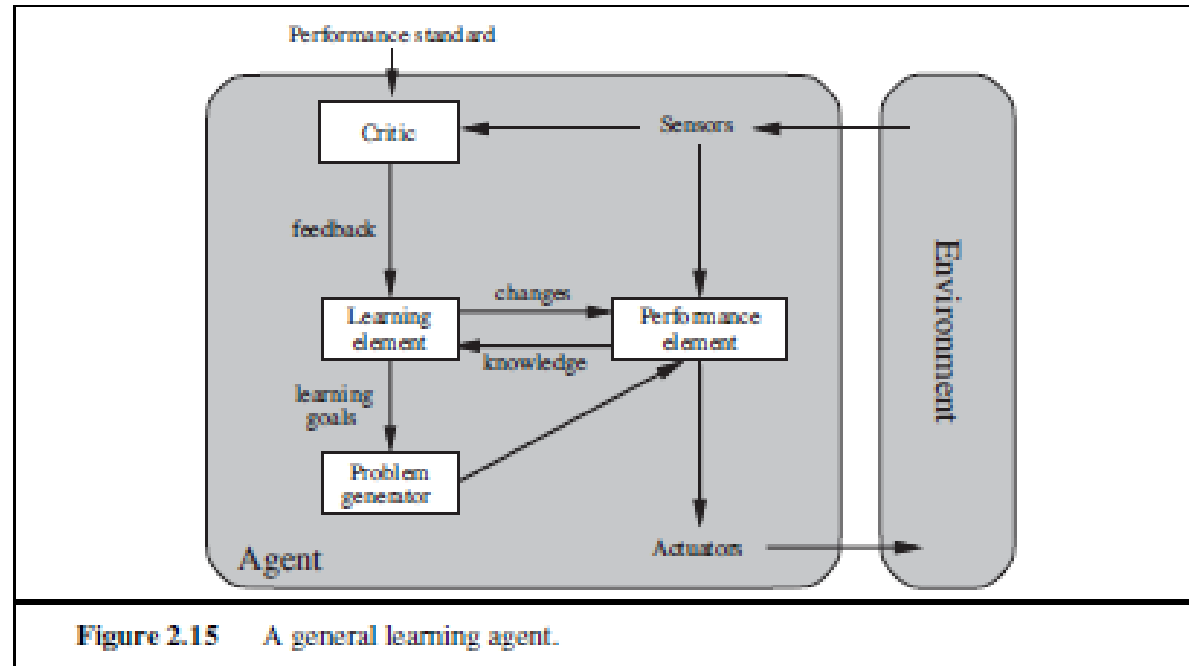
- **Goal-based agents:** Knowing something about the current state of the environment is not always enough to decide what to do.
- For example, at a road junction, the taxi can turn left, turn right, or go straight on. The correct decision depends on where the taxi is trying to get to. In other words, as well as a current state description, the agent needs some sort of **goal** information that describes situations that are desirable.
- Sometimes goal-based action selection is straightforward or sometimes tricky (when the agent has to consider long sequences of twists and turns in order to find a way to achieve the goal)



- **Utility-based agents:** Goals alone are not enough to generate high-quality behavior in most environments. For example, many action sequences will get the taxi to its destination (thereby achieving the goal) but some are quicker, safer, more reliable, or cheaper than others.
- more general performance measure should allow a comparison of different world states according to exactly how happy they would make the agent. Because “**happy**” does not sound very scientific, economists and computer scientists use the term **utility** instead.



Learning agents:



- Turing in 1950 proposes building learning machines and then to teach them. A learning agent can be divided into four conceptual components:
- **learning element:** responsible for making improvement
- **performance element:** responsible for selecting external actions.
- **Critic:** The learning element uses feedback from the **critic** on how the agent is doing and determines how the performance element should be modified to do better in the future.

- **problem generator:** responsible for suggesting actions that will lead to new and informative experiences.
- Ex: Automated Taxi: The performance element consists of whatever collection of knowledge and procedures the taxi has for selecting its driving actions.
- The taxi goes out on the road and drives, using this performance element
- The critic observes the world and passes information along to the learning element. For example, after the taxi makes a quick left turn across three lanes of traffic, the critic observes and learning element is able to formulate a rule saying this was a bad action, and the performance element is modified by installation of the new rule
- The problem generator might identify certain areas of behavior in need of improvement and suggest experiments

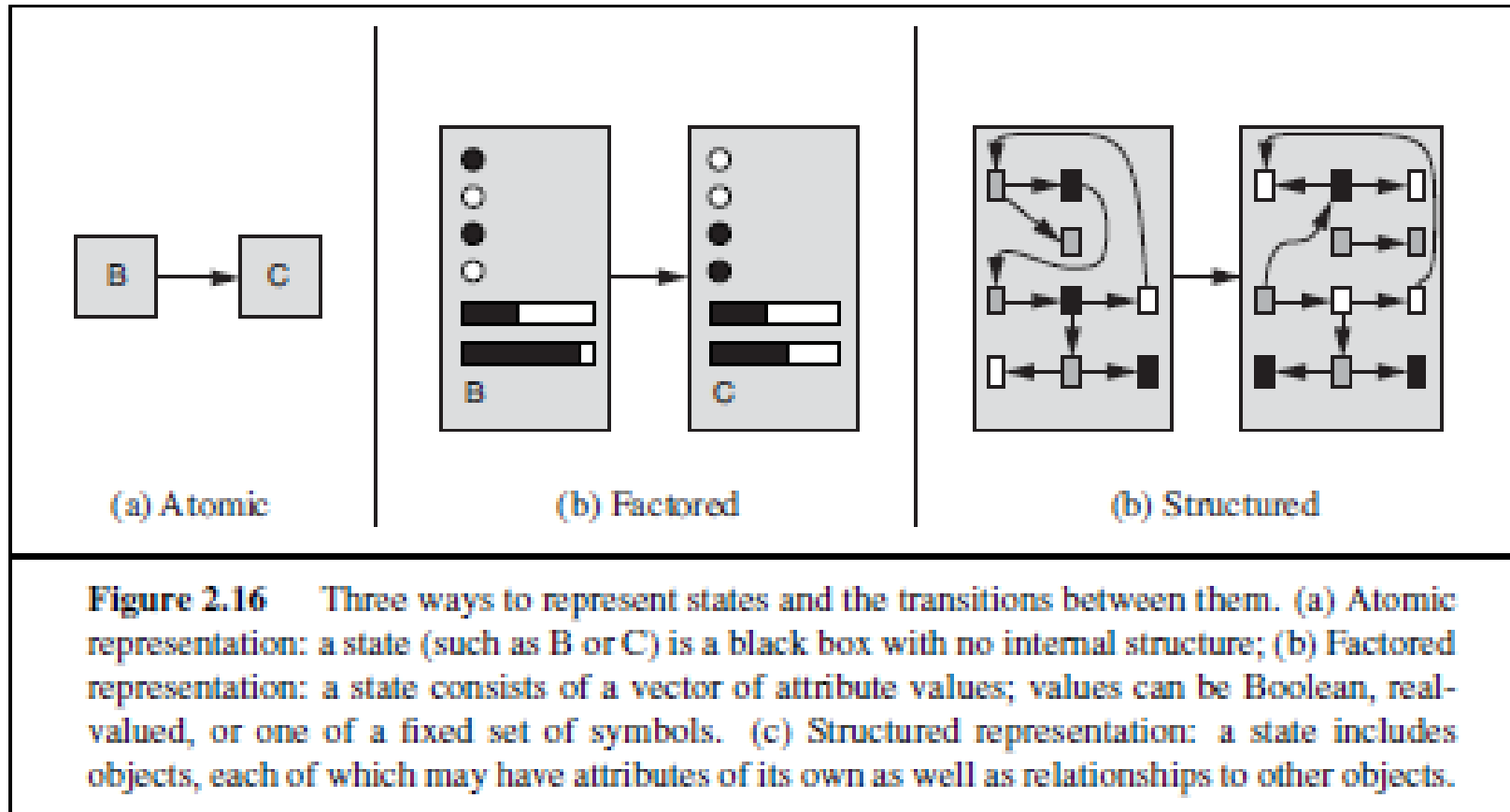
How the components of agent programs work:

3 representations along an axis of increasing complexity and expressive power—

atomic

factored, and

structured.



- In an **atomic representation** each state of the world is indivisible—it has no internal structure.

Ex: problem of finding a driving route from one end of a country to the other via some sequence of cities

- **factored representation** splits up each state into a fixed set of **variables** or **attributes**, each of which can have a **value**.

Ex: our current GPS coordinates

- **Structured representation:** For example, we might notice that a large truck ahead of us is reversing into the driveway of a dairy farm but a cow is blocking the truck's path.
- A factored representation is unlikely to be pre-equipped with the attribute
- Here objects such as cows and trucks and their various and varying relationships can be described explicitly.

Unit 1-chapter 3: Problem solving by searching

- We are discussing a kind of goal-based agent called a **problem-solving agent**.

Problem Solving agents: Intelligent agents are supposed to maximize their performance measure. As we mentioned, achieving this is sometimes simplified if the agent can adopt a **goal** and aim at satisfying it.

- **Goal formulation**, based on the current situation and the agent's performance measure, is the first step in problem solving.
- **Problem formulation** is the process of deciding what actions and states to consider, given a goal.
- **Known knowledge**

Ex: Imagine an agent in the city of **Arad, Romania**, enjoying a touring holiday. The agent's performance measure contains many factors: it wants to improve its insights, enjoy the nightlife (such as it is), avoid hangovers, and so on.

- Now, suppose the agent has a nonrefundable ticket to fly out to **Bucharest** the following day. Three roads lead out of Arad, one toward Sibiu, one to Timisoara, and one to Zerind. None of these achieves the goal, so unless the agent is familiar with the geography of Romania, it will not know which road to follow.

Well-defined problems and solutions:

A **problem** can be defined formally by five components:

- The **initial state** that the agent starts in. For example, the initial state for our agent in Romania might be described as $\text{In}(\text{Arad})$.
- A description of the possible **actions** available to the agent. Given a particular state s , returns the set of actions that can be executed in s . We say that each of these actions is applicable in s . For example, from the state $\text{In}(\text{Arad})$, the applicable actions are $\{\text{Go}(\text{Sibiu}), \text{Go}(\text{Timisoara}), \text{Go}(\text{Zerind})\}$
- description of what each action does; the formal name for this is the **transition model**, specified by a function $\text{RESULT}(s, a)$ that returns the state that results from action a in state s .

Together, the initial state, actions, and transition model implicitly define the **state space** of the problem—the set of all states reachable from the initial state by any sequence of actions.

The state space forms a directed network or **graph** in which the nodes are states and the links between nodes are actions. A **path** in the state space is a sequence of states connected by a sequence of actions.

- The **goal test**, which determines whether a given state is a goal state agent chooses a cost function that reflects its own performance measure. For the agent trying to get to Bucharest, time is of the essence, so the cost of a path might be its length in kilometers.
- A **path cost** function that assigns a numeric cost to each path.

Formulating problems

- In the preceding section we proposed a formulation of the problem of getting to Bucharest in terms of the initial state, actions, transition model, goal test, and path cost. This formulation seems reasonable, but it is still a *model*—an abstract mathematical description

- **States:** A state description specifies the location of each of the eight tiles and the blank in one of the nine squares.
- **Initial state:** Any state can be designated as the initial state. Note that any given goal can be reached from exactly half of the possible initial states
- **Actions:** The simplest formulation defines the actions as movements of the blank space *Left*, *Right*, *Up*, or *Down*. Different subsets of these are possible depending on where the blank is.
- **Transition model:** Given a state and action, this returns the resulting state; for example, if we apply *Left* to the start state the resulting state has the 5 and the blank switched.
- **Goal test:** This checks whether the state matches the goal configuration shown

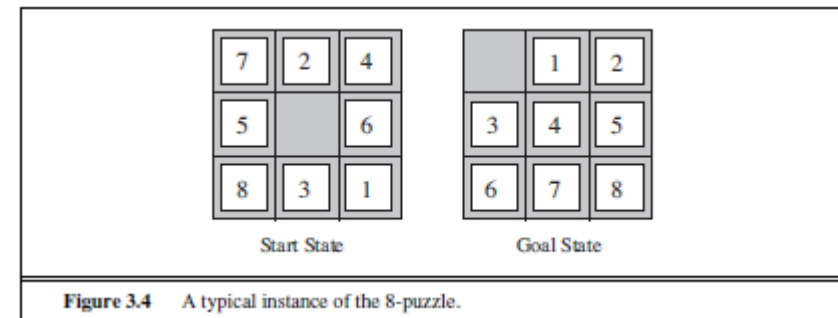
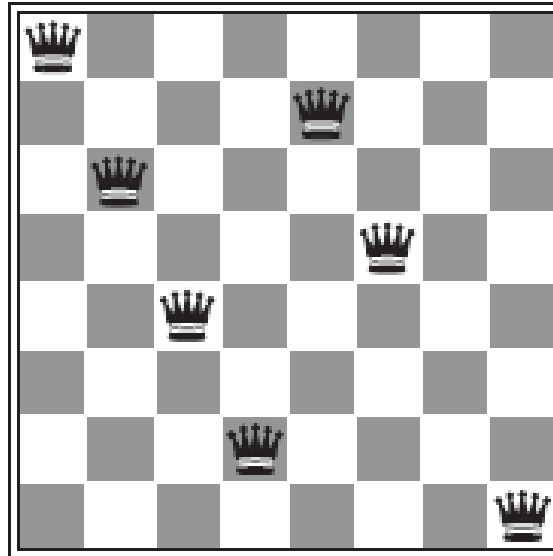


Figure 3.4 A typical instance of the 8-puzzle.

8-Queens problem:

- **States:** Any arrangement of 0 to 8 queens on the board is a state.
- **Initial state:** No queens on the board.
- **Actions:** Add a queen to any empty square.
- **Transition model:** Returns the board with a queen added to the specified square.
- **Goal test:** 8 queens are on the board, none attacked.



There are two types of searching techniques :

1. **Uninformed Search**

2. **Informed Search**

Uninformed: These algorithms do not know anything about what they are searching for and where they should search for it. Uninformed searching takes a lot of time to search as it doesn't know where to head and where the best chances of finding the element are.

Ex: Depth-First Search, & Breadth-First Search, Bidirectional search.

Informed: The algorithm is aware of where the best chances of finding the element are and the algorithm heads that way! **Heuristic** search is an informed search technique.

A heuristic value tells the algorithm which path will provide the solution as early as possible. The heuristic function is used to generate this heuristic value. Different heuristic functions can be designed depending on the search problem. So that **Heuristic search is a technique that uses a heuristic value for optimizing the search.**

8 Puzzle problem:

1	2	3
	4	6
7	5	8

1	2	3
4	5	6
7	8	

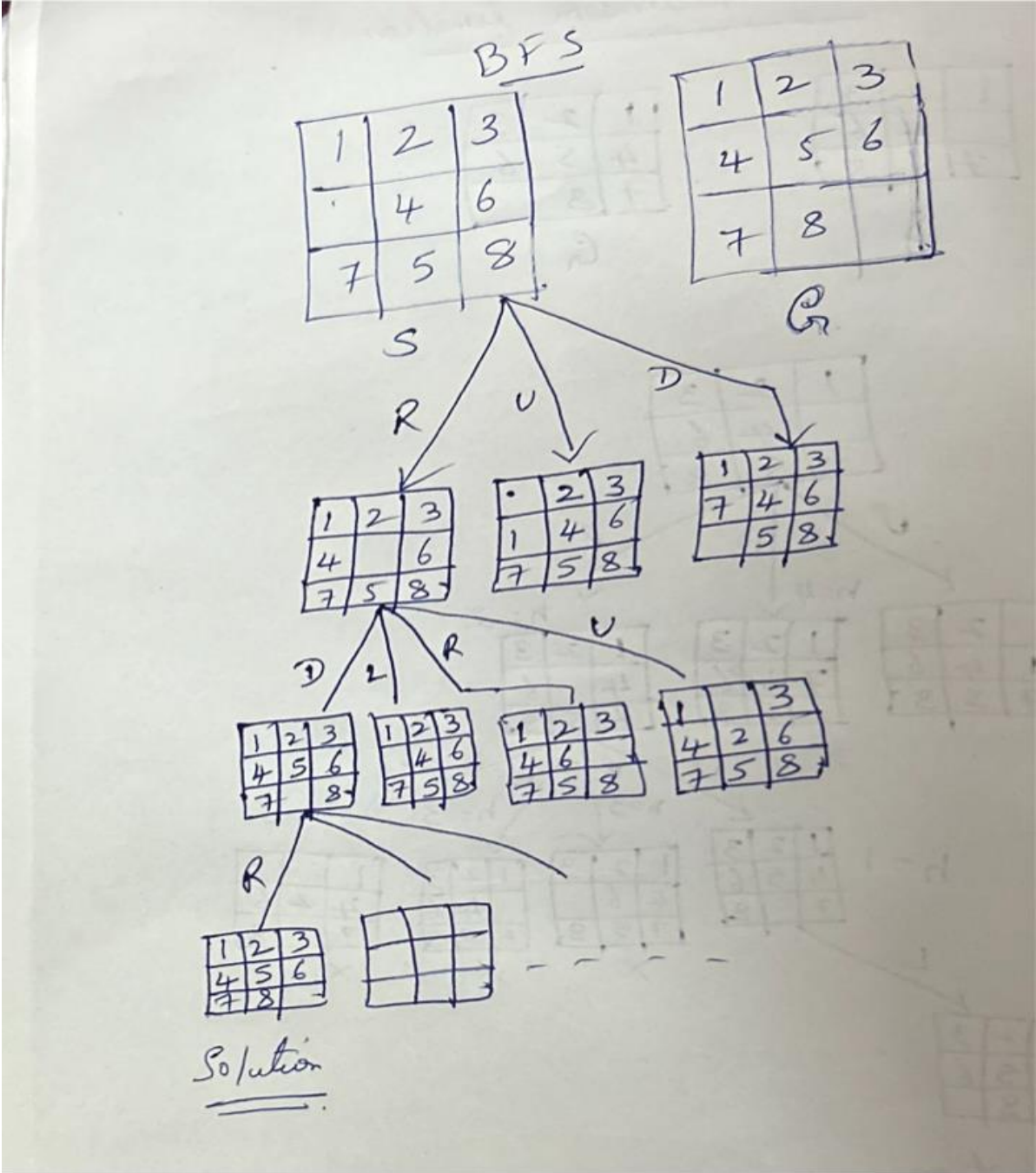
N-Puzzle or sliding puzzle is a popular puzzle that consists of N tiles where N can be 8, 15, 24, and so on. In this example N = 8. The puzzle is divided into $\sqrt{N+1}$ rows and $\sqrt{N+1}$ columns.

The puzzle consists of N tiles and one empty space where the tiles can be moved. Start and Goal configurations (also called state) of the puzzle are provided. The puzzle can be solved by moving the tiles one by one in a single empty space and thus achieving the Goal configuration.

Rules for solving the puzzle.

- Instead of moving the tiles in the empty space, we can visualize moving the empty space in place of the tile, basically swapping the tile with the empty space. The empty space can only move in four directions viz.,
 1. Up
 2. Down
 3. Right or
 4. Left

8 puzzle problem without Heuristics function:



Heuristics:

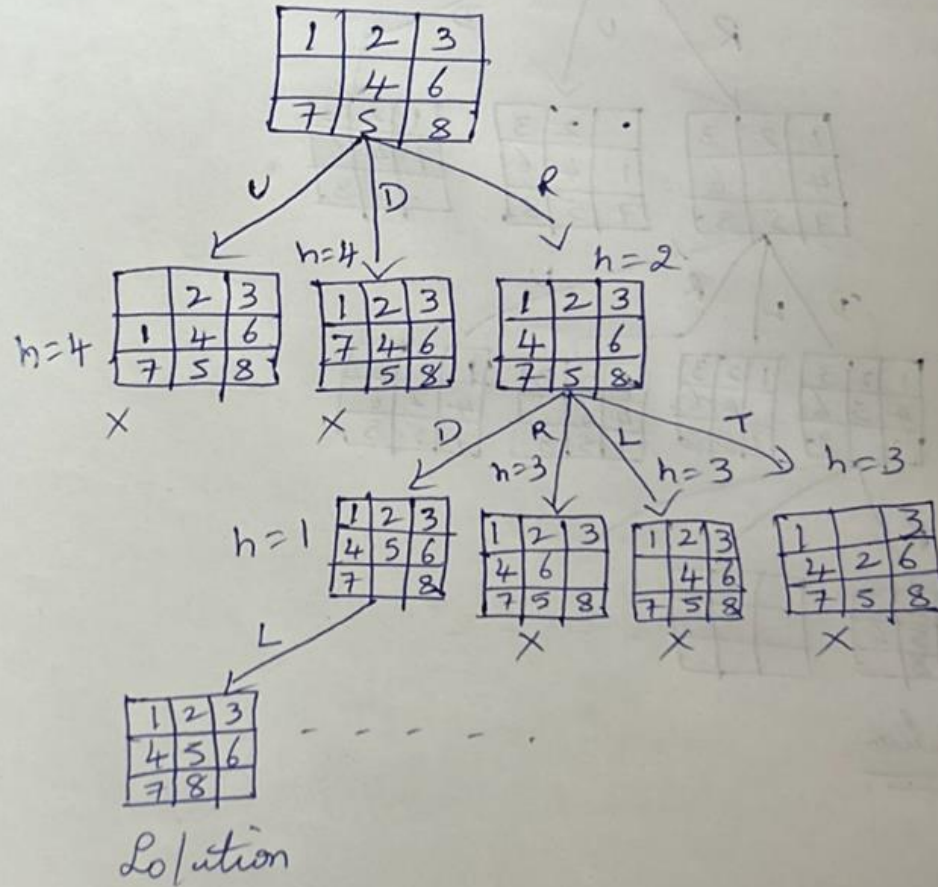
Using heuristic function

1	2	3
	4	6
7	5	8

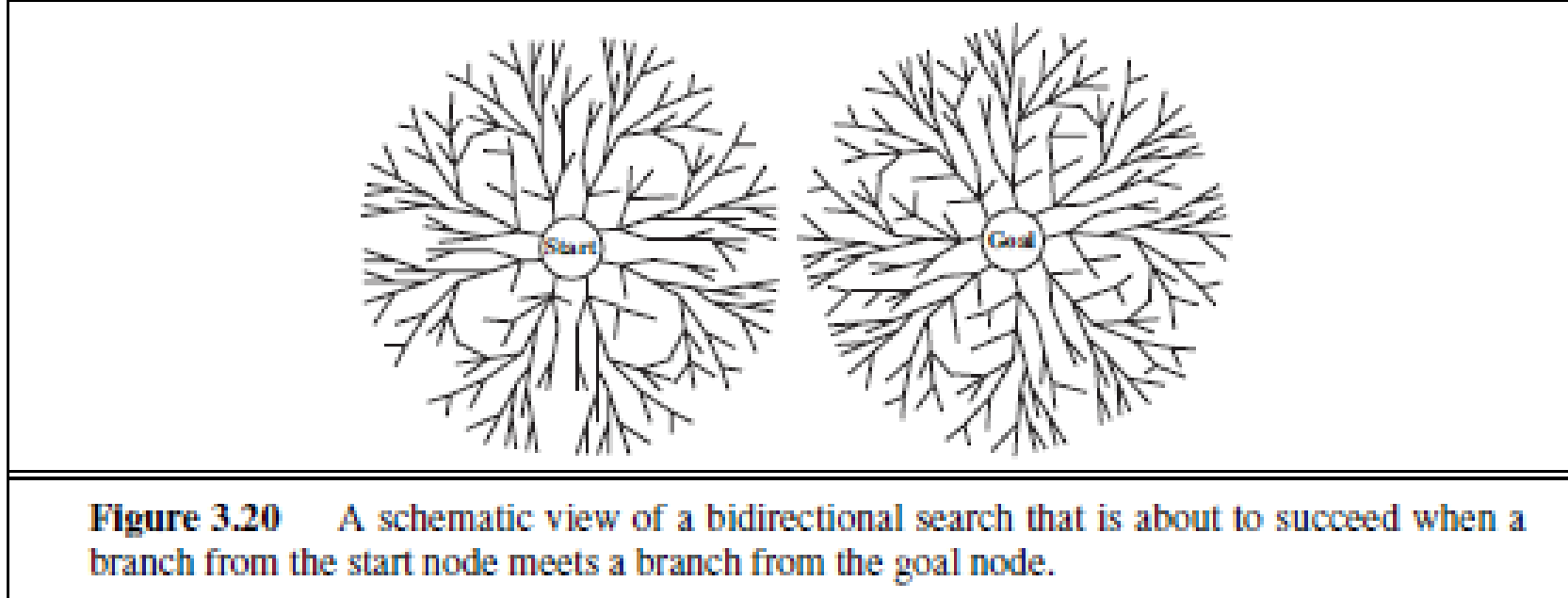
S

1	2	3
4	5	6
7	8	

G



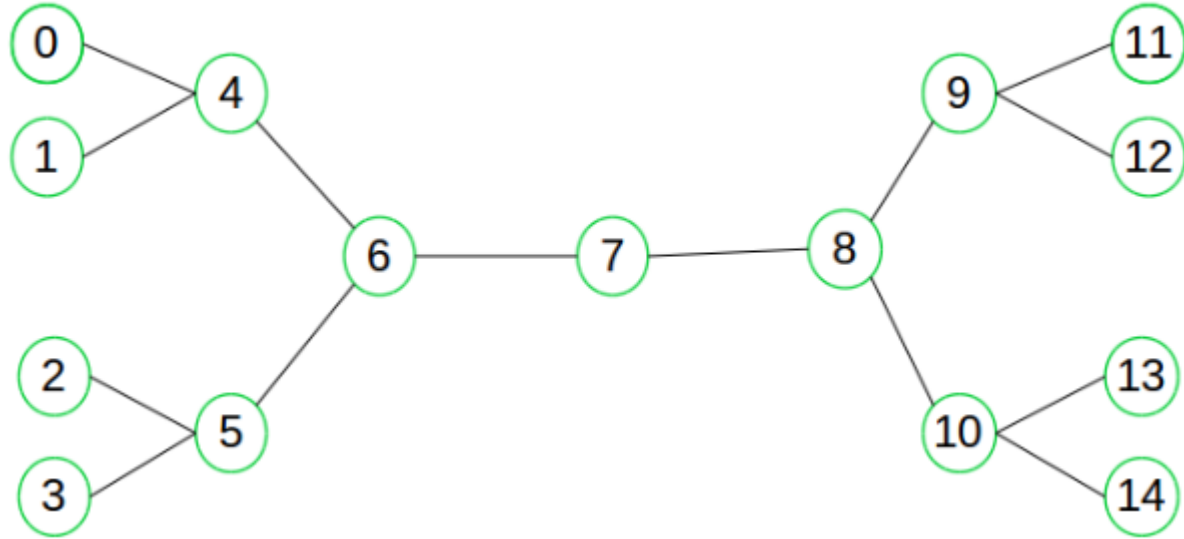
- **Bidirectional search:** It is an uninformed search. The idea behind bidirectional search is to run two simultaneous searches—one forward from the initial state and the other backward from the goal—hoping that the two searches meet in the middle.
- The motivation is that $b^{d/2} + b^{d/2}$ is much less than b^d
- Bidirectional search is implemented by replacing the goal test with a check to see whether the frontiers of the two searches intersect; if they do, a solution has been found.



It runs two simultaneous search –

- Forward search from source/initial vertex toward goal vertex
- Backward search from goal/target vertex toward source vertex

Ex:



When to use bidirectional approach?

- Both initial and goal states are unique and completely defined.
- The branching factor is exactly the same in both directions.

Informed Search: A* search: Minimizing the total estimated solution cost

- The most widely known form of best-first search is called **A* A search** (pronounced “A-star search”). It evaluates nodes by combining $g(n)$, the cost to reach the node, and $h(n)$, the cost to get from the node to the goal:

$$f(n) = g(n) + h(n) .$$

$g(n)$ = path cost from the start node to node n , and

$h(n)$ = is the estimated cost of the cheapest path from n to the goal,

we have $f(n)$ = estimated cost of the cheapest solution through n .

- Thus, if we are trying to find the cheapest solution, a reasonable thing to try first is the node with the lowest value of $g(n) + h(n)$. A* search is both complete and optimal.

Ex:

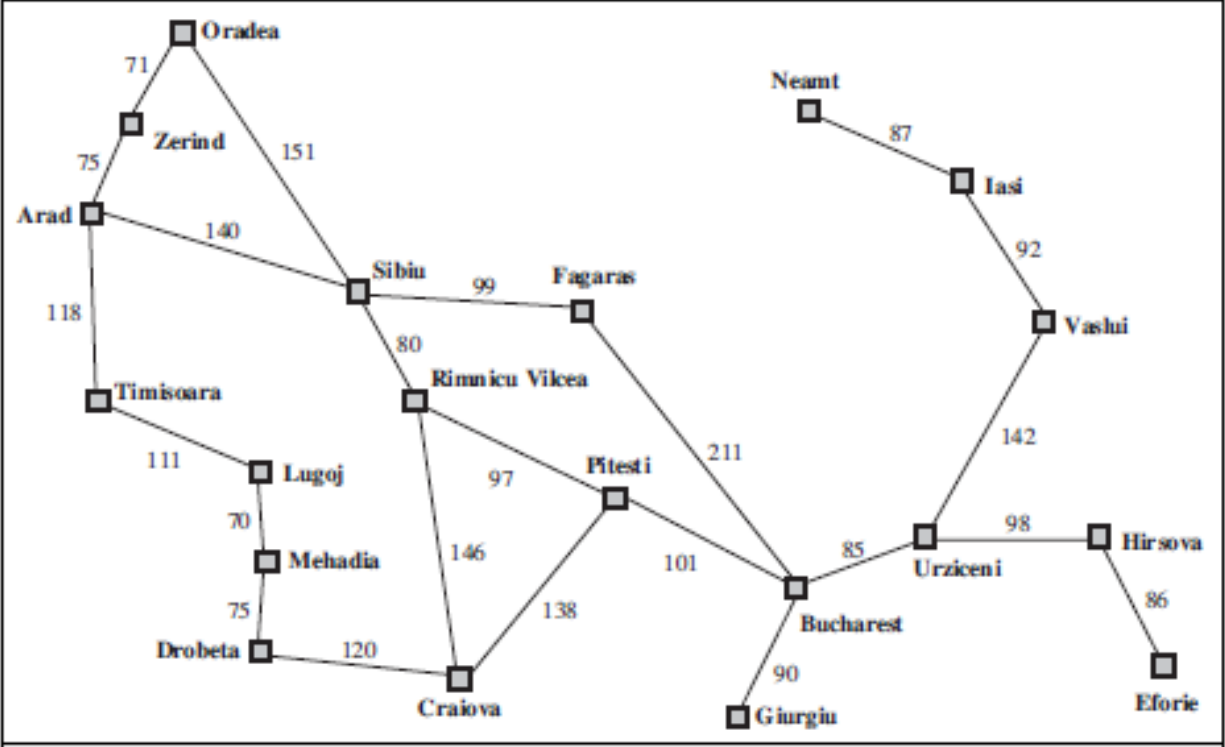


Figure 3.2 A simplified road map of part of Romania.

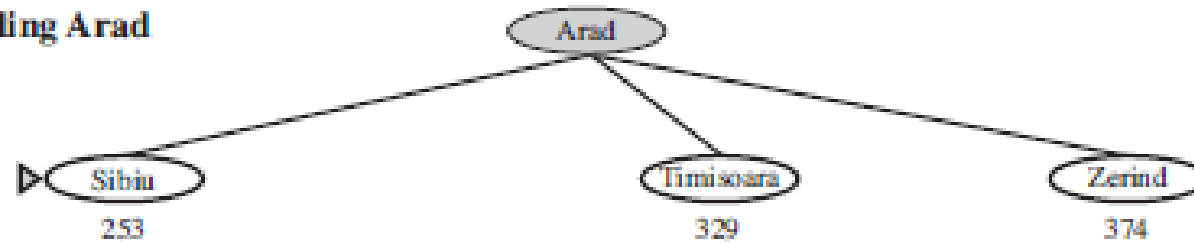
Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Figure 3.22 Values of h_{SLD} —straight-line distances to Bucharest.

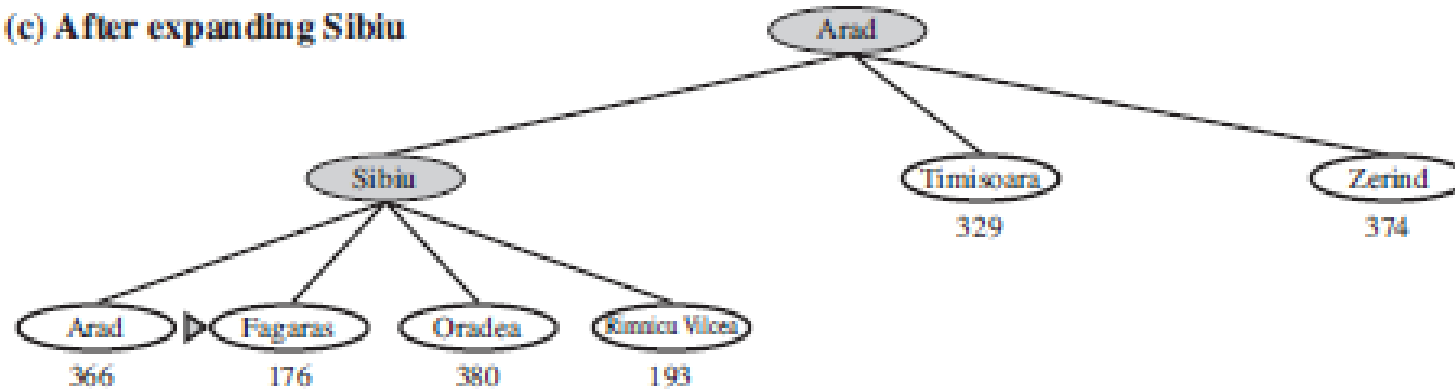
(a) The initial state



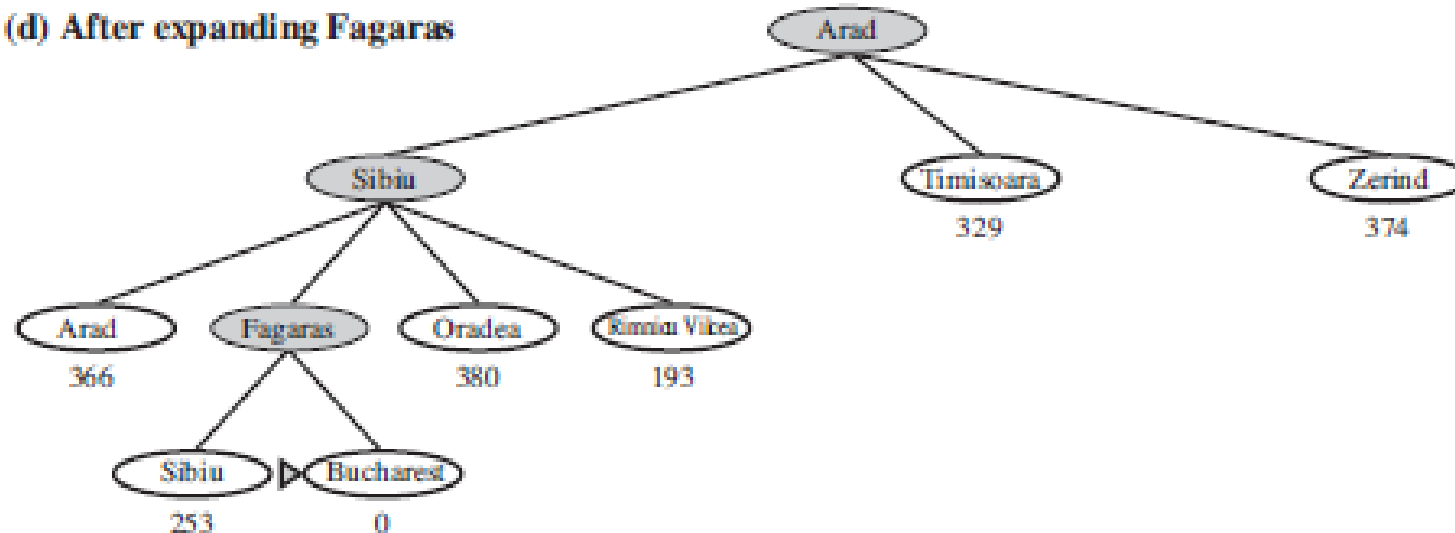
(b) After expanding Arad



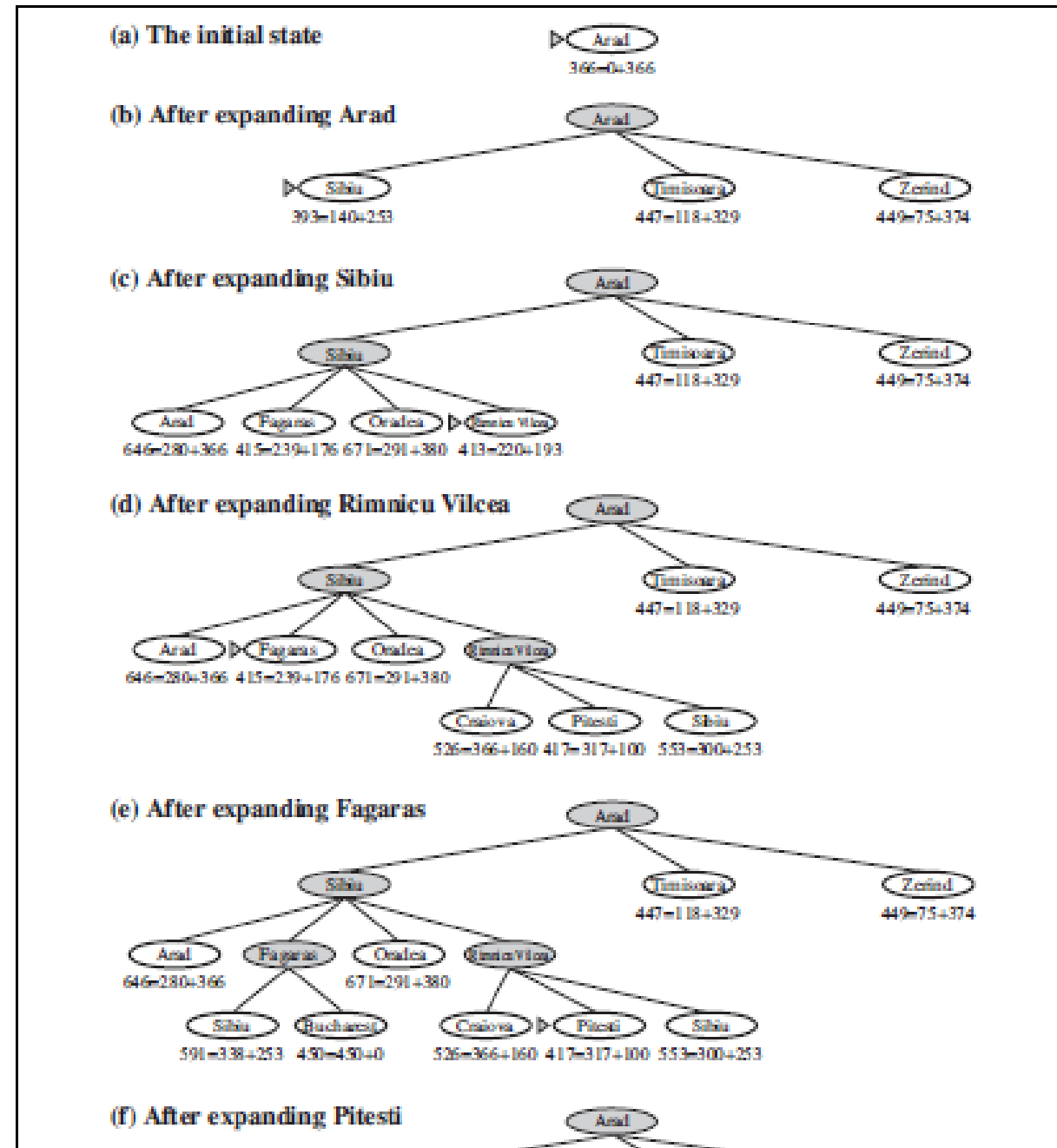
(c) After expanding Sibiu



(d) After expanding Fagaras



- The values of g are computed from the step costs in Figure 3.2, and the values of h_{SLD} are given in Figure 3.22



Conditions for optimality for A*: Admissibility and consistency:

- The first condition we require for optimality is that $h(n)$ be an **admissible heuristic**. An admissible heuristic is one that *never overestimates* the cost to reach the goal.
- Admissible heuristics are by nature optimistic because they think the cost of solving the problem is less than it actually is.

A second, slightly stronger condition called **consistency** (or sometimes **monotonicity**) is required only for applications of A* to graph search.

A heuristic $h(n)$ is consistent if, for every node n and every successor n' of n generated by any action a , the estimated cost of reaching the goal from n' is no greater than the step cost of getting to n' plus the estimated cost of reaching the goal from n .

- $h(n) \leq c(n, a, n') + h(n')$.

- A* has the following properties: *the tree-search version of A* is optimal if $h(n)$ is admissible, while the graph-search version is optimal if $h(n)$ is consistent.*
- That A* search is complete, optimal, and optimally efficient among all such algorithms is rather satisfying. Unfortunately, it does not mean that A* is the answer to all our searching needs. The catch is that, for most problems, the number of states within the goal contour search space is still exponential in the length of the solution.

Exercise:

Apply A* algorithm for solving 8 puzzle problem.

$G(n)$ =depth of the node

$H(n)$ = no of misplaced tiles

Informed Search	Uninformed Search
Use of Heuristics	No Heuristics
More efficient	Inefficient in complex problems
Needs calculation of Heuristic value	Simple to implement
Goal Oriented	Blind search

What is generative AI?

benefits and applications of generative AI

risks of generative AI

practical uses of generative AI today

LLMs

UNIT II: ADVERSARIAL SEARCH

5.1 GAMES:

- **Multiagent** environments: in which each agent needs to consider the actions of other agents and how they affect its own welfare.
- **Competitive** environments: in which the agents goals are in conflict, giving rise to **adversarial search** problems—often known as **games**.
- In AI, the most common games are of a rather specialized kind—what game theorists call deterministic, turn-taking, two-player, **zero-sum games** of **perfect information** (such as chess).
- For example, chess has an average branching factor of about 35, and games often go to 50 moves by each player, so the search tree has about 35^{100} or 10^{154} nodes.

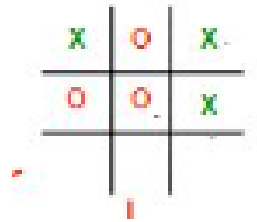
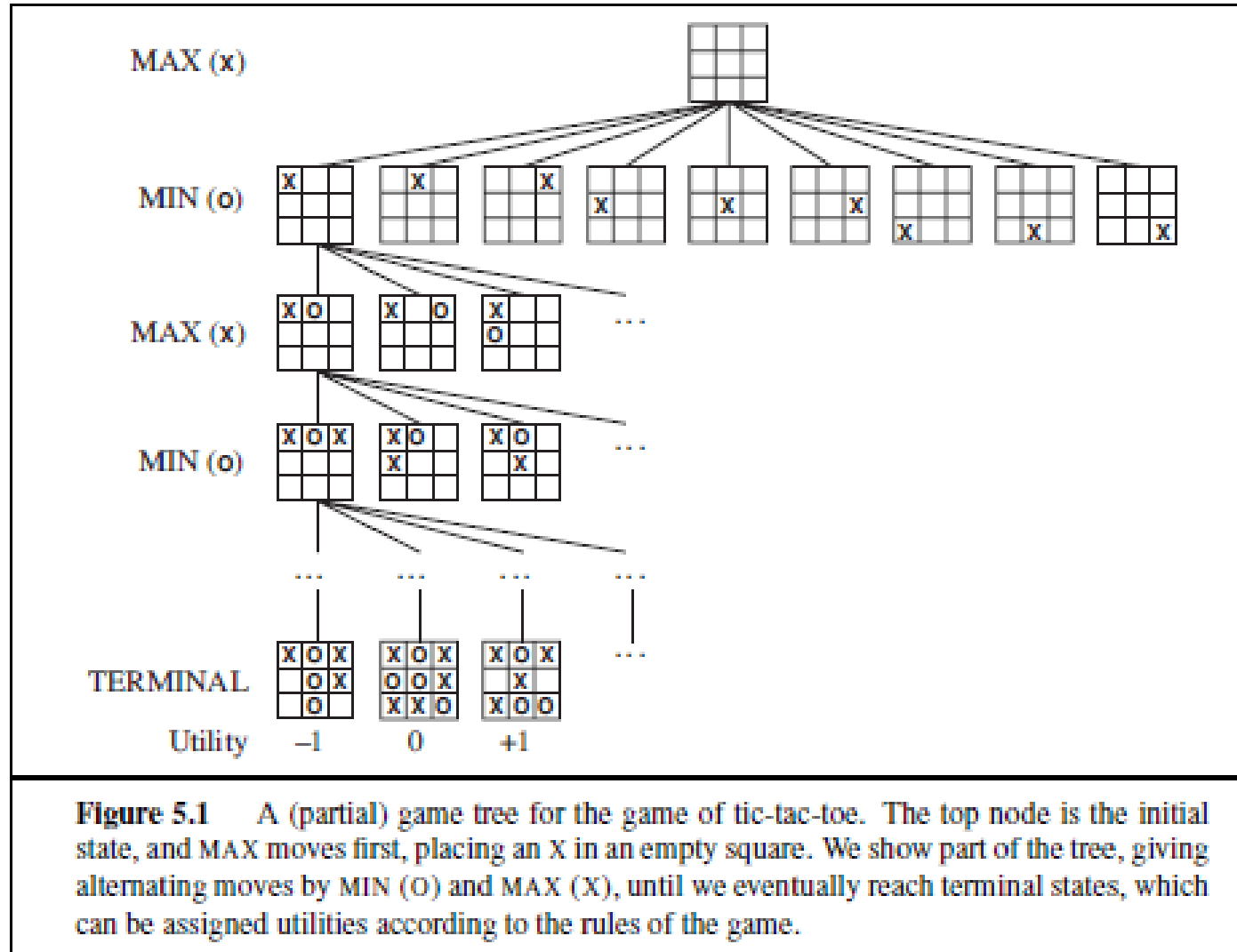
A game can be formally defined as a kind of search problem with the following elements:

- **S0**: The **initial state**, which specifies how the game is set up at the start.
- **PLAYER(s)**: Defines which player has the move in a state.
- **ACTIONS(s)**: Returns the set of legal moves in a state.
- **RESULT(s, a)**: The **transition model**, which defines the result of a move.
- **TERMINAL-TEST(s)**: A **terminal test**, which is true when the game is over and false otherwise. States where the game has ended are called **terminal states**.
- **UTILITY(s, p)**: A **utility function** (also called an objective function or payoff function). It defines the final numeric value for a game that ends in terminal state s for a player p .

In chess, the outcome is a win, loss, or draw, with values $+1$, 0 , or $1/2$.

- The initial state, ACTIONS function, and RESULT function define the **game tree** for the game—a tree where the nodes are game states and the edges are moves.

Ex:



- For tic-tac-toe the game tree is relatively small—fewer than $9! = 362,880$ terminal nodes.

5.2 OPTIMAL DECISIONS IN GAMES

- The optimal solution would be a sequence of actions leading to a goal state—a terminal state that is a win.
- The possible moves for MAX at the root node are labeled a_1 , a_2 , and a_3 . The possible replies to a_1 for MIN are b_1 , b_2 , b_3 , and so on. This particular game ends after one move each by MAX and MIN.

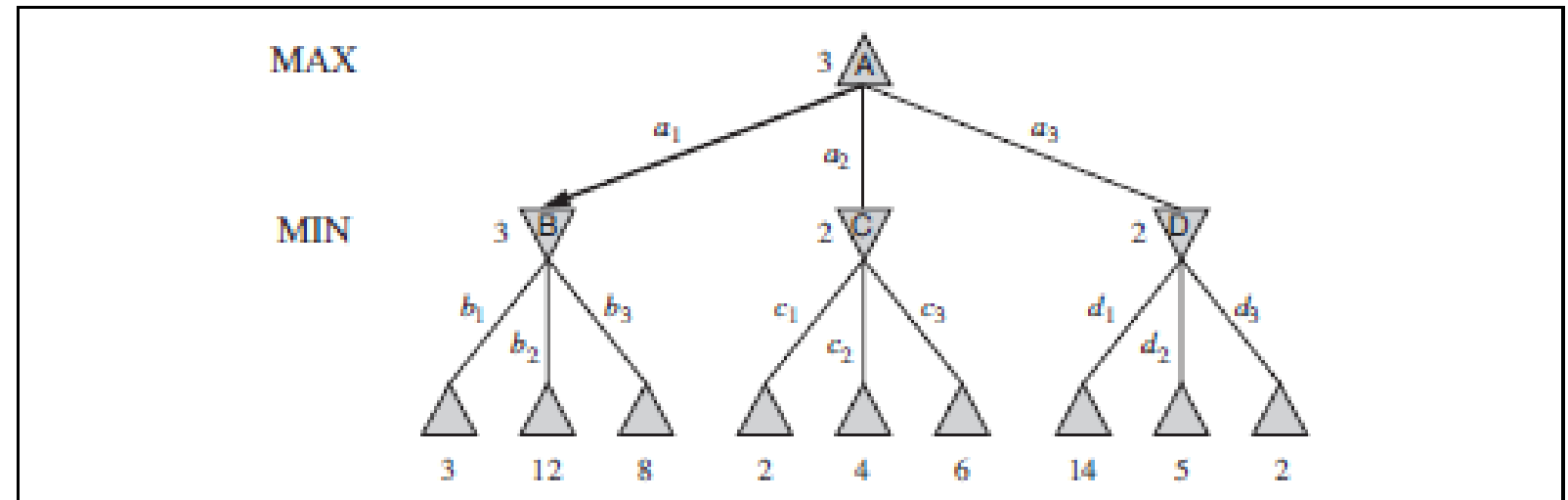


Figure 5.2 A two-ply game tree. The $\triangle$ nodes are “MAX nodes,” in which it is MAX’s turn to move, and the ∇ nodes are “MIN nodes.” The terminal nodes show the utility values for MAX; the other nodes are labeled with their minimax values. MAX’s best move at the root

- Given a game tree, the optimal strategy can be determined from the **minimax value** of each node, which we write as MINIMAX(n).
- The minimax value of a node is the utility (for MAX) of being in the corresponding state, *assuming that both players play optimally* from there to the end of the game. Obviously, the minimax value of a terminal state is just its utility.
- Furthermore, given a choice, MAX prefers to move to a state of maximum value, whereas MIN prefers a state of minimum value. So we have the following:

$$\text{MINIMAX}(s) = \begin{cases} \text{UTILITY}(s) & \text{if } \text{TERMINAL-TEST}(s) \\ \max_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if } \text{PLAYER}(s) = \text{MAX} \\ \min_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if } \text{PLAYER}(s) = \text{MIN} \end{cases}$$

- Let us apply these definitions to the game tree in Figure 5.2.
- The terminal nodes on the bottom level get their utility values from the game's UTILITY function.

- The first MIN node, labeled B, has three successor states with values 3, 12, and 8, so its minimax value is 3 and so on.

5.2.1 The minimax algorithm

- The **minimax algorithm** computes the minimax decision from the current state.
- It uses a simple recursive computation of the minimax values of each successor state, directly implementing the defining equations.
- The recursion proceeds all the way down to the leaves of the tree, and then the minimax values are **backed up** through the tree as the recursion unwinds.

Ex: Figure 5.2

- The minimax algorithm performs a complete depth-first exploration of the game tree. If the maximum depth of the tree is m and there are b legal moves at each point, then the time complexity of the minimax algorithm is $O(b^m)$.

Ex: Minimax algo:

```
function MINIMAX-DECISION(state) returns an action  
  return  $\arg \max_{a \in \text{ACTIONS}(s)} \text{MIN-VALUE}(\text{RESULT}(\text{state}, a))$ 
```

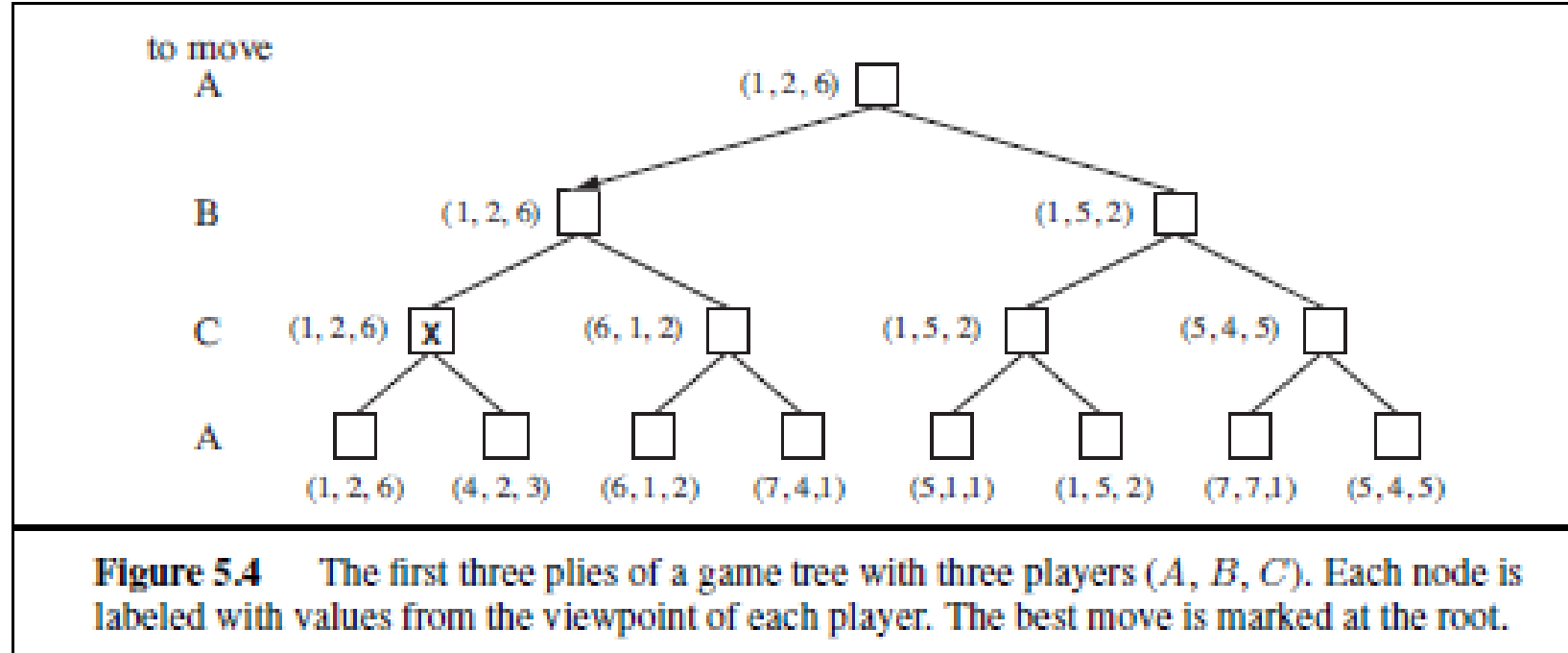
```
function MAX-VALUE(state) returns a utility value  
  if TERMINAL-TEST(state) then return UTILITY(state)  
   $v \leftarrow -\infty$   
  for each a in ACTIONS(state) do  
     $v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(\text{RESULT}(s, a)))$   
  return v
```

```
function MIN-VALUE(state) returns a utility value  
  if TERMINAL-TEST(state) then return UTILITY(state)  
   $v \leftarrow \infty$   
  for each a in ACTIONS(state) do  
     $v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(\text{RESULT}(s, a)))$   
  return v
```

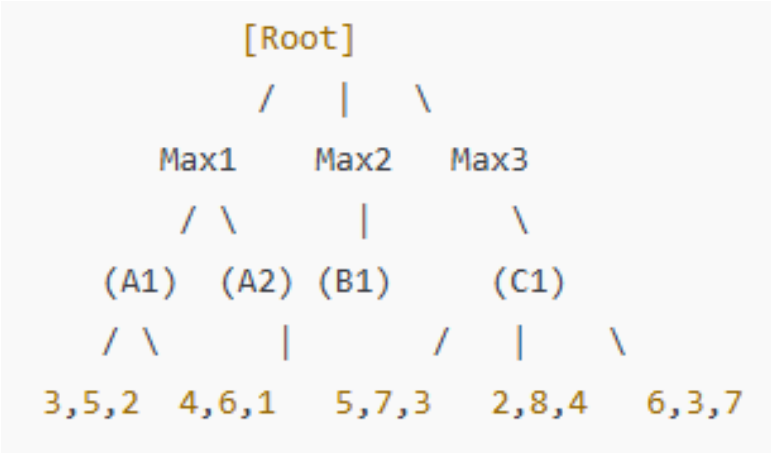
Figure 5.3 An algorithm for calculating minimax decisions. It returns the action corresponding to the best possible move, that is, the move that leads to the outcome with the best utility, under the assumption that the opponent plays to minimize utility. The functions MAX-VALUE and MIN-VALUE go through the whole game tree, all the way to the leaves, to determine the backed-up value of a state. The notation $\arg \max_{a \in S} f(a)$ computes the element *a* of set *S* that has the maximum value of *f(a)*.

Optimal decisions in multiplayer game:

- Let us examine how to extend the minimax idea to multiplayer games.
- First, we need to replace the single value for each node with a *vector* of values for example, in a three-player game with players A, B, and C, a vector v_A, v_B, v_C is associated with each node.
- For terminal states, this vector gives the utility of the state from each player's viewpoint. The simplest way to implement this is to have the UTILITY function return a vector of utilities.



Here is a simplified example tree for a three-player game, with each node representing a game state, and leaf nodes representing scores for each player:



Root: Initial state.

Levels:

- Level 1: Player 1's (Max1) choices.
- Level 2: Player 2's (Max2) choices based on Player 1's action.
- Level 3: Player 3's (Max3) choices based on Players 1 and 2's actions.

Each terminal leaf node represents the scores of each player in that particular end-state: (Player 1 score, Player 2 score, Player 3 score).

Step-by-Step Minimax Evaluation

1. Level 3 (Player 3 Choices):

- **Nodes:**
 - **A1** branches to scores: (3,5,2) and (4,6,1).
 - Player 1 would prefer **4** as it maximizes their score, so **(4,6,1)** is chosen.
 - **B1** has only one branch with a score: (5,7,3).
 - **C1** branches to scores: (2,8,4) and (6,3,7).
 - Player 3 would prefer **7** to maximize their score, so **(6,3,7)** is chosen.

2. Level 2 (Player 2 Choices):

- **A2** has the choice between branches **A1** and **B1**:
 - Player 2 would choose **A1** as **6** is the maximum score Player 2 can get from the options available, so **(4,6,1)** is chosen.
- **C1** stands alone for Player 2 and thus **(6,3,7)** is kept.

3. Level 1 (Player 1 Choices):

- Player 1, comparing **A1** (4,6,1) and **C1** (6,3,7), would prefer **C1** because **6** is the highest score achievable among the options.

Result

The Minimax optimal path for Player 1 is to choose **C1** with the score (6,3,7), as this path maximizes their score after considering the best responses from Players 2 and 3.

- Multiplayer games usually involve **alliances**, whether formal or informal, among the players. Alliances are made and broken as the game proceeds.

ALPHA–BETA PRUNING:

- The problem with minimax search is that the number of game states it has to examine is exponential in the depth of the tree.
- In this method, we can't eliminate the exponent, but it turns out we can effectively cut it in half.
- **alpha–beta pruning**- When applied to a standard minimax tree, it returns the same move as minimax would, but prunes away branches that cannot possibly influence the final decision.
- Consider again the two-ply game tree from Figure 5.2. Let's go through the calculation of the optimal decision once more.
- The outcome is that we can identify the minimax decision without ever evaluating two of the leaf nodes

- Let the two unevaluated successors of node C in Figure 5.5 have values x and y . Then the value of the root node is given by

$$\begin{aligned}\text{MINIMAX}(\text{root}) &= \max(\min(3, 12, 8), \min(2, x, y), \min(14, 5, 2)) \\ &= \max(3, \min(2, x, y), 2) \\ &= \max(3, z, 2) \text{ where } z = \min(2, x, y) \leq 2 \\ &= 3.\end{aligned}$$

- Alpha–beta pruning can be applied to trees of any depth, and it is often possible to prune entire subtrees rather than just leaves.
- **General principle:** consider a node n somewhere in the tree, such that Player has a choice of moving to that node. If Player has a better choice m either at the parent node of n or at any choice point further up, then n *will never be reached in actual play*.
- So once we have found out enough about n (by examining some of its descendants) to reach this conclusion, we can prune it.

Ex:

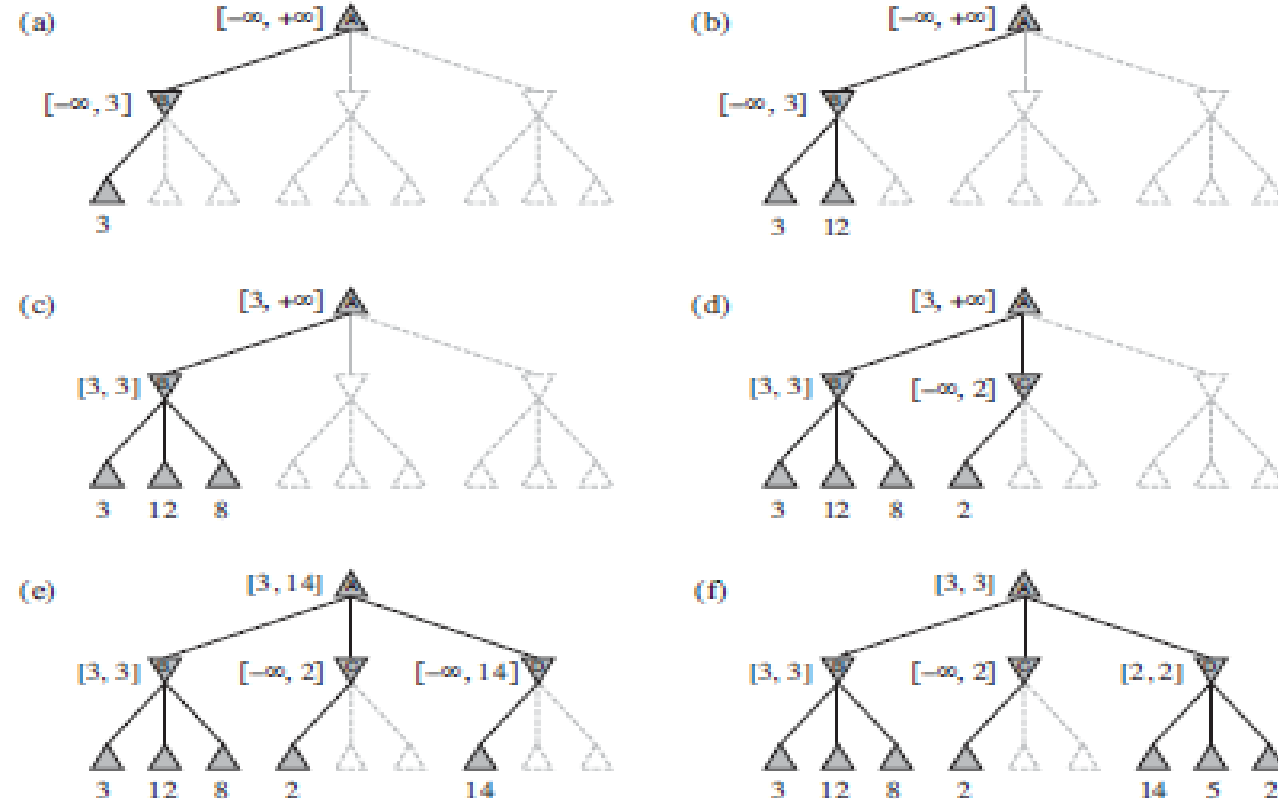


Figure 5.5 Stages in the calculation of the optimal decision for the game tree in Figure 5.2. At each point, we show the range of possible values for each node. (a) The first leaf below B has the value 3. Hence, B , which is a MIN node, has a value of *at most* 3. (b) The second leaf below B has a value of 12; MIN would avoid this move, so the value of B is still at most 3. (c) The third leaf below B has a value of 8; we have seen all B 's successor states, so the value of B is exactly 3. Now, we can infer that the value of the root is *at least* 3, because MAX has a choice worth 3 at the root. (d) The first leaf below C has the value 2. Hence, C , which is a MIN node, has a value of *at most* 2. But we know that B is worth 3, so MAX would never choose C . Therefore, there is no point in looking at the other successor states of C . This is an example of alpha–beta pruning. (e) The first leaf below D has the value 14, so D is worth *at most* 14. This is still higher than MAX's best alternative (i.e., 3), so we need to keep exploring D 's successor states. Notice also that we now have bounds on all of the successors of the root, so the root's value is also at most 14. (f) The second successor of D is worth 5, so again we need to keep exploring. The third successor is worth 2, so now D is worth exactly 2. MAX's decision at the root is to move to B , giving a value of 3.

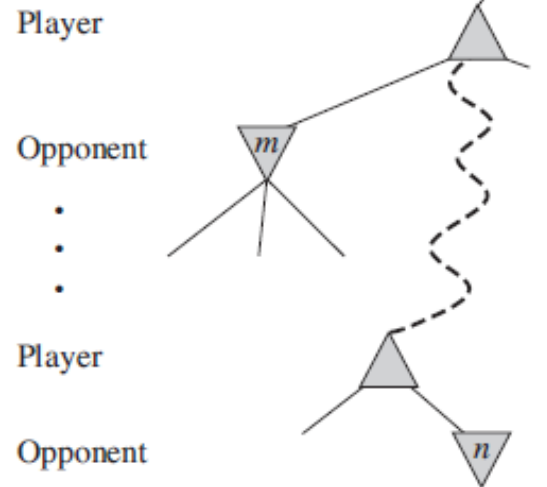


Figure 5.6 The general case for alpha-beta pruning. If m is better than n for Player, we will never get to n in play.

- α = the value of the best (i.e., highest-value) choice we have found so far at any choice point along the path for MAX. The initial value of alpha is $-\infty$.
- β = the value of the best (i.e., lowest-value) choice we have found so far at any choice point along the path for MIN. The initial value of beta is $+\infty$.
- The Alpha-beta pruning to a standard minimax algorithm returns the same move as the standard algorithm does, but it removes all the nodes which are not really affecting the final decision but making algorithm slow. Hence by pruning these nodes, it makes the algorithm fast.

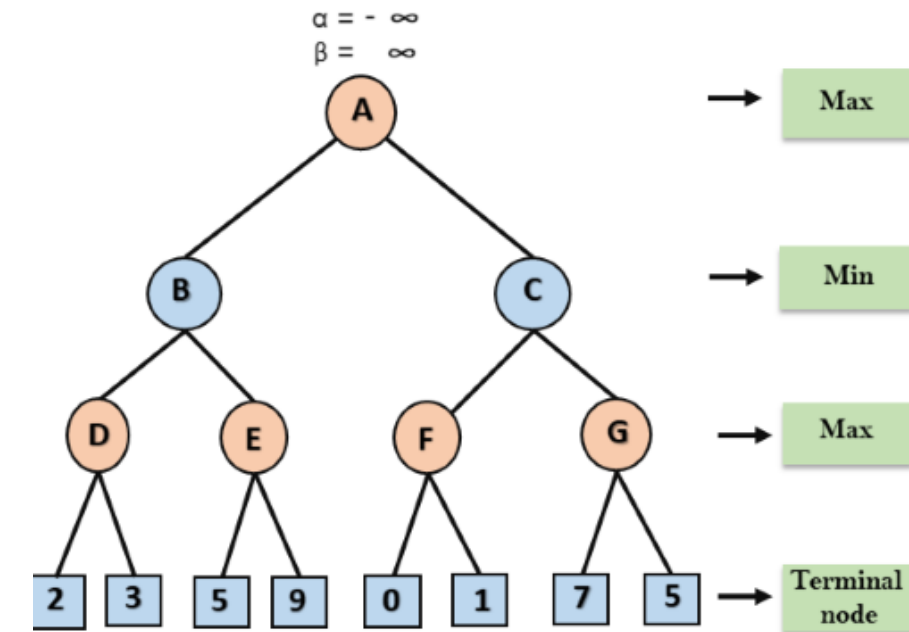
Condition for Alpha-beta pruning:

The main condition which required for alpha-beta pruning is:

$$\alpha \geq \beta$$

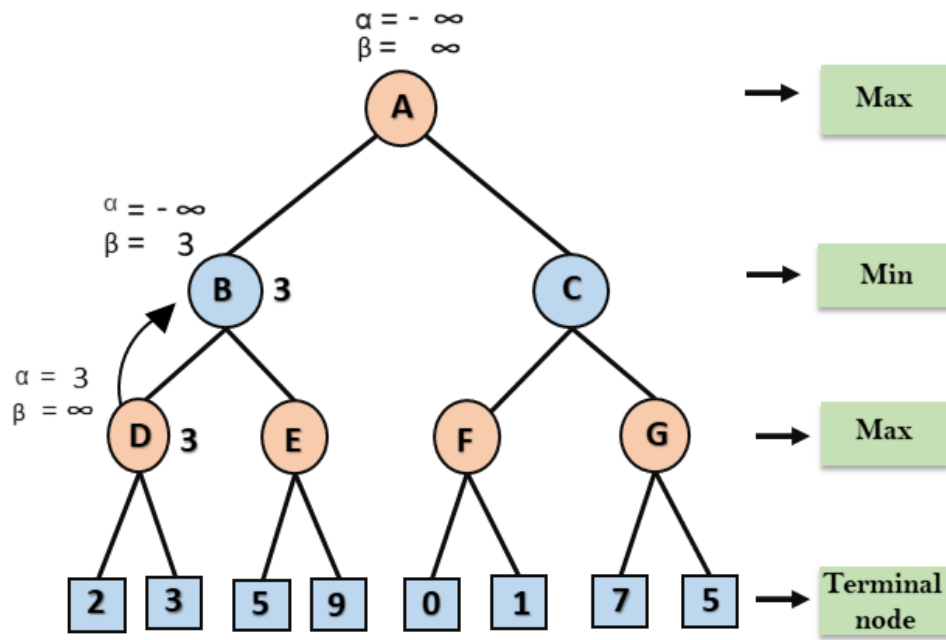
Key points about alpha-beta pruning:

- The Max player will only update the value of alpha.
- The Min player will only update the value of beta.
- While backtracking the tree, the node values will be passed to upper nodes instead of values of alpha and beta.
- We will only pass the alpha, beta values to the child nodes



Step 1:

At the first step the, Max player will start first move from node A where $\alpha = -\infty$ and $\beta = +\infty$, these value of alpha and beta passed down to node B where again $\alpha = -\infty$ and $\beta = +\infty$, and Node B passes the same value to its child D.

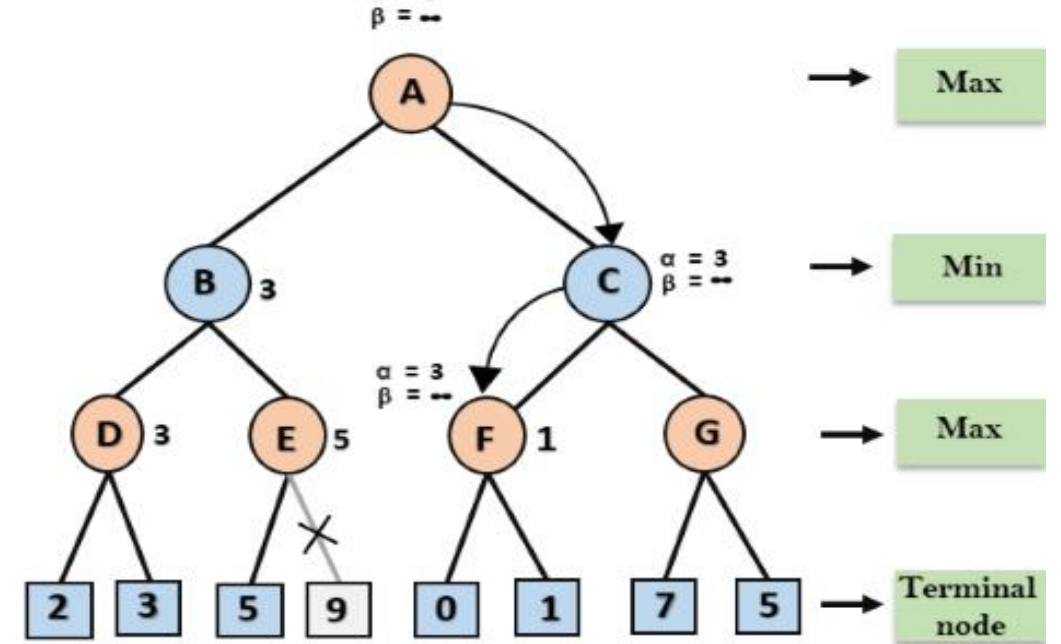
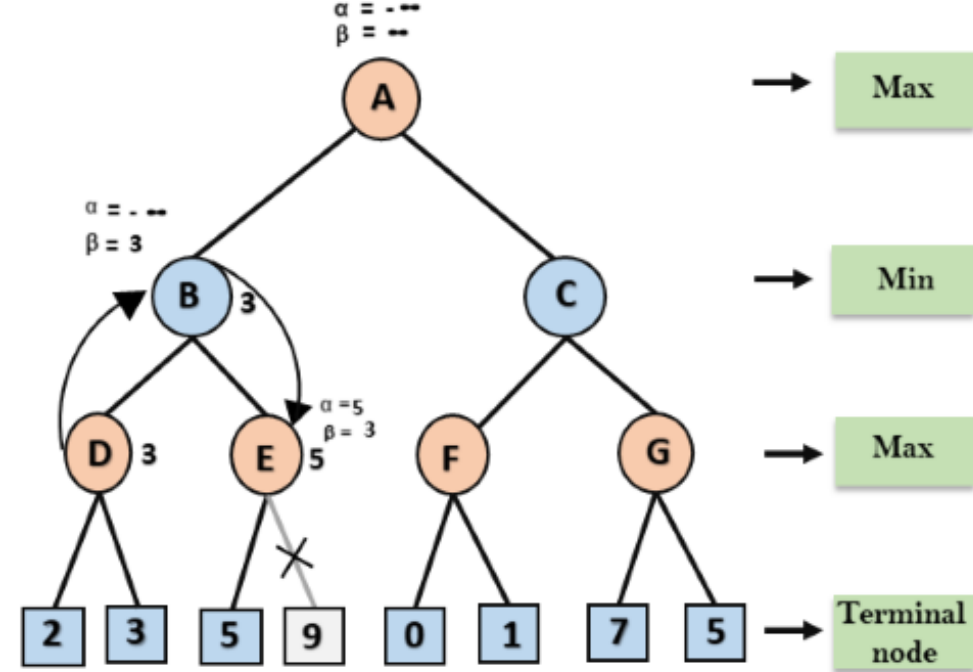


Step 2: At Node D, the value of α will be calculated as its turn for Max. The value of α is compared with firstly 2 and then 3, and the $\max(2, 3) = 3$ will be the value of α at node D and node value will also 3.

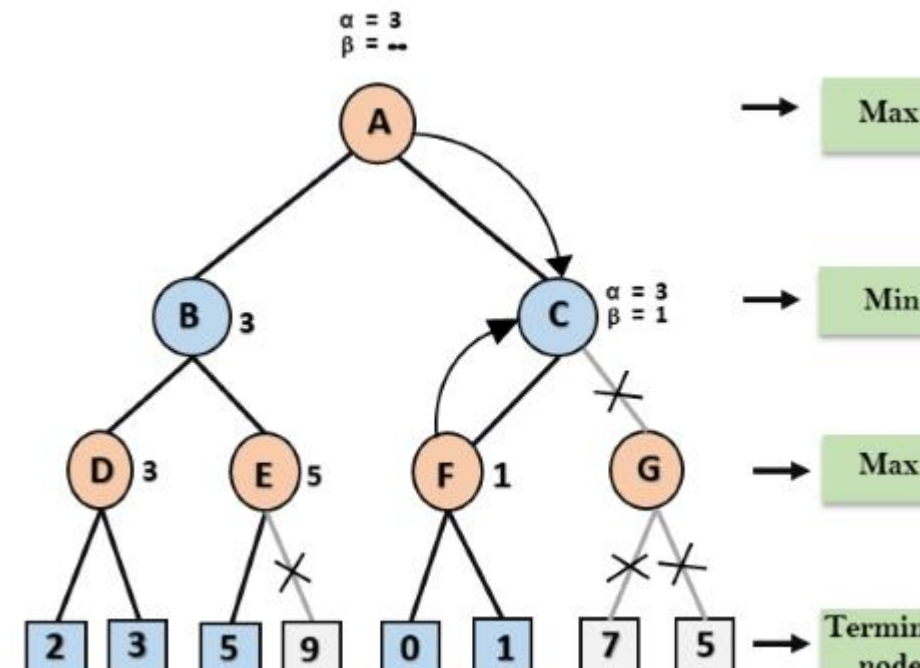
Step 3: Now algorithm backtrack to node B, where the value of β will change as this is a turn of Min, Now $\beta = +\infty$, will compare with the available subsequent nodes value, i.e. $\min(\infty, 3) = 3$, hence at node B now $\alpha = -\infty$, and $\beta = 3$.

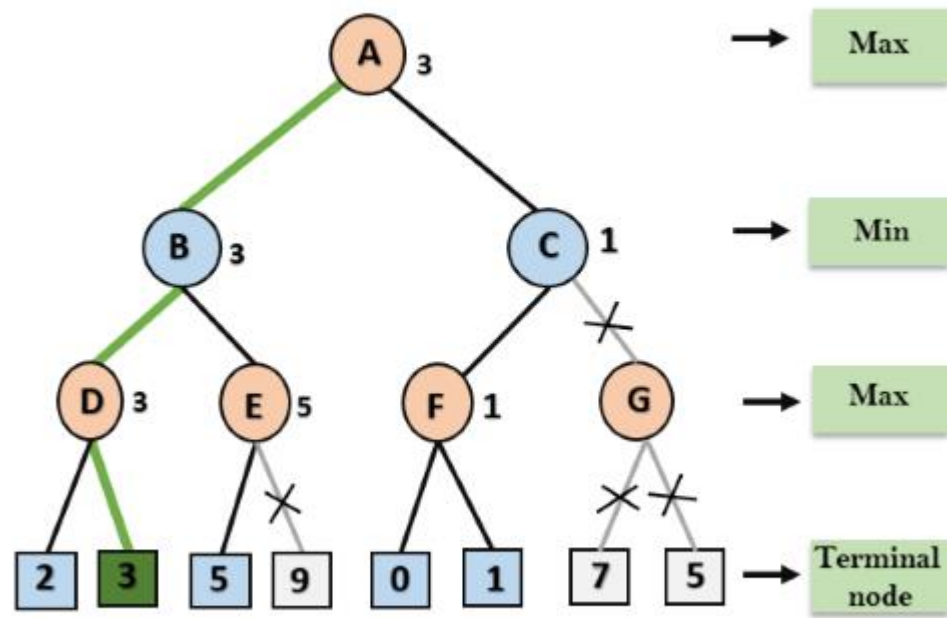
In the next step, algorithm traverse the next successor of Node B which is node E, and the values of $\alpha = -\infty$, and $\beta = 3$ will also be passed.

Step 4: At node E, Max will take its turn, and the value of alpha will change. The current value of alpha will be compared with 5, so $\max(-\infty, 5) = 5$, hence at node E $\alpha = 5$ and $\beta = 3$, where $\alpha \geq \beta$, so the right successor of E will be pruned, and algorithm will not traverse it, and the value at node E will be 5.



Step 5: At next step, algorithm again backtrack the tree, from node B to node A. At node A, the value of alpha will be changed the maximum available value is 3 as $\max(-\infty, 3) = 3$, and $\beta = +\infty$, these two values now passes to right successor of A which is Node C. At node C, $\alpha = 3$ and $\beta = +\infty$, and the same values will be passed on to node F.





Step 6: At node F, again the value of α will be compared with left child which is 0, and $\max(3,0)= 3$, and then compared with right child which is 1, and $\max(3,1)= 3$ still α remains 3, but the node value of F will become 1.

Step 7: Node F returns the node value 1 to node C, at C $\alpha= 3$ and $\beta= +\infty$, here the value of beta will be changed, it will compare with 1 so $\min(\infty, 1) = 1$. Now at C, $\alpha=3$ and $\beta= 1$, and again it satisfies the condition $\alpha \geq \beta$, so the next child of C which is G will be pruned, and the algorithm will not compute the entire sub-tree G.

```
function ALPHA-BETA-SEARCH(state) returns an action  
   $v \leftarrow \text{MAX-VALUE}(\text{state}, -\infty, +\infty)$   
  return the action in  $\text{ACTIONS}(\text{state})$  with value  $v$ 
```

```
function MAX-VALUE(state,  $\alpha$ ,  $\beta$ ) returns a utility value  
  if  $\text{TERMINAL-TEST}(\text{state})$  then return  $\text{UTILITY}(\text{state})$   
   $v \leftarrow -\infty$   
  for each  $a$  in  $\text{ACTIONS}(\text{state})$  do  
     $v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(\text{RESULT}(s, a), \alpha, \beta))$   
    if  $v \geq \beta$  then return  $v$   
     $\alpha \leftarrow \text{MAX}(\alpha, v)$   
  return  $v$ 
```

```
function MIN-VALUE(state,  $\alpha$ ,  $\beta$ ) returns a utility value  
  if  $\text{TERMINAL-TEST}(\text{state})$  then return  $\text{UTILITY}(\text{state})$   
   $v \leftarrow +\infty$   
  for each  $a$  in  $\text{ACTIONS}(\text{state})$  do  
     $v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(\text{RESULT}(s, a), \alpha, \beta))$   
    if  $v \leq \alpha$  then return  $v$   
     $\beta \leftarrow \text{MIN}(\beta, v)$   
  return  $v$ 
```

Figure 5.7 The alpha-beta search algorithm. Notice that these routines are the same as the MINIMAX functions in Figure 5.3, except for the two lines in each of MIN-VALUE and MAX-VALUE that maintain α and β (and the bookkeeping to pass these parameters along).

5.3.1: Move Ordering

- The effectiveness of alpha–beta pruning is highly dependent on the order in which the states are examined.
- we could not prune any successors of D in fig 5.5 at all because the worst successors (from the point of view of MIN) were generated first.
- If the third successor of D had been generated first, we would have been able to prune the other two.
- This suggests that it might be worthwhile to try to examine first the successors that are likely to be best.

If above method is followed then alpha–beta needs to examine only $O(b^m/2)$ nodes to pick the best move, instead of $O(b^m)$ for minimax.

- Adding dynamic move-ordering schemes, such as trying first the moves that were found to be best in the past, brings us quite close to the theoretical limit

- The best moves are often called **killer moves** and to try them first is called the killer move heuristic.

5.4 IMPERFECT REAL-TIME DECISIONS

- Disadvantages of minimax and alpha beta pruning.
- Claude Shannon's paper "*Programming a Computer for Playing Chess*" (1950) proposed instead that programs should cut off the search earlier and apply a heuristic **evaluation function** to states in the search, effectively turning nonterminal nodes into terminal leaves.

Based on above paper, minimax or alpha–beta can be altered in two ways:

- replace the utility function by a heuristic evaluation function EVAL, which estimates the position's utility.
- And replace the terminal test by a **cutoff test** that decides when to apply EVAL. That gives us the following for heuristic minimax for state s and maximum depth d :

$$H\text{-MINIMAX}(s, d) = \begin{cases} \text{EVAL}(s) & \text{if CUTOFF-TEST}(s, d) \\ \max_{a \in \text{Actions}(s)} H\text{-MINIMAX}(\text{RESULT}(s, a), d + 1) & \text{if PLAYER}(s) = \text{MAX} \\ \min_{a \in \text{Actions}(s)} H\text{-MINIMAX}(\text{RESULT}(s, a), d + 1) & \text{if PLAYER}(s) = \text{MIN}. \end{cases}$$

5.4.1 Evaluation functions:

- An evaluation function returns an *estimate* of the expected utility of the game from a given position, just as the heuristic functions.
- The idea of an estimator was not new when Shannon proposed it.
- So it should be clear that the performance of a game-playing program depends strongly on the quality of its evaluation function.

How to design good evaluation functions?

- **First**, the evaluation function should order the *terminal* states in the same way as the true utility function: states that are wins must evaluate better than draws, which in turn must be better than losses.
- **Second**, the computation must not take too long! (The whole point is to search faster.)
- **Third**, for nonterminal states, the evaluation function should be strongly correlated with the actual chances of winning.

- One might well wonder about the phrase “chances of winning.”
- chess is not a game of chance: we know the current state with certainty, and no dice are involved.
- But if the search must be cut off at nonterminal states, then the algorithm will necessarily be *uncertain* about the final outcomes of those states.

Let us make this idea more concrete with an ex of chess.

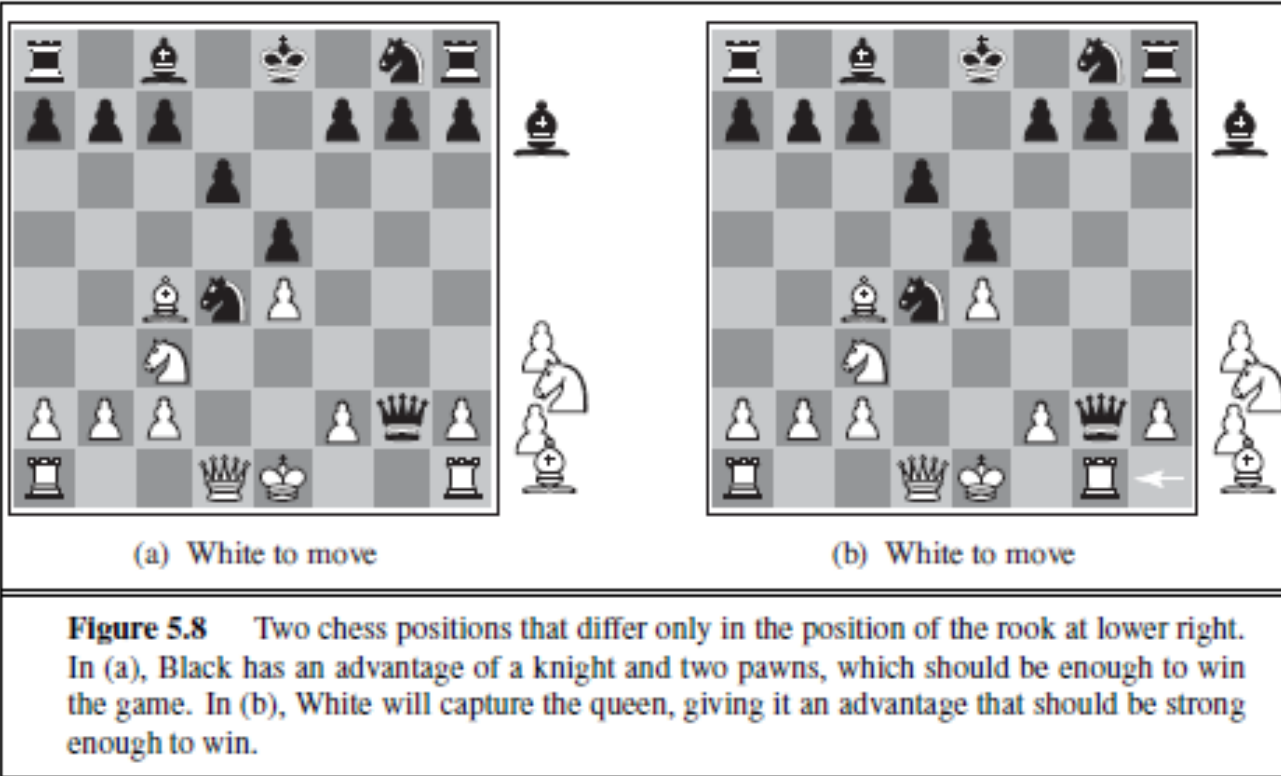
- Most evaluation functions work by calculating various **features** of the state—for example, in chess, we would have features for the number of white pawns, black pawns, white queens, black queens, and so on..
- For example any given category, generally speaking, will contain some states that lead to wins, some that lead to draws, and some that lead to losses.

- For example, suppose our experience suggests that 72% of the states encountered in the two-pawns vs. one-pawn category lead to a win (utility +1); 20% to a loss (0), and 8% to a draw (1/2). Then a reasonable evaluation for states in the category is the **expected value**: $(0.72 \times +1) + (0.20 \times 0) + (0.08 \times 1/2) = 0.76$.
- In practice, this kind of analysis requires too many categories and hence too much experience to estimate all the probabilities of winning. Instead, most evaluation functions compute separate numerical contributions from each feature and then *combine* them to find the total value.
- Mathematically, this kind of evaluation function is called a **weighted linear function** because it can be expressed as

$$\text{EVAL}(s) = w_1 f_1(s) + w_2 f_2(s) + \cdots + w_n f_n(s) = \sum_{i=1}^n w_i f_i(s) ,$$

- where each w_i is a weight and each f_i is a feature of the position. For chess, the f_i could be the numbers of each kind of piece on the board, and the w_i could be the values of the pieces (1 for pawn, 3 for bishop, etc.).

- Ex:



5.4.2 Cutting off search:

- The next step is to modify ALPHA-BETA-SEARCH so that it will call the heuristic EVAL function when it is appropriate to cut off the search. We replace the two lines in Figure 5.7 that mention TERMINAL-TEST with the following line:

if CUTOFF-TEST(state, depth) then return EVAL(state)

Logical Agents:

- Humans make strong claims about how the intelligence is achieved—not by purely reflex mechanisms but by processes of **reasoning** that operate on internal **representations** of knowledge.
- In AI, this approach to intelligence is embodied in **knowledge-based agents**.

KNOWLEDGE-BASED AGENTS:

- The central component of a knowledge-based agent is its **knowledge base**, or KB. A knowledge base is a set of **sentences**.
- Each sentence is expressed in a language called a **knowledge representation language** and represents some assertion about the world. (like **axiom**).
- The standard names for these operations are TELL and ASK, respectively. Both operations may involve **inference**.
- Inference must obey the requirement that when one ASKs a question of the knowledge base, the answer should follow from what has been told (or TELLed) to the knowledge base previously

- Like all our agents, it takes a percept as input and returns an action. The agent maintains a knowledge base, KB, which may initially contain some **background knowledge** as given in below fig.

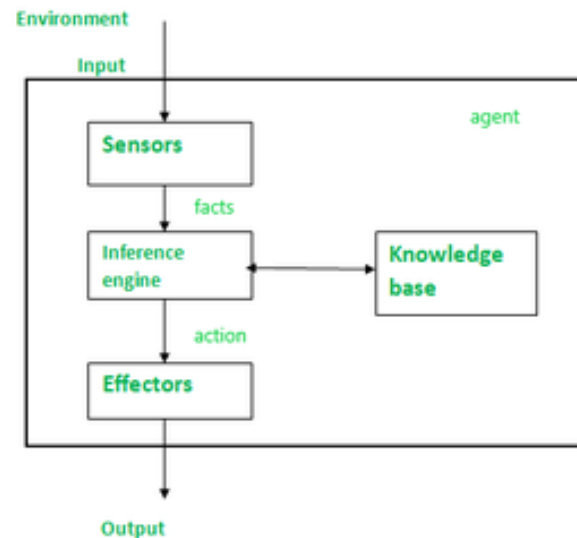
```
function KB-AGENT(percept) returns an action  
  persistent: KB, a knowledge base  
               t, a counter, initially 0, indicating time  
  
  TELL(KB, MAKE-PERCEPT-SENTENCE(percept, t))  
  action ← ASK(KB, MAKE-ACTION-QUERY(t))  
  TELL(KB, MAKE-ACTION-SENTENCE(action, t))  
  t ← t + 1  
  return action
```

Figure 7.1 A generic knowledge-based agent. Given a percept, the agent adds the percept to its knowledge base, asks the knowledge base for the best action, and tells the knowledge base that it has in fact taken that action.

- If a percept is given, agent adds it to KB, then it will ask KB for the best action and then tells KB that it has in fact taken that action.

Each time the agent program is called, it does following 3 things:

- First, it TELLS the knowledge base what it perceives.
- Second, it ASKS the knowledge base what action it should perform. In the process of answering this query, extensive reasoning may be done about the current state of the world, about the outcomes of possible action sequences, and so on.
- Third, the agent program TELLS the knowledge base on which action was chosen, and the agent executes the action.



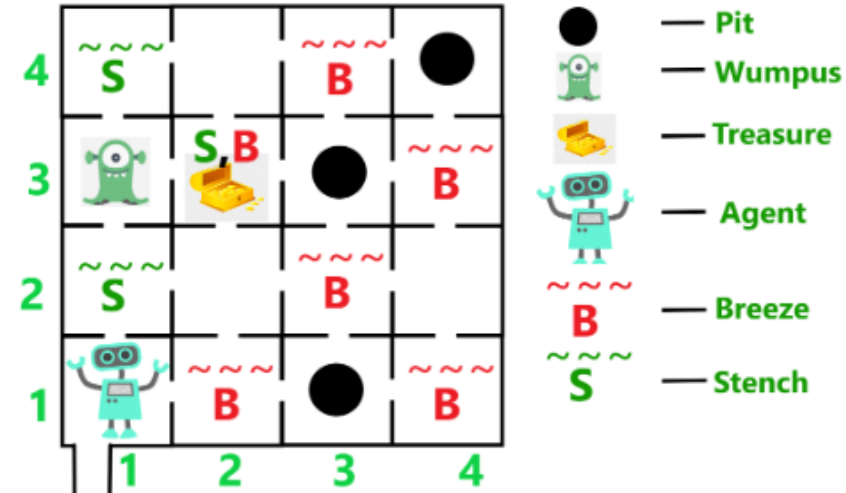
A Knowledge based system behavior can be designed in following approaches:-

- ***Declarative Approach:*** In this beginning from an empty knowledge base, the agent can TELL sentences one after another till the agent has knowledge of how to work with its environment. This is known as the declarative approach. It stores required information in empty knowledge-based system.
- ***Procedural Approach:*** This converts required behaviors directly into program code in empty knowledge-based system. It is a contrast approach when compared to Declarative approach. In this by coding behavior of system is designed .

THE WUMPUS WORLD:

It is an example of a knowledge-based agent that represents Knowledge representation, reasoning and planning. **Knowledge-Based agent** links general knowledge with current percepts to infer hidden characters of current state before selecting actions.

- The **wumpus world** is a cave consisting of rooms connected by passageways. Each room is connected to others through walkways(not diagonally connected).
- The knowledge-based agent starts from *Room[1, 1]*. The cave has – some **pits**, a **treasure** and a beast named **Wumpus**.
- Cave looks as shown:



Overview:

- Grid Environment: The Wumpus World is typically represented as a 4x4 grid.
- Goal: The agent's goal is to find and retrieve a piece of gold within the grid while avoiding dangerous obstacles and exiting safely.
- Obstacles and Hazards:
 - Wumpus: A monster that kills the agent if they enter its square. The Wumpus does not move.
 - Pits: Bottomless pits that the agent falls into and dies if it enters a square containing one.
 - Gold: The target item the agent seeks to find and retrieve.

Percepts available to the agent:

The agent does not have a full view of the grid. Instead, it receives percepts from the environment:

- **Breeze:** If a square is adjacent to a pit, the agent feels a breeze.
- **Stench:** If a square is adjacent to the Wumpus, the agent smells a stench.
- **Glitter:** When the agent is in the same square as the gold, it sees a glitter.
- **Bump:** If the agent attempts to move into a wall, it feels a bump.
- **Scream:** If the agent successfully shoots the Wumpus, it hears a scream.

- The Wumpus cannot move but eats the one who enters its room.
- If the agent enters the pit, it gets stuck there.
- The goal of the agent is to take the treasure and come out of the cave.
- The agent is rewarded, when the goal conditions are met.
- The agent is penalized, when it falls into a pit or being eaten by the Wumpus.
- Some elements support the agent to explore the cave, like -The wumpus's adjacent rooms are stenchy.
- The agent is given one arrow which it can use to kill the wumpus when facing it (Wumpus screams when it is killed).
- The adjacent rooms of the room with pits are filled with breeze.
- The treasure room is always glittery.

PEAS description for wumpus world problem:

1. Performance measures:

- Agent gets the gold and return back safe = *+1000 points*
- Agent dies = *-1000 points*
- Each move of the agent = *-1 point*
- Agent uses the arrow = *-10 points*

2. Environment:

- A cave with *16(4×4)* rooms
- Rooms adjacent (not diagonally) to the Wumpus are stinking
- Rooms adjacent (not diagonally) to the pit are breezy
- The room with the gold glitters
- Agent's initial position – *Room[1, 1]* and facing right side
- Location of Wumpus, gold and *3 pits* can be anywhere, except in *Room[1, 1]*.

3. Actuators:

Devices that allow the agent to perform the following actions in the environment.

- Move forward
- Turn right by 90°
- Turn left 90°
- Shoot: The agent dies a miserable death if it enters a square containing a pit or a live wumpus. The agent has only one arrow, so only the first *Shoot* action has any effect.
- Grab The action *Grab* can be used to pick up the gold if it is in the same square as the agent.
- Finally, the action *Climb* can be used to climb out of the cave, but only from square [1,1].

4. Sensors:

The agent has **five** sensors, each of which gives a single bit of information

- In the square containing the wumpus and in the directly (not diagonally) adjacent squares, the agent will perceive a ***Stench***.
- In the squares directly adjacent to a pit, the agent will perceive a ***Breeze***.
- In the square where the gold is, the agent will perceive a ***Glitter***.
- When an agent walks into a wall, it will perceive a ***Bump***.
- When the wumpus is killed, it emits a woeful ***Scream*** that can be perceived anywhere in the cave.

- The percepts will be given to the agent program in the form of a list of five symbols;
- for example, if there is a stench and a breeze, but no glitter, bump, or scream, the agent program will get [Stench, Breeze, None, None, None].

Characterization of Wumpus World:

- **Partially Observable:** knows only the local perceptions
- **Deterministic:** outcome is precisely specified
- **Sequential:** subsequent level of actions performed
- **Static:** Wumpus, pits are immobile
- **Discrete:** discrete environment
- **Single-agent:** The knowledge-based agent is the only agent whereas the wumpus is considered as the environment's feature.

Solution:

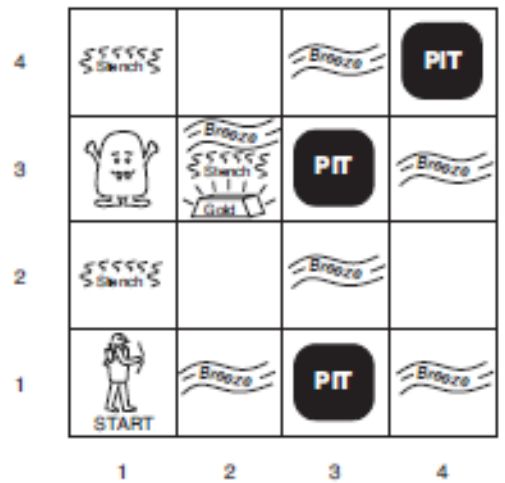


Figure 7.2 A typical wumpus world. The agent is in the bottom left corner, facing right.

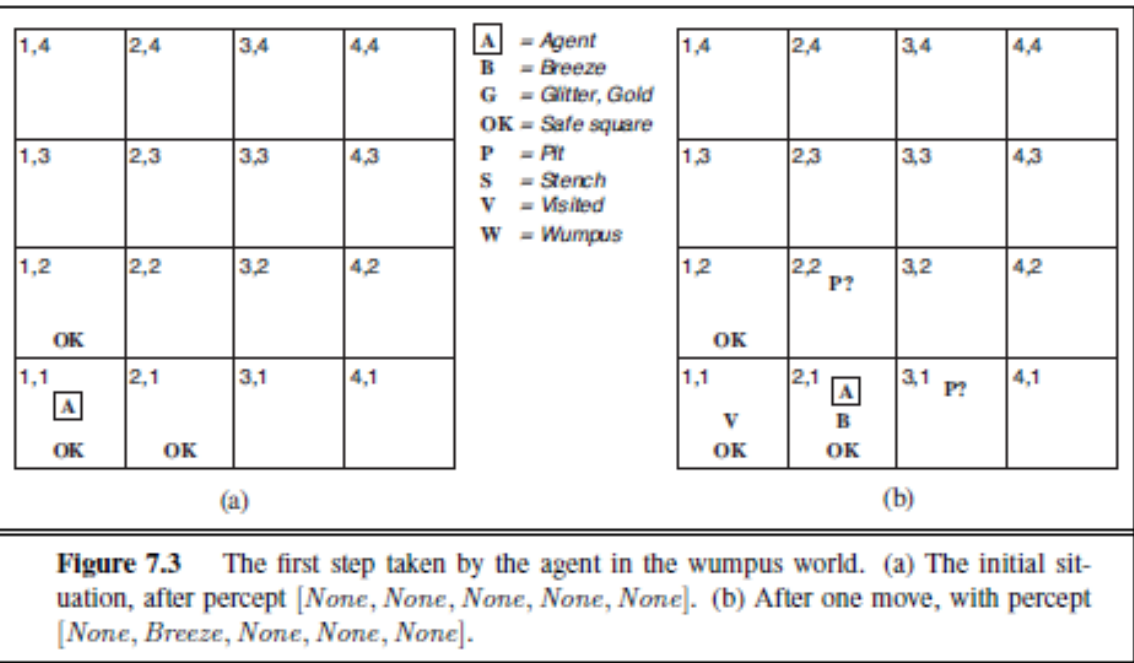


Figure 7.3 The first step taken by the agent in the wumpus world. (a) The initial situation, after percept [None, None, None, None, None]. (b) After one move, with percept [None, Breeze, None, None, None].

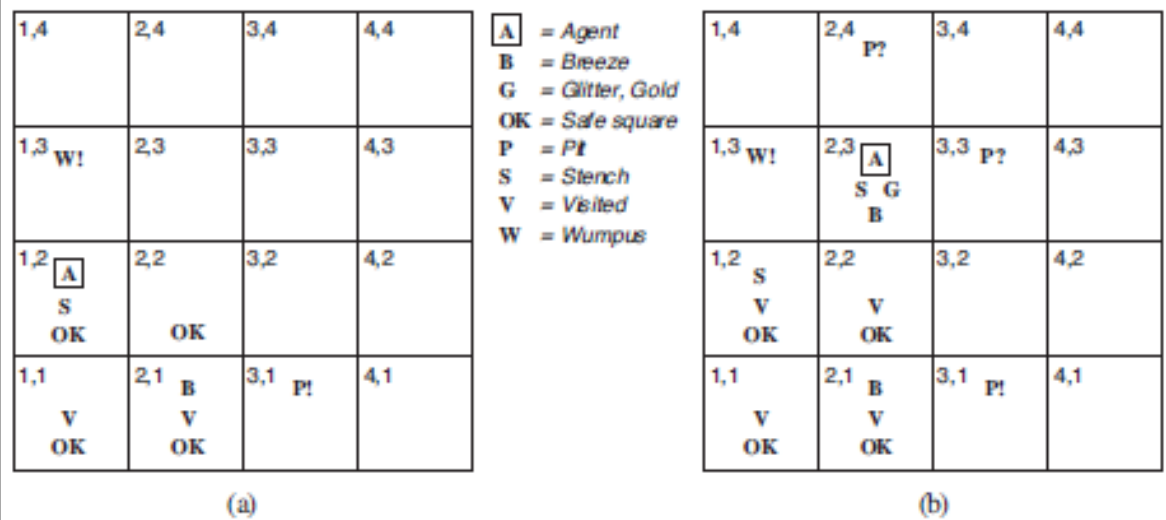


Figure 7.4 Two later stages in the progress of the agent. (a) After the third move, with percept [Stench, None, None, None, None]. (b) After the fifth move, with percept [Stench, Breeze, Glitter, None, None].

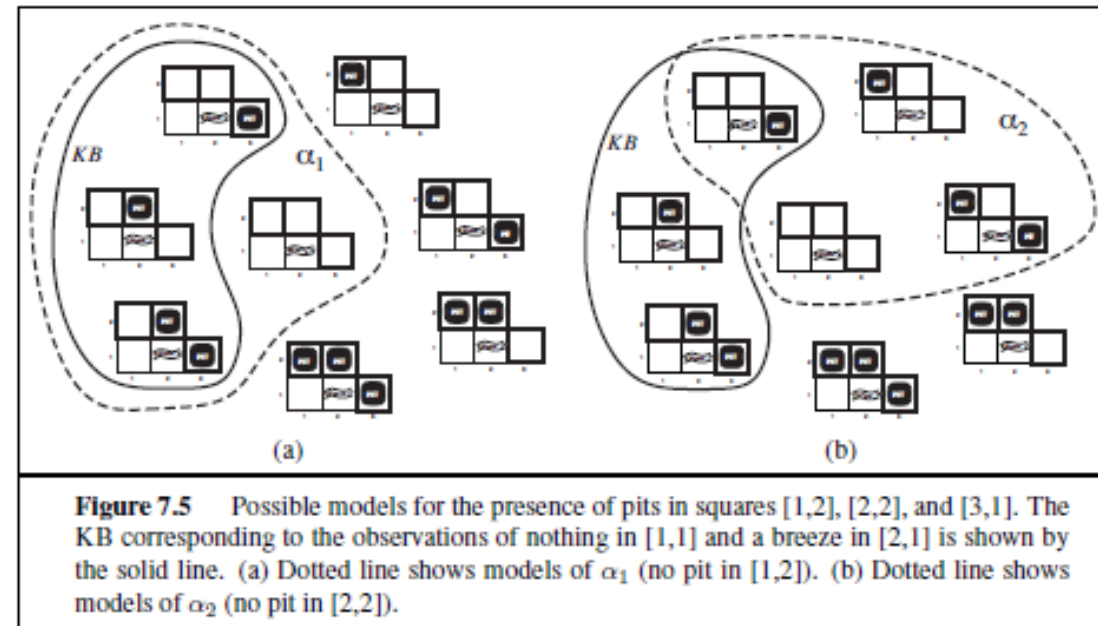
- The first percept is [None, None, None, None, None], from which the agent can conclude that its neighboring squares, [1,2] and [2,1], are free of dangers—they are OK.
- Let us assume that the agent decides to move forward to [2,1]. The agent perceives a breeze (denoted by “B”) in [2,1], so there must be a pit in a neighboring square. The pit cannot be in [1,1], by the rules of the game, so there must be a pit in [2,2] or [3,1] or both. At this point, there is only one known square that is OK and that has not yet been visited, the agent will turn around, go back to [1,1], and then proceed to [1,2].
- The stench in [1,2] means that there must be a wumpus nearby. But the wumpus cannot be in [1,1], by the rules of the game, and it cannot be in [2,2] (or the agent would have detected a stench when it was in [2,1]). Therefore, the agent can infer that the wumpus is in [1,3].
- This is a fairly difficult inference, because it combines knowledge gained at different times in different places and relies on the lack of a percept to make one crucial step.

7.3 LOGIC:

- The Knowledge based sentences are expressed according to the **syntax** of the representation language, which specifies all the sentences that are well formed.
- A logic must also define the **semantics** or meaning of sentences.
- The semantics defines the **truth** of each sentence with respect to each **possible world**. For example, the semantics for arithmetic specifies that the sentence “ $x + y = 4$ ” is true in a world where x is 2 and y is 2.
- When we need to be precise, we use the term **model** in place of “possible world”.
- If a sentence α is true in model m , we say that m **satisfies** α or sometimes m **is a model of** α . We use the notation $M(\alpha)$ to mean the set of all models of α .
- In mathematical notation, we write $\alpha \models \beta$, meaning α entails β .

- Using the notation just introduced, we can write

$$\alpha \models \beta \text{ if and only if } M(\alpha) \subseteq M(\beta)$$
- Apply the Same kind of analysis to the wumpus-world reasoning example.(7.3b).
- Each of the three squares might or might not contain a pit, so (for the purposes of this example) there are $2^3 = 8$ possible models. These eight models are shown in Figure 7.5.
- There are in fact just three models in which the KB is true, and these are shown surrounded by a solid line in Figure 7.5



7.4 PROPOSITIONAL LOGIC: A VERY SIMPLE LOGIC

A powerful logic called **propositional logic**. Here we cover the syntax of propositional logic and its semantics—the way in which the truth of sentences is determined.

Syntax:

- The **syntax** of propositional logic defines the allowable sentences. The **atomic sentences** consist of a single **proposition symbol**. Each such symbol stands for a proposition that can be true or false.
- We use symbols that start with an uppercase letter and may contain other letters or subscripts, for example: P, Q, R, $W_{1,3}$ and North.
- $W_{1,3}$ to stand for the proposition that the wumpus is in [1,3].
- **Complex sentences** are constructed from simpler sentences, using parentheses and **logical connectives**.

There are five connectives in common use:

- $\neg$ (not). A sentence such as $\neg W_{1,3}$ is called the **negation** of $W_{1,3}$. A **literal** is either an atomic sentence (a **positive literal**) or a negated atomic sentence (a **negative literal**).
- $\wedge$ (and). A sentence whose main connective is $\wedge$, such as $W_{1,3} \wedge P_{3,1}$, is called a **conjunction**; its parts are the **conjuncts**. (The $\wedge$ looks like an “A” for “And.”)
- $\vee$ (or). A sentence using $\vee$, such as $(W_{1,3} \wedge P_{3,1}) \vee W_{2,2}$, is a **disjunction** of the **disjuncts** $(W_{1,3} \wedge P_{3,1})$ and $W_{2,2}$.
- $\Rightarrow$ (implies). A sentence such as $(W_{1,3} \wedge P_{3,1}) \Rightarrow \neg W_{2,2}$ is called an **implication** (or conditional). Its **premise** or **antecedent** is $(W_{1,3} \wedge P_{3,1})$, and its **conclusion** or **consequent** is $\neg W_{2,2}$. Implications are also known as **rules** or **if–then** statements. The implication symbol is sometimes written in other books as $\supset$ or $\rightarrow$.
- $\Leftrightarrow$ (if and only if). The sentence $W_{1,3} \Leftrightarrow \neg W_{2,2}$ is a **biconditional**. Some other books write this as $\equiv$.

For implication

P	Q	$P \rightarrow Q$
True	True	True
True	False	False
False	True	True
False	False	True

For biconditional

P	Q	$P \leftrightarrow Q$
True	True	True
True	False	False
False	True	False
False	False	True

P	Q	R	$\neg R$	$P \vee Q$	$P \vee Q \rightarrow \neg R$
True	True	True	False	True	False
True	True	False	True	True	True
True	False	True	False	True	False
True	False	False	True	True	True
False	True	True	False	True	False
False	True	False	True	True	True
False	False	True	False	False	True
False	False	False	True	False	True

BNF:

```
Sentence → AtomicSentence | ComplexSentence  
AtomicSentence → True | False | P | Q | R | ...  
ComplexSentence → ( Sentence ) | [ Sentence ]  
                  | ¬ Sentence  
                  | Sentence ∧ Sentence  
                  | Sentence ∨ Sentence  
                  | Sentence ⇒ Sentence  
                  | Sentence ⇔ Sentence  
  
OPERATOR PRECEDENCE : ¬, ∧, ∨, ⇒, ⇔
```

Figure 7.7 A BNF (Backus–Naur Form) grammar of sentences in propositional logic, along with operator precedences, from highest to lowest.

Semantics:

- The semantics defines the rules for determining the truth of a sentence with respect to a particular model.
- For example, if the sentences in the knowledge base make use of the proposition symbols $P_{1,2}$, $P_{2,2}$ and $P_{3,1}$ then one possible model is

$$m1 = \{P_{1,2} = \text{false}, P_{2,2} = \text{false}, P_{3,1} = \text{true}\} .$$

To compute the truth of sentences formed with each of the five connectives.

For Atomic sentences, it is easy:

- True is true in every model and False is false in every model. The truth value of every other proposition symbol must be specified directly in the model. For example, in the model m_1 given earlier, $P_{1,2}$ is false.

For complex sentences, we have five rules, which hold for any subsentences P and Q in any model m :

- $\neg P$ is true iff P is false in m .
- $P \wedge Q$ is true iff both P and Q are true in m .
- $P \vee Q$ is true iff either P or Q is true in m .
- $P \Rightarrow Q$ is true unless P is true and Q is false in m .
- $P \Leftrightarrow Q$ is true iff P and Q are both true or both false in m .

Truth Table for complex sentence:

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
false	false	true	false	false	true	true
false	true	true	false	true	true	false
true	false	false	false	true	false	false
true	true	false	true	true	true	true

Figure 7.8 Truth tables for the five logical connectives. To use the table to compute, for example, the value of $P \vee Q$ when P is true and Q is false, first look on the left for the row where P is true and Q is false (the third row). Then look in that row under the $P \vee Q$ column to see the result: true.

Ex: $\neg P_{1,2} \wedge (P_{2,2} \vee P_{3,1})$.

- Many of the rules of the wumpus world are best written using $\Leftrightarrow$. For example, a square is breezy *if* a neighboring square has a pit, and a square is breezy *only if* a neighboring square has a pit. So we need a biconditional,

$$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1}),$$

where $B_{1,1}$ means that there is a breeze in $[1,1]$.

A simple knowledge base:

After defining semantics for propositional logic, we can construct a knowledge base for the wumpus world.

We focus first on the *immutable* aspects of the wumpus world, leaving the mutable aspects for a later section. For now, we need the following symbols for each $[x, y]$ location:

- $P_{x,y}$ is true if there is a pit in $[x, y]$.
- $W_{x,y}$ is true if there is a wumpus in $[x, y]$, dead or alive.
- $B_{x,y}$ is true if the agent perceives a breeze in $[x, y]$.
- $S_{x,y}$ is true if the agent perceives a stench in $[x, y]$.

The sentences we write will suffice to derive $\neg P_{1,2}$ (there is no pit in $[1,2]$), as was done informally.

Label each sentence R_i so that we can refer to them:

- There is no pit in $[1,1]$:
- $R1 : \neg P_{1,1}$.

A square is breezy if and only if there is a pit in a neighboring square. This has to be stated for each square; for now, we include just the relevant squares:

- $R2 : B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$.
- $R3 : B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$.

The preceding sentences are true in all wumpus worlds. Now we include the breeze percepts for the first two squares visited in the specific world the agent is in, leading up to the situation in Figure 7.3(b).

- $R4 : \neg B_{1,1}$.
- $R5 : B_{2,1}$.

- Figure 7.9 reproduces in a more precise form the reasoning illustrated in Figure 7.5. A general algorithm for deciding entailment in propositional logic is shown in Figure 7.10.

$B_{1,1}$	$B_{2,1}$	$P_{1,1}$	$P_{1,2}$	$P_{2,1}$	$P_{2,2}$	$P_{3,1}$	R_1	R_2	R_3	R_4	R_5	KB
false	false	false	false	false	false	false	true	true	true	true	false	false
false	false	false	false	false	false	true	true	true	false	true	false	false
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
false	true	false	false	false	false	false	true	true	false	true	true	false
false	true	false	false	false	false	true	true	true	true	true	true	<u>true</u>
false	true	false	false	false	true	false	true	true	true	true	true	<u>true</u>
false	true	false	false	false	true	true	true	true	true	true	true	<u>true</u>
false	true	false	false	true	false	false	true	false	false	true	true	false
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
true	true	true	true	true	true	true	false	true	true	false	true	false

Figure 7.9 A truth table constructed for the knowledge base given in the text. KB is true if R_1 through R_5 are true, which occurs in just 3 of the 128 rows (the ones underlined in the right-hand column). In all 3 rows, $P_{1,2}$ is false, so there is no pit in $[1,2]$. On the other hand, there might (or might not) be a pit in $[2,2]$.

$(\alpha \wedge \beta) \equiv (\beta \wedge \alpha)$	commutativity of $\wedge$
$(\alpha \vee \beta) \equiv (\beta \vee \alpha)$	commutativity of $\vee$
$((\alpha \wedge \beta) \wedge \gamma) \equiv (\alpha \wedge (\beta \wedge \gamma))$	associativity of $\wedge$
$((\alpha \vee \beta) \vee \gamma) \equiv (\alpha \vee (\beta \vee \gamma))$	associativity of $\vee$
$\neg(\neg\alpha) \equiv \alpha$	double-negation elimination
$(\alpha \Rightarrow \beta) \equiv (\neg\beta \Rightarrow \neg\alpha)$	contraposition
$(\alpha \Rightarrow \beta) \equiv (\neg\alpha \vee \beta)$	implication elimination
$(\alpha \Leftrightarrow \beta) \equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha))$	biconditional elimination
$\neg(\alpha \wedge \beta) \equiv (\neg\alpha \vee \neg\beta)$	De Morgan
$\neg(\alpha \vee \beta) \equiv (\neg\alpha \wedge \neg\beta)$	De Morgan
$(\alpha \wedge (\beta \vee \gamma)) \equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma))$	distributivity of $\wedge$ over $\vee$
$(\alpha \vee (\beta \wedge \gamma)) \equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma))$	distributivity of $\vee$ over $\wedge$

Figure 7.11 Standard logical equivalences. The symbols α , β , and γ stand for arbitrary sentences of propositional logic.

7.6 EFFECTIVE PROPOSITIONAL MODEL CHECKING

2 efficient algorithms for general propositional inference based on model checking:

1. Backtracking search.
2. Local hill-climbing search.

These algorithms are part of the “technology” of propositional logic. These algorithms describe for checking satisfiability: the SAT problem ($\alpha \models \beta$).

Backtracking algorithm: Davis–Putnam algorithm or DPLL algorithm.

It is essentially a recursive, depth-first enumeration of possible models. It embodies three improvements over the simple scheme of TT-ENTAILS.

- ***Early termination***: The algorithm detects whether the sentence must be true or false, even with a partially completed model.

For $\text{ex}(A \vee B) \wedge (A \vee C)$ – is true if A is true regardless of B and C

Pure symbol heuristic: **pure symbol** is a symbol that always appears with the same “sign” in all clauses.

For example, in the three clauses $(A \vee \neg B)$, $(\neg B \vee \neg C)$, and $(C \vee A)$, the symbol A is pure because only the positive literal appears, B is pure because only the negative literal appears, and C is impure.

Unit clause heuristic: A **unit clause** was defined earlier as a clause with just one literal. In the context of DPLL, it also means clauses in which all literals but one are already assigned false by the model.

- **Ex:** if the model contains $B = \text{true}$, then $(\neg B \vee \neg C)$ simplifies to $\neg C$, which is a unit clause. Obviously, for this clause to be true, C must be set to false. The unit clause heuristic assigns all such symbols. Notice also that assigning one unit clause can create another unit clause.

Local search algorithms: One of the simplest and most effective algorithms to emerge from all this work is called WALKSAT.

- On every iteration, the algorithm picks an unsatisfied clause and picks a symbol in the clause to flip.
- It chooses randomly between two ways to pick which symbol to flip:
 - (1) a “min-conflicts” step that minimizes the number of unsatisfied clauses in the new state and
 - (2) a “random walk” step that picks the symbol randomly.
- When WALKSAT returns a model, the input sentence is indeed satisfiable, but when it returns failure, there are two possible causes: either the sentence is unsatisfiable or we need to give the algorithm more time.

Limitations:

- WALKSAT is most useful when we expect a solution to exist
- WALKSAT cannot always detect *unsatisfiability*, which is required for deciding entailment

Ex for local search: Hill Climbing algorithm-

- The algorithm loop continually moves in the direction of increasing value—that is, uphill. It terminates when it reaches a “peak” where no neighbor has a higher value.
- The algorithm does not maintain a search tree, so the data structure for the current node need only record the state and the value of the objective function.
- Ex 8-queens problem or 8 puzzle problem.
- Hill climbing is sometimes called greedy local search, because it grabs a good neighbor state without thinking ahead about where to go next.

Unfortunately, hill climbing often gets stuck for the following reasons:

- **Local maxima:**

a local maximum is a peak that is higher than each of its neighboring states but lower than the global maximum. Hill-climbing algorithms that reach the vicinity of a local maximum will be drawn upward toward the peak but will then be stuck with nowhere else to go.

- **Ridges:** Ridges result in a sequence of local maxima that is very difficult for greedy algorithms to navigate.
- **Plateaux:** a plateau is a flat area of the state-space landscape. It can be a flat local maximum, from which no uphill exit exists, or a **shoulder**, from which progress is possible.

Exercise Question:

Decide whether each of the following sentences is valid, unsatisfiable, or neither. Verify your decisions using truth tables or the equivalence rules of Figure 7.11

a. $\text{Smoke} \Rightarrow \text{Smoke}$

This is a valid sentence. This can be verified using truth tables, but it is easier to observe that the sentence is equivalent to $\neg \text{Smoke} \vee \text{Smoke}$ which is true always true.

b. $\text{Smoke} \Rightarrow \text{Fire}$

This is just implication with two propositional variables, so we can refer to the truth table for " $\Rightarrow$ ". It is satisfiable since the implication is true when Smoke is false or when Fire is true.

c. $(\text{Smoke} \Rightarrow \text{Fire}) \Rightarrow (\neg \text{Smoke} \Rightarrow \neg \text{Fire})$

Again we can use truth tables, but let's try simplification using equivalence rules:

$(\text{Smoke} \Rightarrow \text{Fire}) \Rightarrow (\neg \text{Smoke} \Rightarrow \neg \text{Fire}) \quad (\neg \text{Smoke} \vee \text{Fire}) \Rightarrow (\text{Smoke} \vee \neg \text{Fire})$ [elimination the inside implications]

$\neg (\neg \text{Smoke} \vee \text{Fire}) \vee (\text{Smoke} \vee \neg \text{Fire})$ [eliminating the remaining implication]

$(\text{Smoke} \wedge \neg \text{Fire}) \vee (\text{Smoke} \vee \neg \text{Fire})$ [moving negation inside - DeMorgan's laws]

$(\text{Smoke} \vee \text{Smoke} \vee \neg \text{Fire}) \wedge (\text{Fire} \vee \text{Smoke} \vee \neg \text{Fire})$ [distributivity]

$(\text{Smoke} \vee \neg \text{Fire}) \wedge \text{True}$ [simplification - see also part (d)]

$(\text{Smoke} \vee \neg \text{Fire})$ [note: $P \wedge \text{True}$ is equivalent to P]

$(\text{Fire} \Rightarrow \text{Smoke})$ [converting back to implication] So, we have reduced the formula to $\text{Fire} \Rightarrow \text{Smoke}$ which is satisfiable using similar reasoning as in part (b).

d. $\text{Smoke} \vee \text{Fire} \vee \neg \text{Fire}$

This is just equivalent to True, since it contains $\text{Fire} \vee \neg \text{Fire}$ which is valid (always true). Disjunction of a true statement with anything is still true. So the above formula is valid.

e. $((\text{Smoke} \wedge \text{Heat}) \Rightarrow \text{Fire}) \Leftrightarrow ((\text{Smoke} \Rightarrow \text{Fire}) \vee (\text{Heat} \Rightarrow \text{Fire}))$

This is valid. You can show this validity by constructing truth tables for the two sides of logical equivalence $\Leftrightarrow$ and verifying this equivalence (by showing that for any assignment of truth values to the variables Smoke, Heat, and Fire, the two formulas are either both true or both false).

First Order Logic: FOL is much more expressive language than the propositional logic. It is an extension of PL. Also called as predicate logic.

- PL is a declarative language because its semantics is based on a truth relation between sentences and possible worlds. It also has sufficient expressive power to deal with partial information, using disjunction and negation.
- Drawback of PL is it lacks the expressive power to *concisely* describe an environment with many objects.
- For example, we were forced to write a separate rule about breezes and pits for each square, such as $B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$.

FOL is known as the powerful language which is used to develop information related to objects in a very easy way.

About objects, relations and properties:

- **Objects:** people, houses, numbers, theories, Ronald McDonald, colors, baseball games, wars, centuries . . .
- **Relations:** these can be unary relations or **properties** such as red, round, bogus, prime, multistoried . . ., or more general n-ary relations such as brother of, bigger than, inside, part of, has color, occurred after, owns, comes between, . . .
- **Functions:** father of, best friend, third inning of, one more than, beginning of . . .

Ex: Evil King John ruled England in 1200.”

Objects:

Relations:

Properties:

SYNTAX:

Sentence → *AtomicSentence* | *ComplexSentence*

AtomicSentence → *Predicate* | *Predicate*(*Term*, ...) | *Term* = *Term*

ComplexSentence → (*Sentence*) | [*Sentence*]

 | ¬ *Sentence*

 | *Sentence* ∧ *Sentence*

 | *Sentence* ∨ *Sentence*

 | *Sentence* ⇒ *Sentence*

 | *Sentence* ⇔ *Sentence*

 | *Quantifier* *Variable*, ... *Sentence*

Term → *Function*(*Term*, ...)

 | *Constant*

 | *Variable*

Quantifier → ∀ | ∃

Constant → *A* | *X*₁ | *John* | ...

Variable → *a* | *x* | *s* | ...

Predicate → *True* | *False* | *After* | *Loves* | *Raining* | ...

Function → *Mother* | *LeftLeg* | ...

OPERATOR PRECEDENCE : ¬, =, ∧, ∨, ⇒, ⇔

Quantifiers in FOL:

- It express the properties of entire collections of objects, instead of enumerating the objects by name.
- First-order logic contains two standard quantifiers, called *universal* and *existential*.

Universal quantification ($\forall$):

“Squares neighboring the wumpus are smelly” and “All kings are persons”.

The second rule, “All kings are persons,” is written in first-order logic as

$$\forall x \text{ King}(x) \Rightarrow \text{Person}(x) .$$

“For all x , if x is a king, then x is a person.” The symbol x is called a **variable**.

A term with no variables is called a **ground term**.

Consider the model shown in Figure 8.2 and the intended interpretation that goes with it. We can extend the interpretation in 5 ways:

$x \rightarrow$ Richard the Lionheart,

$x \rightarrow$ King John,

$x \rightarrow$ Richard's left leg,

$x \rightarrow$ John's left leg,

$x \rightarrow$ the crown.

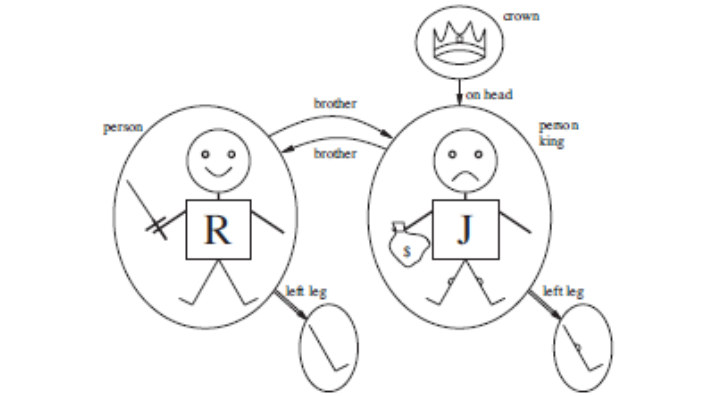


Figure 8.2 A model containing five objects, two binary relations, three unary relations (indicated by labels on the objects), and one unary function, left-leg.

- The universally quantified sentence $\forall x \text{ King}(x) \Rightarrow \text{Person}(x)$ is true in the original model if the sentence $\text{King}(x) \Rightarrow \text{Person}(x)$ is true under each of the five extended interpretations.
- Richard the Lionheart is a king $\Rightarrow$ Richard the Lionheart is a person.
- King John is a king $\Rightarrow$ King John is a person.
- Richard's left leg is a king $\Rightarrow$ Richard's left leg is a person.
- John's left leg is a king $\Rightarrow$ John's left leg is a person.
- The crown is a king $\Rightarrow$ the crown is a person.

Existential quantification ($\exists$): we can make a statement about *some* object in the universe without naming it.

Ex: King John has a crown on his head, can be written as

$$\exists x \text{Crown}(x) \wedge \text{OnHead}(x, \text{John})$$

$\exists x$ is pronounced “There exists an x such that . . .” or “For some x . . .”.

Intuitively, the sentence $\exists x P$ says that P is true for at least one object x .

Wumpus world representation using FOL:

- The wumpus agent receives a percept vector with five elements. The corresponding first-order sentence stored in the knowledge base must include both the percept and the time at which it occurred;

Percept ([Stench, Breeze, Glitter , None, None], 5) .

Here, Percept is a binary predicate, and Stench and so on are constants placed in a list.

- The actions in the wumpus world can be represented by logical terms:

Turn(Right), Turn(Left), Forward , Shoot , Grab, Climb .

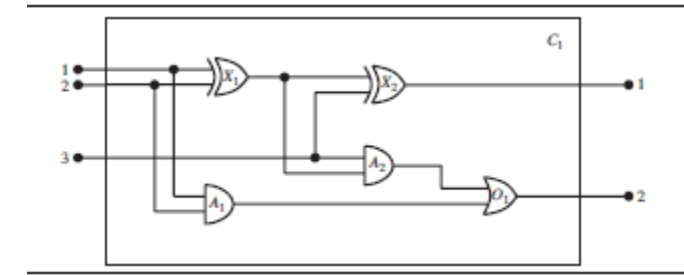
- To determine which is best, the agent program executes the query

ASK V ARS($\exists$ a BestAction(a, 5)) ,

which returns a binding list such as {a/Grab}. The agent program can then return Grab as the action to take.

The knowledge-engineering process: Knowledge engineering projects vary widely in content, scope, and difficulty, but all such projects include the following steps:

- *Identify the task.*
- *Assemble the relevant knowledge*
- *Decide on a vocabulary of predicates, functions, and constants.*
- *Encode general knowledge about the domain.*
- *Encode a description of the specific problem instance.*
- *Pose queries to the inference procedure and get answers.*
- *Debug the knowledge base*



Ex: The electronic circuits domain

- **Identify the task:** does the circuit in Figure 8.6 actually add properly? If all the inputs are high, what is the output of gate A2?

- **Assemble the relevant knowledge:** What do we know about digital circuits?
- **Decide on a vocabulary:** The next step is to choose functions, predicates, and constants to represent them.

First, we need to be able to distinguish gates from each other and from other objects. Each gate is represented as an object named by a constant, about which we assert that it is a gate with, say, $\text{Gate}(X1)$. Identify predicate $\text{Terminal}(x)$.

- **Encode general knowledge of the domain:** One sign that we have a good ontology is that we require only a few general rules, which can be stated clearly and concisely. These are all the axioms we will need:

1. If two terminals are connected, then they have the same signal:

$$\forall t1, t2 \text{ Terminal}(t1) \wedge \text{Terminal}(t2) \wedge \text{Connected}(t1, t2) \Rightarrow \text{Signal}(t1) = \text{Signal}(t2)$$

Encode the specific problem instance:

Ex: $\text{Gate}(X1) \wedge \text{Type}(X1) = \text{XOR}$

- **Pose queries to the inference procedure:** What combinations of inputs would cause the first output of C1 to be 0 and the second output of C1 to be 1?
- **Debug the knowledge base:** We can perturb the knowledge base in various ways to see what kinds of erroneous behaviors emerge.

Reduction to propositional inference: For example, suppose our knowledge base contains just the sentences

$\forall x \text{ King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$

$\text{King}(\text{John})$

$\text{Greedy}(\text{John})$

$\text{Brother}(\text{Richard}, \text{John})$.

Then we apply UI to the first sentence using all possible ground-term substitutions from the vocabulary of the knowledge base—in this case, $\{x/\text{John}\}$ and $\{x/\text{Richard}\}$. We obtain

- $\text{King}(\text{John}) \wedge \text{Greedy}(\text{John}) \Rightarrow \text{Evil}(\text{John})$
- $\text{King}(\text{Richard}) \wedge \text{Greedy}(\text{Richard}) \Rightarrow \text{Evil}(\text{Richard})$

Now, the knowledge base is essentially propositional if we view the ground atomic sentences— $\text{King}(\text{John})$, $\text{Greedy}(\text{John})$, and so on—as proposition symbols.

Exercise 8.10

Consider a vocabulary with the following symbols:

- Occupation(p, o): Predicate. Person p has occupation o.
- Customer (p1, p2): Predicate. Person p1 is a customer of person p2.
- Boss(p1, p2): Predicate. Person p1 is a boss of person p2.
- Doctor , Surgeon, Lawyer , Actor : Constants denoting occupations.
- Emily, Joe: Constants denoting people.

Use these symbols to write the following assertions in first-order logic:

- a. Emily is either a surgeon or a lawyer.
- b. Joe is an actor, but he also holds another job.
- c. All surgeons are doctors.
- d. Joe does not have a lawyer (i.e., is not a customer of any lawyer).
- e. Emily has a boss who is a lawyer.
- f. There exists a lawyer all of whose customers are doctors.
- g. Every surgeon has a lawyer.

a) Emily is either a surgeon or a lawyer.

$\text{Occupation}(\text{Emily}, \text{Surgeon}) \vee \text{Occupation}(\text{Emily}, \text{Lawyer})$

b) Joe is an actor, but he also holds another job.

$\text{Occupation}(\text{Joe}, \text{Actor}) \wedge (\text{Occupation}(\text{Joe}, \text{Doctor}) \vee \text{Occupation}(\text{Joe}, \text{Surgeon}) \vee \text{Occupation}(\text{Joe}, \text{Lawyer}))$

c) All surgeons are doctors.

$\forall p \text{Occupation}(p, \text{Surgeon}) \Rightarrow \text{Occupation}(p, \text{Doctor})$

d) Joe does not have a lawyer.

$\sim \text{Customer}(\text{Joe}, \text{Lawyer})$

e) Emily has a boss who is a lawyer.

$\text{Boss}(\text{Emily}, \text{Lawyer})$

f) There exists a lawyer all of whose customers are doctors.

$\exists p1 \forall p2 \text{Occupation}(p1, \text{Lawyer}) \Rightarrow \text{Customer}(p2, p1) \wedge \text{Occupation}(p2, \text{Doctor})$

g) Every surgeon has a lawyer.

$\forall p1 \exists p2 \text{Occupation}(p1, \text{Surgeon}) \Rightarrow \text{Customer}(p1, p2) \wedge \text{Occupation}(p2, \text{Lawyer})$

UNIT 3

Inference in FOL: To infer (deduce) the facts from the given sentences.

2 terms in inference are

- i. substitution – substitute a variable in place of constant
- ii. Equality – brother(A)=B

Inference rules for quantifiers:

Suppose our knowledge base contains the standard axiom stating that “all greedy kings are evil”.

$$\forall x \text{ King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x) .$$

Then it seems quite permissible to infer any of the following sentences:

- $\text{King}(\text{John}) \wedge \text{Greedy}(\text{John}) \Rightarrow \text{Evil}(\text{John})$
- $\text{King}(\text{Richard}) \wedge \text{Greedy}(\text{Richard}) \Rightarrow \text{Evil}(\text{Richard})$
- $\text{King}(\text{Father}(\text{John})) \wedge \text{Greedy}(\text{Father}(\text{John})) \Rightarrow \text{Evil}(\text{Father}(\text{John})) .$

Universal Generalization:

- Universal generalization is a valid inference rule which states that if premise $P(c)$ is true for any arbitrary element c in the universe of discourse, then we can have a conclusion as $\forall x P(x)$.
- It can be represented as:
$$\frac{P(c)}{\forall x P(x)}$$
- This rule can be used if we want to show that every element has a similar property.
- In this rule, x must not appear as a free variable.

Example:

Let's represent, $P(c)$: "**A byte contains 8 bits**", so for $\forall x P(x)$ "**All bytes contain 8 bits.**", it will also be true.

Universal Instantiation (UI):

- Universal instantiation is also called as universal elimination or UI is a valid inference rule. It can be applied multiple times to add new sentences.
- The new KB is logically equivalent to the previous KB.
- As per UI, **we can infer any sentence obtained by substituting a ground term for the variable.**
- The UI rule state that we can infer any sentence $P(c)$ by substituting a ground term c (a constant within domain x) from **$\forall x P(x)$ for any object in the universe of discourse.**

It can be represented as:
$$\frac{\forall x P(x)}{P(c)}$$

Ex:

IF "Every person like ice-cream" $\Rightarrow \forall x P(x)$ so we can infer that
"John likes ice-cream" $\Rightarrow P(c)$

Ex 2: Let's take another example,

"All kings who are greedy are Evil." So let our knowledge base contains this detail as in the form of FOL: $\forall x \text{ king}(x) \wedge \text{greedy}(x) \rightarrow \text{Evil}(x)$

we can infer any of the following statements using Universal Instantiation:

- $\text{King}(\text{John}) \wedge \text{Greedy}(\text{John}) \rightarrow \text{Evil}(\text{John}),$
- $\text{King}(\text{Richard}) \wedge \text{Greedy}(\text{Richard}) \rightarrow \text{Evil}(\text{Richard}),$
- $\text{King}(\text{Father}(\text{John})) \wedge \text{Greedy}(\text{Father}(\text{John})) \rightarrow \text{Evil}(\text{Father}(\text{John})),$

Existential Instantiation:

- Existential instantiation is also called as Existential Elimination, which is a valid inference rule in first-order logic.
- It can be applied only once to replace the existential sentence.
- The new KB is not logically equivalent to old KB, but it will be satisfiable if old KB was satisfiable.
- This rule states that one can infer $P(c)$ from the formula given in the form of $\exists x P(x)$ for a new constant symbol c .
- The restriction with this rule is that c used in the rule must be a new term for which $P(c)$ is true.
- It can be represented as
$$\frac{\exists x P(x)}{P(c)}$$

Ex: $\exists x \text{Crown}(x) \wedge \text{OnHead}(x, \text{John})$, this can be inferred as:

$\text{Crown}(K) \wedge \text{OnHead}(K, \text{John})$, as long as K does not appear in the knowledge base

Generalized Modus Ponens Rule:

- For the inference process in FOL, we have a single inference rule which is called Generalized Modus Ponens. It is lifted version of Modus ponens.
- Generalized Modus Ponens can be summarized as, " P implies Q and P is asserted to be true, therefore Q must be True."
- According to Modus Ponens, for atomic sentences **p_i, p_i', q** . Where there is a substitution θ such that **$SUBST(\theta, p_i') = SUBST(\theta, p_i)$** , it can be represented as:

$$\frac{p_1', p_2', \dots, p_n', (p_1 \wedge p_2 \wedge \dots \wedge p_n \Rightarrow q)}{SUBST(\theta, q)}$$

Ex: We will use this rule for Kings are evil, so we will find some x such that x is king, and x is greedy so we can infer that x is evil.

- Here let say, p_1' is king(John) p_1 is king(x)
- p_2' is Greedy(y) p_2 is Greedy(x)
- θ is {x/John, y/John} q is evil(x)
- $SUBST(\theta, q)$.

Unification:

- Unification is a process of making two different logical atomic expressions identical by finding a substitution. Unification depends on the substitution process.
- It takes two literals as input and makes them identical using substitution.
- Let Ψ_1 and Ψ_2 be two atomic sentences and σ be a unifier such that, $\Psi_1\sigma = \Psi_2\sigma$, then it can be expressed as **UNIFY**(Ψ_1, Ψ_2).

Ex: Find the MGU for Unify{King(x), King(John)}

Let $\Psi_1 = \text{King}(x)$, $\Psi_2 = \text{King}(\text{John})$,

Substitution $\theta = \{\text{John}/x\}$ is a unifier for these atoms and applying this substitution, and both expressions will be identical.

- The UNIFY algorithm is used for unification, which takes two atomic sentences and returns a unifier for those sentences (If any exist).
- Unification is a key component of all first-order inference algorithms.
- It returns fail if the expressions do not match with each other.
- The substitution variables are called Most General Unifier or MGU.

Ex2: Let's say there are two different expressions, **$P(x, y)$** , and **$P(a, f(z))$** .

In this example, we need to make both above statements identical to each other.

$P(x, y)$ (i)

$P(a, f(z))$ (ii)

- Substitute x with a , and y with $f(z)$ in the first expression, and it will be represented as **a/x** and **$f(z)/y$** .
- With both the substitutions, the first expression will be identical to the second expression and the substitution set will be: **$[a/x, f(z)/y]$** .

Following are some basic conditions for unification:

- Predicate symbol must be same, atoms or expression with different predicate symbol can never be unified.
- Number of Arguments in both expressions must be identical.
- Unification will fail if there are two similar variables present in the same expression.

Forward Chaining:

- **Definite clause:** A clause which is a disjunction of literals with **exactly one positive literal** is known as a definite clause or strict horn clause.
- **Horn clause:** A clause which is a disjunction of literals with **at most one positive literal** is known as horn clause. Hence all the definite clauses are horn clauses.
- **Example:** $(\neg p \vee \neg q \vee k)$. It has only one positive literal k.

Forward chaining is also known as a forward deduction or forward reasoning method when using an inference engine.

Forward chaining is a form of reasoning which start with atomic sentences in the knowledge base and applies inference rules (Modus Ponens) in the forward direction to extract more data until a goal is reached.

Properties of Forward-Chaining:

- It is a bottom-up approach, as it moves from bottom to top.
- It is a process of making a conclusion based on known facts or data, by starting from the initial state and reaches the goal state.
- Forward-chaining approach is also called as data-driven as we reach to the goal using available data.
- Forward -chaining approach is commonly used in the expert system, such as CLIPS, business, and production rule systems.

Ex: "As per the law, it is a crime for an American to sell weapons to hostile nations. Country A, an enemy of America, has some missiles, and all the missiles were sold to it by Robert, who is an American citizen."

- Prove that "**Robert is criminal.**"
- To solve the above problem, first, convert all the above facts into first-order definite clauses, and then use a forward-chaining algorithm to reach the goal.

Facts Conversion into FOL:

- It is a crime for an American to sell weapons to hostile nations. (Let's say p, q, and r are variables)
American (p) $\wedge$ weapon(q) $\wedge$ sells (p, q, r) $\wedge$ hostile(r) $\rightarrow$ Criminal(p)(1)
- Country A has some missiles. **$\exists p$ Owns(A, p) $\wedge$ Missile(p).** It can be written in two definite clauses by using Existential Instantiation, introducing new Constant T1.
Owns(A, T1)(2)
Missile(T1)(3)

- All of the missiles were sold to country A by Robert.
 $\forall p \text{ Missiles}(p) \wedge \text{Owns} (A, p) \rightarrow \text{Sells} (\text{Robert}, p, A)$ (4)
- Missiles are weapons.
 $\text{Missile}(p) \rightarrow \text{Weapons} (p)$ (5)
- Enemy of America is known as hostile.
 $\text{Enemy}(p, \text{America}) \rightarrow \text{Hostile}(p)$ (6)
- Country A is an enemy of America.
 $\text{Enemy} (A, \text{America})$ (7)
- Robert is American
 $\text{American}(\text{Robert}).$ (8)

Forward chaining proof:

Step-1:

- In the first step we will start with the known facts and will choose the sentences which do not have implications, such as: **American(Robert), Enemy(A, America), Owns(A, T1), and Missile(T1).**

All these facts will be represented as below:

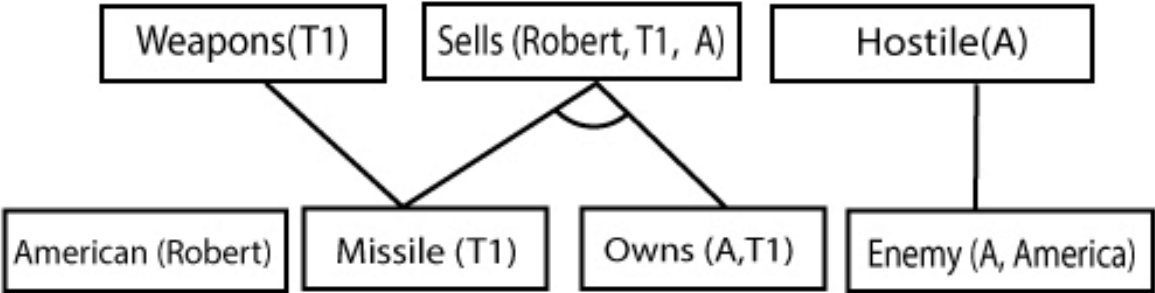


Step-2:

At the second step, we will see those facts which infer from available facts and with satisfied premises. Rule-(1) does not satisfy premises, so it will not be added in the first iteration.

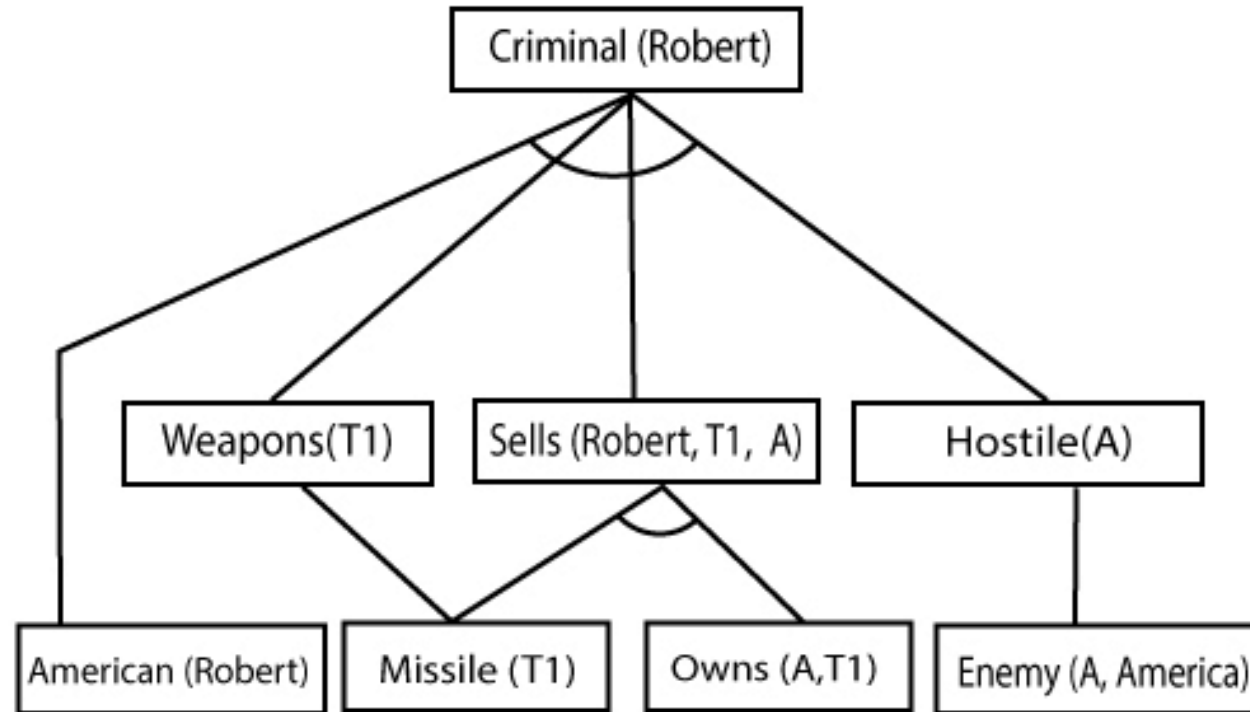
Rule-(2) and (3) are already added.

Rule-(6) is satisfied with the substitution(p/A), so Hostile(A) is added and which infers from Rule-(7).



Step-3:

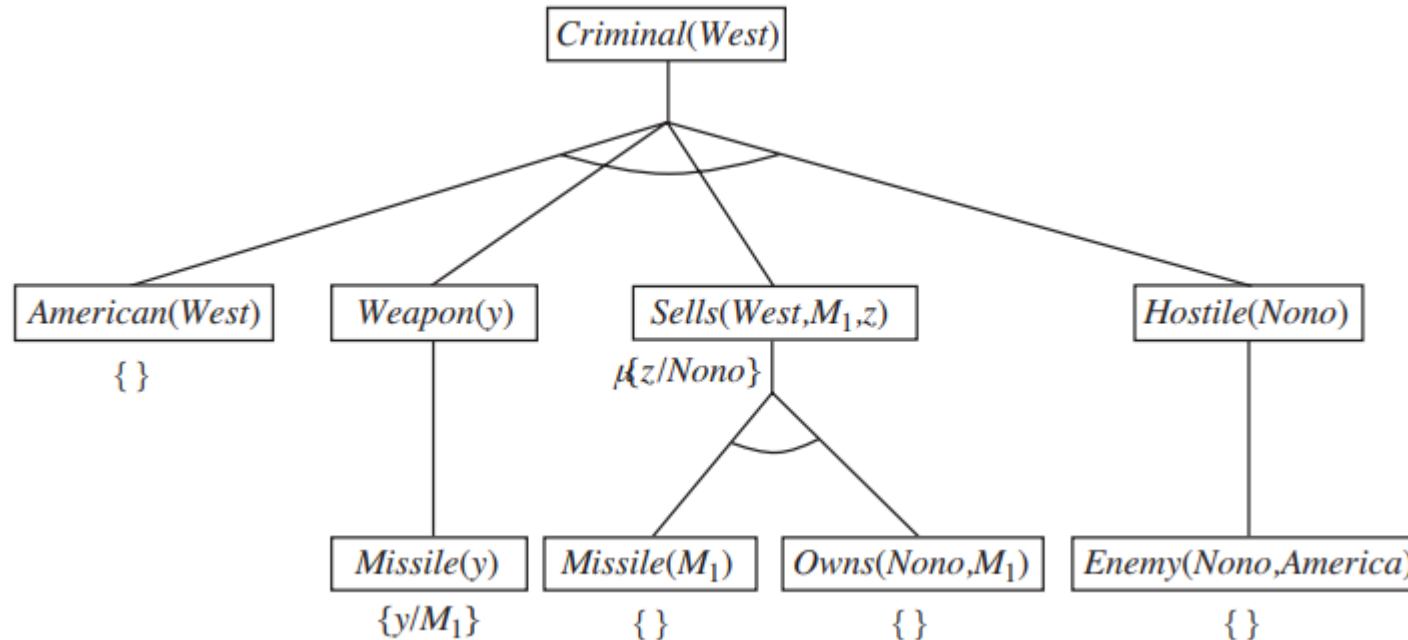
- At step-3, as we can check Rule-(1) is satisfied with the substitution **{p/Robert, q/T1, r/A}**, so we can add **Criminal(Robert)** which infers all the available facts and hence goal statement is reached.



Incremental forward chaining: refer textbook

Backward chaining

- The second major family of logical inference algorithms uses the backward chaining approach.
- These algorithms work backward from the goal, chaining through rules to find known facts that support the proof. (Refer text book for algorithm)



Conjunctive Normal forms:

- to prove both the expressions are equal, a standard form called CNF is required. An expression in CNF is a POS.

- converting the sentence $B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$ into CNF. The steps are as follows:

1. Eliminate $\Leftrightarrow$, replacing $\alpha \Leftrightarrow \beta$ with $(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$.

$$(B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1}) .$$

2. Eliminate $\Rightarrow$, replacing $\alpha \Rightarrow \beta$ with $\neg\alpha \vee \beta$:

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg(P_{1,2} \vee P_{2,1}) \vee B_{1,1})$$

3. CNF requires $\neg$ to appear only in literals, so “move $\neg$ inwards” by repeated application of the following equivalences:

$$\neg(\neg\alpha) \equiv \alpha \text{ (double-negation elimination)}$$

$$\neg(\alpha \wedge \beta) \equiv (\neg\alpha \vee \neg\beta) \text{ (De Morgan)}$$

$$\neg(\alpha \vee \beta) \equiv (\neg\alpha \wedge \neg\beta) \text{ (De Morgan)}$$

In the example, we require just one application of the last rule:

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge ((\neg P_{1,2} \wedge \neg P_{2,1}) \vee B_{1,1}) .$$

4. Now we have a sentence containing nested $\wedge$ and $\vee$ operators applied to literals. We apply the distributivity law:

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1}) .$$

The original sentence is now in CNF, as a conjunction of three clauses. It is much harder to read, but it can be used as input to a resolution procedure.

Resolution in FOL

- It is a theorem-proving technique that proceeds by building refutation proofs, i.e., proofs by contradictions.
- Resolution is used, if there are various statements are given, and we need to prove a conclusion of those statements.
- Resolution is a single inference rule which can efficiently operate on the **CNF or clausal form**.
- Resolution can resolve two clauses if they contain complementary literals, which are assumed to be standardized apart so that they share no variables.

$$\frac{l_1 \vee \dots \vee l_k, \quad m_1 \vee \dots \vee m_n}{\text{SUBST}(\theta, l_1 \vee \dots \vee l_{i-1} \vee l_{i+1} \vee \dots \vee l_k \vee m_1 \vee \dots \vee m_{j-1} \vee m_{j+1} \vee \dots \vee m_n)}$$

- This rule is also called the **binary resolution rule** because it only resolves exactly two literals.

Example:

We can resolve two clauses which are given below:

$[\text{Animal}(g(x) \vee \text{Loves}(f(x), x)]$ and $[\neg \text{Loves}(a, b) \vee \neg \text{Kills}(a, b)]$

Where two complimentary literals are: **$\text{Loves}(f(x), x)$ and $\neg \text{Loves}(a, b)$**

These literals can be unified with unifier **$\theta = [a/f(x), \text{ and } b/x]$** , and it will generate a resolvent clause:

$[\text{Animal}(g(x) \vee \neg \text{Kills}(f(x), x)]$.

Steps for Resolution:

- Conversion of facts into first-order logic.
- Convert FOL statements into CNF
- Negate the statement which needs to prove (proof by contradiction)
- Draw resolution graph (unification).

Example:

- John likes all kind of food.
- Apple and vegetable are food
- Anything anyone eats and not killed is food.
- Anil eats peanuts and still alive
- Harry eats everything that Anil eats.

a. $\forall x: \text{food}(x) \rightarrow \text{likes}(\text{John}, x)$

b. $\text{food}(\text{Apple}) \wedge \text{food}(\text{vegetables})$

c. $\forall x \forall y: \text{eats}(x, y) \wedge \neg \text{killed}(x) \rightarrow \text{food}(y)$

d. $\text{eats}(\text{Anil}, \text{Peanuts}) \wedge \text{alive}(\text{Anil})$.

e. $\forall x : \text{eats}(\text{Anil}, x) \rightarrow \text{eats}(\text{Harry}, x)$

f. $\forall x: \neg \text{killed}(x) \rightarrow \text{alive}(x)$ } **added predicates.**

g. $\forall x: \text{alive}(x) \rightarrow \neg \text{killed}(x)$ }

h. $\text{likes}(\text{John}, \text{Peanuts})$

Prove by resolution that: “John likes peanuts.”

Step-1: Conversion of Facts into FOL

- a. $\forall x: \text{food}(x) \rightarrow \text{likes}(\text{John}, x)$
 - b. $\text{food}(\text{Apple}) \wedge \text{food}(\text{vegetables})$
 - c. $\forall x \forall y: \text{eats}(x, y) \wedge \neg \text{killed}(x) \rightarrow \text{food}(y)$
 - d. $\text{eats}(\text{Anil}, \text{Peanuts}) \wedge \text{alive}(\text{Anil})$.
 - e. $\forall x : \text{eats}(\text{Anil}, x) \rightarrow \text{eats}(\text{Harry}, x)$
 - f. $\forall x: \neg \text{killed}(x) \rightarrow \text{alive}(x)$
 - g. $\forall x: \text{alive}(x) \rightarrow \neg \text{killed}(x)$
 - h. $\text{likes}(\text{John}, \text{Peanuts})$
- } **added predicates.**

Step-2: Conversion of FOL into CNF

- **Eliminate all implication ($\rightarrow$) and rewrite**

- $\forall x \neg \text{food}(x) \vee \text{likes}(\text{John}, x)$
- $\text{food}(\text{Apple}) \wedge \text{food}(\text{vegetables})$
- $\forall x \forall y \neg [\text{eats}(x, y) \wedge \neg \text{killed}(x)] \vee \text{food}(y)$
- $\text{eats}(\text{Anil}, \text{Peanuts}) \wedge \text{alive}(\text{Anil})$
- $\forall x \neg \text{eats}(\text{Anil}, x) \vee \text{eats}(\text{Harry}, x)$
- $\forall x \neg [\neg \text{killed}(x)] \vee \text{alive}(x)$
- $\forall x \neg \text{alive}(x) \vee \neg \text{killed}(x)$
- $\text{likes}(\text{John}, \text{Peanuts})$.

- **Move negation ($\neg$) inwards and rewrite**
 - $\forall x \neg \text{food}(x) \vee \text{likes}(\text{John}, x)$
 - $\text{food}(\text{Apple}) \wedge \text{food}(\text{vegetables})$
 - $\forall x \forall y \neg \text{eats}(x, y) \vee \text{killed}(x) \vee \text{food}(y)$
 - $\text{eats}(\text{Anil}, \text{Peanuts}) \wedge \text{alive}(\text{Anil})$
 - $\forall x \neg \text{eats}(\text{Anil}, x) \vee \text{eats}(\text{Harry}, x)$
 - $\forall x \neg \text{killed}(x) \vee \text{alive}(x)$
 - $\forall x \neg \text{alive}(x) \vee \neg \text{killed}(x)$
 - $\text{likes}(\text{John}, \text{Peanuts})$.
- **Rename variables or standardize variables**
 - $\forall x \neg \text{food}(x) \vee \text{likes}(\text{John}, x)$
 - $\text{food}(\text{Apple}) \wedge \text{food}(\text{vegetables})$
 - $\forall y \forall z \neg \text{eats}(y, z) \vee \text{killed}(y) \vee \text{food}(z)$
 - $\text{eats}(\text{Anil}, \text{Peanuts}) \wedge \text{alive}(\text{Anil})$
 - $\forall w \neg \text{eats}(\text{Anil}, w) \vee \text{eats}(\text{Harry}, w)$
 - $\forall g \neg \text{killed}(g) \vee \text{alive}(g)$
 - $\forall k \neg \text{alive}(k) \vee \neg \text{killed}(k)$
 - $\text{likes}(\text{John}, \text{Peanuts})$.

Eliminate existential instantiation quantifier by elimination.

In this step, we will eliminate existential quantifier $\exists$, and this process is known as **Skolemization**. But in this example problem since there is no existential quantifier so all the statements will remain same in this step.

Drop Universal quantifiers.

In this step we will drop all universal quantifier since all the statements are not implicitly quantified so we don't need it.

- $\neg \text{food}(x) \vee \text{likes}(\text{John}, x)$
- $\text{food}(\text{Apple})$
- $\text{food}(\text{vegetables})$
- $\neg \text{eats}(y, z) \vee \text{killed}(y) \vee \text{food}(z)$
- $\text{eats}(\text{Anil}, \text{Peanuts})$
- $\text{alive}(\text{Anil})$
- $\neg \text{eats}(\text{Anil}, w) \vee \text{eats}(\text{Harry}, w)$
- $\text{killed}(g) \vee \text{alive}(g)$
- $\neg \text{alive}(k) \vee \neg \text{killed}(k)$
- $\text{likes}(\text{John}, \text{Peanuts})$.

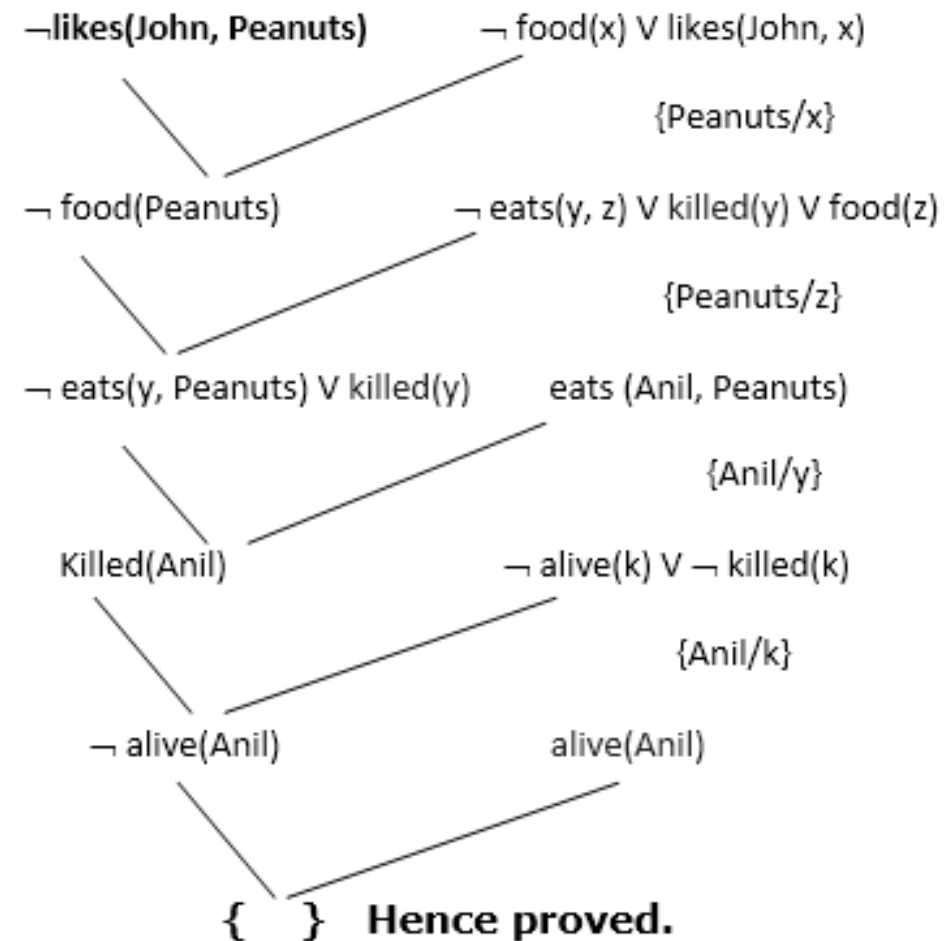
Distribute conjunction $\wedge$ over disjunction $\neg$.

This step will not make any change in this problem.

Step-3: Negate the statement to be proved

- In this statement, we will apply negation to the conclusion statements, which will be written as $\neg \text{likes}(\text{John}, \text{Peanuts})$

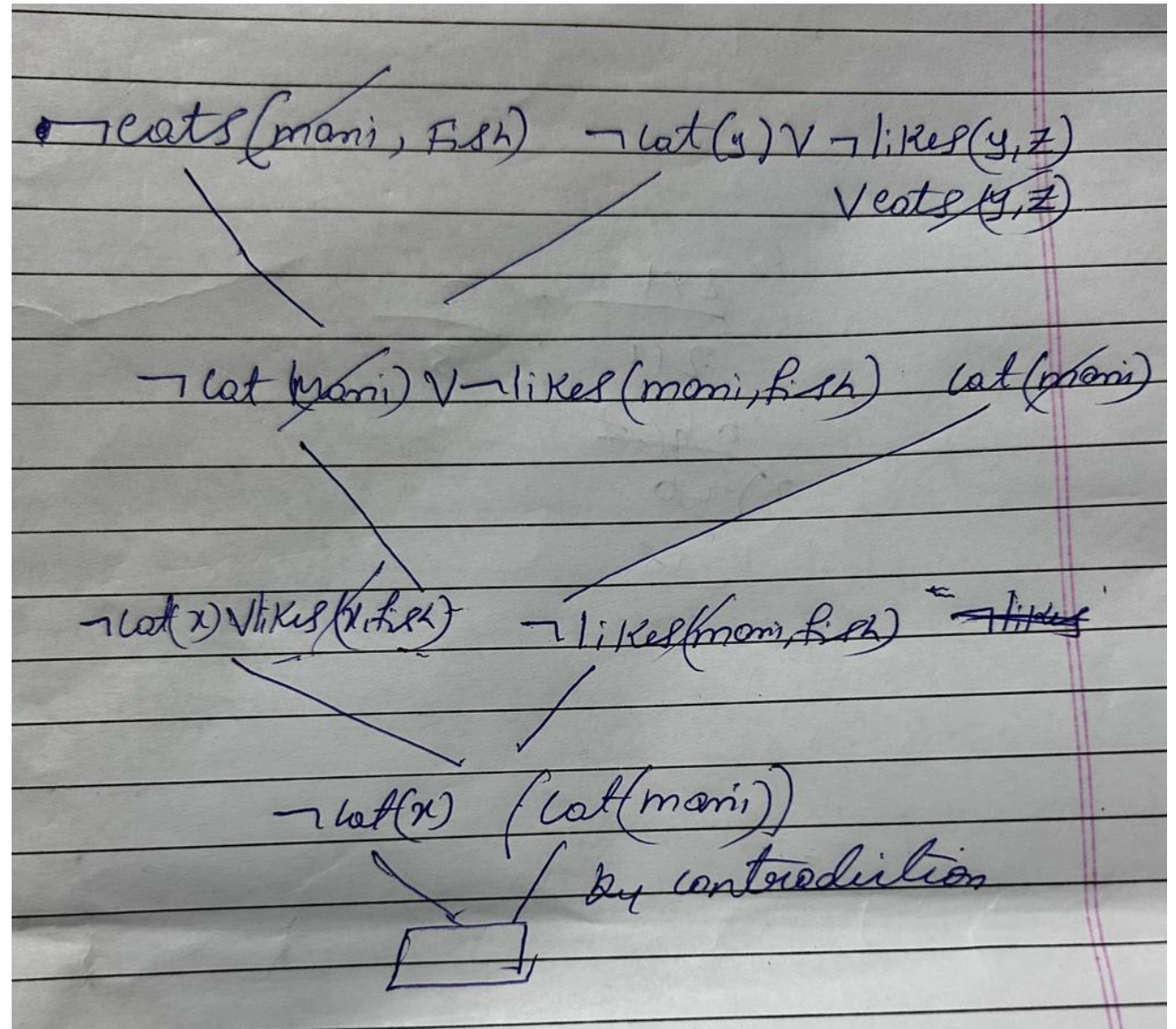
Step-4: Draw Resolution graph



Another ex:

1. Cat likes fish
2. Cat eats everything they like
3. Mani is a cat

To prove "mani eats fish"



Another ex:

1. Cat likes fish
2. Cat eats everything they like
3. Mani is a cat

To prove “mani eats fish”

Conversion to FOL

1. $\text{Cat}(x) \rightarrow \text{likes}(x, \text{fish})$
2. $[\text{Cat}(x) \wedge \text{eats}(x, y)] \rightarrow \text{eats}(x, y)$
3. $\text{Cat}(\text{mani})$

Convert to CNF

1. $\neg \text{cat}(x) \vee \text{likes}(x, \text{fish})$
2. $\neg \text{cat}(x) \vee \neg \text{eats}(x, y) \vee \text{eats}(x, y)$

Represent in graph

Classical planning:

- planning is one of the classic AI problems, study of rational action to achieve a goal.
- it has been used as the basis for applications like controlling robots and having conversations
- a plan is a sequence of actions
- an action is a transformation of a state
- we usually specify an initial state and a goal state, and then try to find (a short!) plan that transform the initial state to the goal state.

Example Planning Problem: Making Tea

- suppose you have a robot that can serve tea
- think about what the robot must do to make tea, e.g.:
- it must put water in the kettle
- heat the kettle
- get a cup

- pour hot water into the cup (after the water is hot enough)
- get a tea bag
- leave the tea bag in the water for enough time
- remove the tea bag
- add milk
- add sugar
- mix the tea
- serve the tea

there are many, many actions to consider! and most actions consist of many smaller sub-actions and in some situations very long sub-plans might be triggered

- e.g. if there is no milk, the robot might go to the store, or ask if creme would be an acceptable substitute
- e.g. if the robot drops the spoon on the floor, then it must pick it up and clean it before continuing

time and sensing is important as well

- how long should the robot mix the tea for?

Example Planning Problem: Blocks World

- suppose you have three cubical blocks, A, B, and C, and a robot arm that can pick up and move one block at a time
- suppose they are initially arranged on a table like this:

```
  C
B A  initial state
-----
```

- A and B are on the table, and C is on top of A
- suppose that we want to re-arrange them into this state:

```
A
B  goal state
C
---
```

furthermore, suppose we re-arrange blocks by picking up one block at a time and moving it to the table, or on top of another block

- we can only pick up a block if there is nothing on top of it. it's easy to see that these actions will solve this particular problem:
- move C to the table
- move B on top of C
- move A on top of B

- **PDDL**, the Planning Domain Definition Language is a **formal knowledge representation language designed to express planning models**.
- PDDL describes the four things we need to define a search problem: the initial state, the actions that are available in a state, the result of applying an action, and the goal test.

Components of a PDDL planning task:

- Objects: Things in the world that interest us.
- Predicates: Properties of objects that we are interested in; can be true or false.
- Initial state: The state of the world that we start in.
- Goal specification: Things that we want to be true. .
- Actions/Operators: Ways of changing the state of the world.

A set of ground (variable-free) actions can be represented by a single **action schema**. For example, here is an action schema for flying a plane from one location to another:

- Action(Fly(p, from, to),
 - PRECOND: $\text{At}(p, \text{from}) \wedge \text{Plane}(p) \wedge \text{Airport}(\text{from}) \wedge \text{Airport}(\text{to})$
 - EFFECT: $\neg \text{At}(p, \text{from}) \wedge \text{At}(p, \text{to})$
- The schema consists of the action name, a list of all the variables used in the schema, a **precondition** and an **effect**.
- The precondition and effect of an action are each conjunctions of literals (positive or negated atomic sentences).
 - The precondition defines the states in which the action can be executed,
 - Effect defines the result of executing the action

Example: The blocks world

- The blocks can be stacked, but only one block can fit directly on top of another. A robot arm can pick up a block and move it to another position, either on the table or on top of another block. The arm can pick up only one block at a time, so it cannot pick up a block that has another one on it.

```
Init( $On(A, Table) \wedge On(B, Table) \wedge On(C, A)$   
     $\wedge Block(A) \wedge Block(B) \wedge Block(C) \wedge Clear(B) \wedge Clear(C)$ )  
Goal( $On(A, B) \wedge On(B, C)$ )  
Action( $Move(b, x, y)$ ,  
    PRECOND:  $On(b, x) \wedge Clear(b) \wedge Clear(y) \wedge Block(b) \wedge Block(y) \wedge$   
             $(b \neq x) \wedge (b \neq y) \wedge (x \neq y)$ ,  
    EFFECT:  $On(b, y) \wedge Clear(x) \wedge \neg On(b, x) \wedge \neg Clear(y)$ )  
Action( $MoveToTable(b, x)$ ,  
    PRECOND:  $On(b, x) \wedge Clear(b) \wedge Block(b) \wedge (b \neq x)$ ,  
    EFFECT:  $On(b, Table) \wedge Clear(x) \wedge \neg On(b, x)$ )
```

Figure 10.3 A planning problem in the blocks world: building a three-block tower. One solution is the sequence [$MoveToTable(C, A)$, $Move(B, Table, C)$, $Move(A, Table, B)$].



- $\text{On}(b, x)$: To indicate that block b is on x , where x is either another block or the table.
- $\text{Move}(b, x, y)$: action for moving block b from the top of x to the top of y .
- $\text{Clear}(x)$: preconditions on moving b is that no other block be on it. In FOL, this would be $\neg \exists x \text{On}(x, b)$ or, $\forall x \neg \text{On}(x, b)$. However, PDDL doesn't support the quantifiers. This is represented using $\text{clear}(x)$.

The action Move moves a block b from x to y if both b and y are clear. After the move is made, b is still clear but y is not. A first attempt at the Move schema is

- $\text{Action}(\text{Move}(b, x, y),$
- $\text{PRECOND: } \text{On}(b, x) \wedge \text{Clear}(b) \wedge \text{Clear}(y),$
- $\text{EFFECT: } \text{On}(b, y) \wedge \text{Clear}(x) \wedge \neg \text{On}(b, x) \wedge \neg \text{Clear}(y))$.

When x is the Table, this action has the effect $\text{Clear}(\text{Table})$, but the table should not become clear; and

- when $y = \text{Table}$, it has the precondition $\text{Clear}(\text{Table})$, but the table does not have to be clear

To fix this, we do two things. First, we introduce another action to move a block b from x to the table:

Action(MoveToTable(b, x),

- PRECOND: $\text{On}(b, x) \wedge \text{Clear}(b)$,
- EFFECT: $\text{On}(b, \text{Table}) \wedge \text{Clear}(x) \wedge \neg \text{On}(b, x)$).

- Second, we take the interpretation of $\text{Clear}(x)$ to be “there is a clear space on x to hold a block.” Under this interpretation, $\text{Clear}(\text{Table})$ will always be true.
- The only problem is that nothing prevents the planner from using $\text{Move}(b, x, \text{Table})$ instead of $\text{MoveToTable}(b, x)$.

Init ($\text{At}(\text{C1}, \text{SFO}) \wedge \text{At}(\text{C2}, \text{JFK}) \wedge \text{At}(\text{P1}, \text{SFO}) \wedge \text{At}(\text{P2}, \text{JFK})$
 $\wedge \text{Cargo}(\text{C1}) \wedge \text{Cargo}(\text{C2}) \wedge \text{Plane}(\text{P1}) \wedge \text{Plane}(\text{P2})$
 $\wedge \text{Airport}(\text{JFK}) \wedge \text{Airport}(\text{SFO}))$

Goal ($\text{At}(\text{C1}, \text{JFK}) \wedge \text{At}(\text{C2}, \text{SFO})$)

Action(Load (c, p, a),

PRECOND: $\text{At}(\text{c}, \text{a}) \wedge \text{At}(\text{p}, \text{a}) \wedge \text{Cargo}(\text{c}) \wedge \text{Plane}(\text{p}) \wedge \text{Airport}(\text{a})$

EFFECT: $\neg \text{At}(\text{c}, \text{a}) \wedge \text{In}(\text{c}, \text{p})$)

Action(Unload(c, p, a),

PRECOND: $\text{In}(\text{c}, \text{p}) \wedge \text{At}(\text{p}, \text{a}) \wedge \text{Cargo}(\text{c}) \wedge \text{Plane}(\text{p}) \wedge \text{Airport}(\text{a})$

EFFECT: $\text{At}(\text{c}, \text{a}) \wedge \neg \text{In}(\text{c}, \text{p})$)

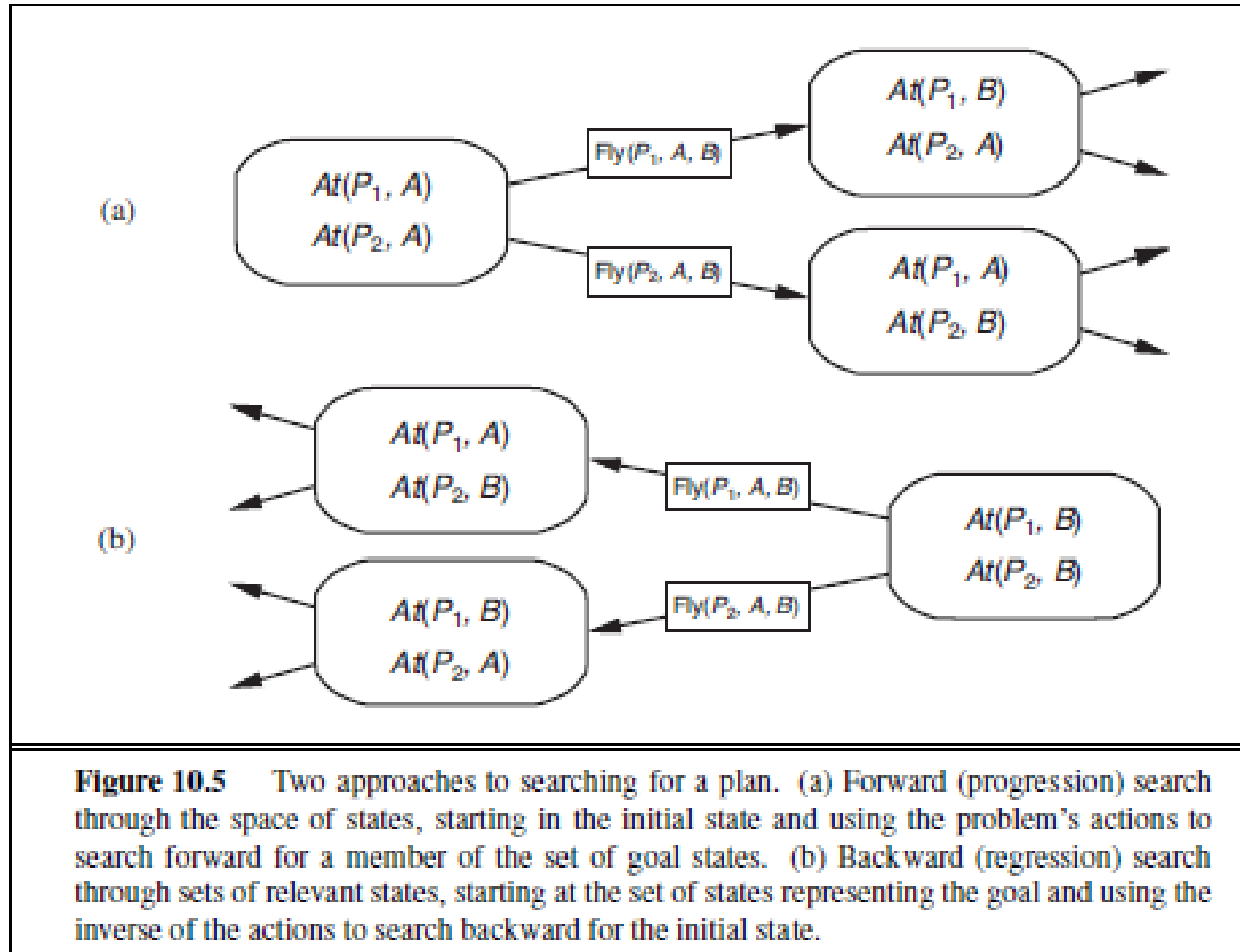
Action(Fly(p, from, to),

PRECOND: $\text{At}(\text{p}, \text{from}) \wedge \text{Plane}(\text{p}) \wedge \text{Airport}(\text{from}) \wedge \text{Airport}(\text{to})$

EFFECT: $\neg \text{At}(\text{p}, \text{from}) \wedge \text{At}(\text{p}, \text{to})$)

ALGORITHMS FOR PLANNING AS STATE-SPACE SEARCH:

Forward (progression) & backward state-space search:



Forward search is prone to exploring irrelevant actions, ex:

- Consider the noble task of buying a copy of *AI: A Modern Approach* from an online bookseller. Suppose there is an action schema `Buy(isbn)` with effect `Own(isbn)`. ISBNs are 10 digits, so this action schema represents 10 billion ground actions. An uninformed forward-search algorithm would have to start enumerating these 10 billion actions to find one that leads to the goal.
- Second, planning problems often have large state spaces. Consider an air cargo problem with 10 airports, where each airport has 5 planes and 20 pieces of cargo. The goal is to move all the cargo at airport A to airport B.

Backward (regression) relevant-states search:

- In regression search we start at the goal and apply the actions backward until we find a sequence of steps that reaches the initial RELEVANT-STATES state. It is called **relevant-states** search because we only consider actions that are relevant to the goal.

- In general, backward search works only when we know how to regress from a state description to the predecessor state description. For example, it is hard to search backwards for a solution to the n-queens problem because there is no easy way to describe the states that are one move away from the goal.

Heuristics for planning:

- Neither forward nor backward search is efficient without a good heuristic function.
- a heuristic function $h(s)$ estimates the distance from a state s to the goal and that if we can derive an **admissible** heuristic for this distance—one that does not overestimate.

PLANNING GRAPHS(PG):

- PG tells how special data structure called a **planning graph** can be used to give better heuristic estimates.
- This section shows how a special data structure called a **planning graph** can be used to give better heuristic estimates.
- The planning graph can't answer definitively whether G is reachable from S0, but it can *estimate* how many steps it takes to reach G.
- A planning graph is a directed graph organized into **levels**: first a level S0 for the initial state, consisting of nodes representing each fluent that holds in S0;
- then a level A0 consisting of nodes for each ground action that might be applicable in S0; then alternating levels Si followed by Ai; until we reach a termination condition.

Ex:

Init(Have(Cake))
Goal(Have(Cake) $\wedge$ Eaten(Cake))
Action(Eat(Cake))
 PRECOND: *Have(Cake)*
 EFFECT: $\neg$ *Have(Cake)* $\wedge$ *Eaten(Cake)*
Action(Bake(Cake))
 PRECOND: $\neg$ *Have(Cake)*
 EFFECT: *Have(Cake)*

Figure 10.7 The “have cake and eat cake too” problem.

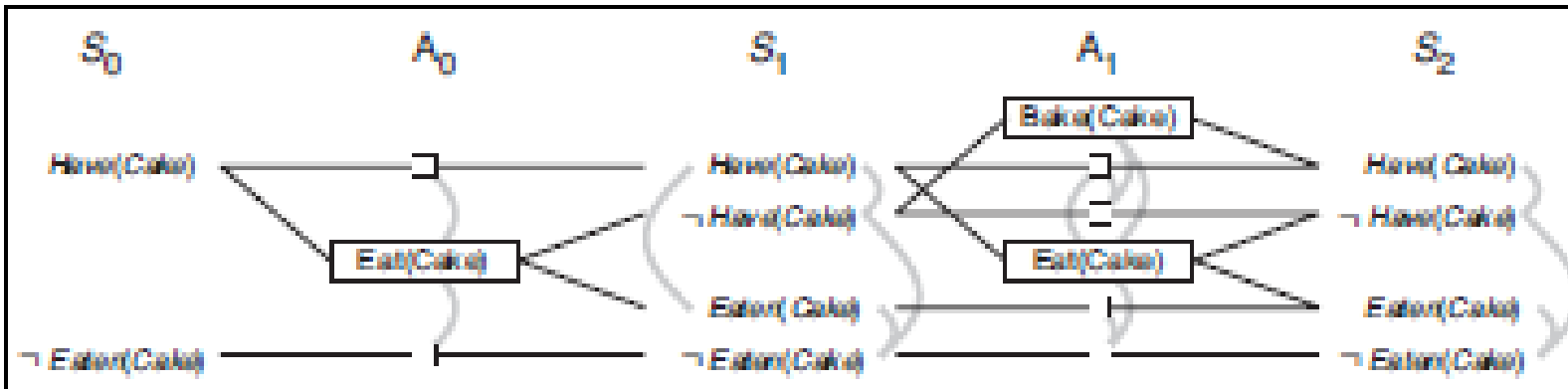


Figure 10.8 The planning graph for the “have cake and eat cake too” problem up to level S_2 . Rectangles indicate actions (small squares indicate persistence actions), and straight lines indicate preconditions and effects. Mutex links are shown as curved gray lines. Not all mutex links are shown, because the graph would be too cluttered. In general, if two literals are mutex at S_i , then the persistence actions for those literals will be mutex at A_i and we need not draw that mutex link.

UNIT IV

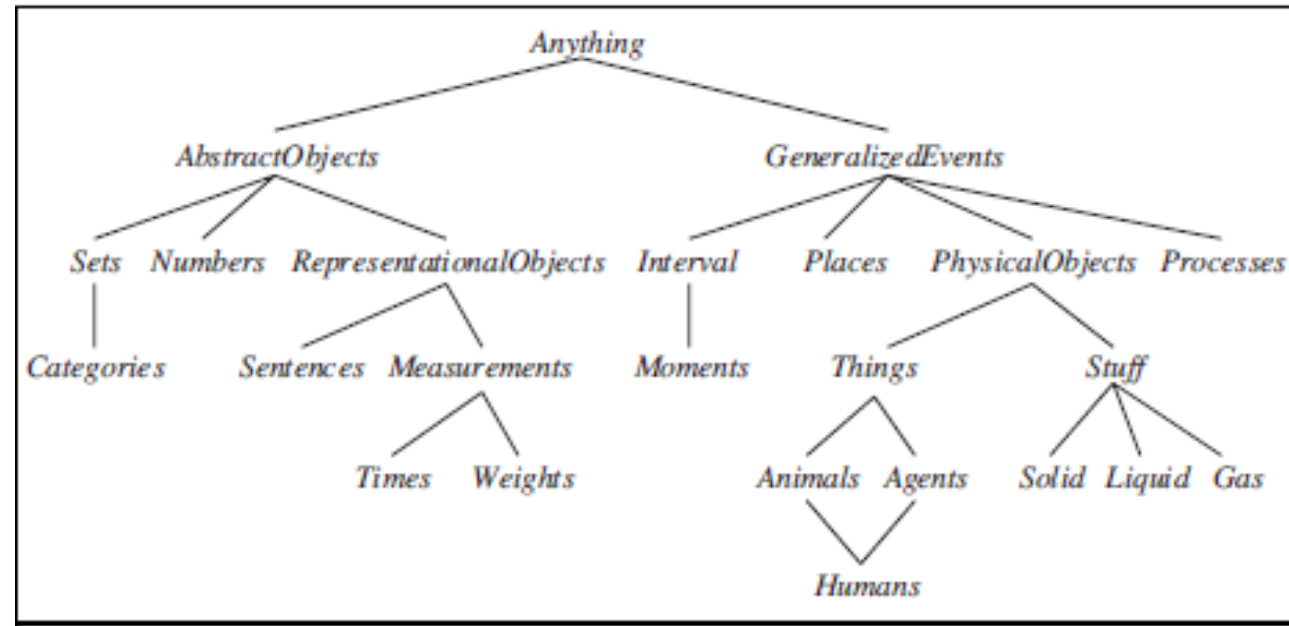
Knowledge Representation:

ONTOLOGICAL ENGINEERING:

- Complex domains such as shopping on the Internet or driving a car in traffic require more general and flexible representations.
- This chapter shows how to create these representations, concentrating on general concepts—such as *Events*, *Time*, *Physical Objects*, and *Beliefs*— that occur in many different domains.
- Representing these abstract concepts is sometimes called **ontological engineering**.

The general framework of concepts is called an **upper ontology** because of the convention of drawing graphs with the general concepts at the top and the more specific concepts below them, as in Figure 12.1

Ex: Each link indicates that the lower concept is a specialization of the upper one. Specializations are not necessarily disjoint; a human is both an animal and an agent for an example



Two major characteristics of general-purpose ontologies distinguish them from collections of special-purpose ontologies:

- A general-purpose ontology should be applicable in more or less any special-purpose domain (with the addition of domain-specific axioms).
- In any sufficiently demanding domain, different areas of knowledge must be ***unified***, because reasoning and problem solving could involve several areas simultaneously

CATEGORIES AND OBJECTS:

- The organization of objects into **categories** is a vital part of knowledge representation.
- Although interaction with the world takes place at the level of individual objects, *much reasoning takes place at the level of categories*.
- For example, a shopper would normally have the goal of buying a basketball, rather than a *particular* basketball such as BB9.
- Categories also serve to make predictions about objects once they are classified.

There are two choices for representing categories in first-order logic: predicates and objects.

- That is, we can use the predicate Basketball (b), or we can **reify** the category as an object, Basketballs.
- Member(b, Basketballs), abbreviated as $b \in \text{Basketballs}$ - b is a member of the category of basketballs.

- Subset(Basketballs, Balls), abbreviated as $\text{Basketballs} \subset \text{Balls}$ - Basketballs is a **subcategory** of Balls.
- Categories serve to organize and simplify the knowledge base through **inheritance**.

Ex: all instances of the category Food are edible

- Subclass relations organize categories into a **taxonomy**, or **taxonomic hierarchy**.
- FOL makes it easy to state facts about categories, either by relating objects to categories or by quantifying over their members. Here are some types of facts, with examples of each:
- An object is a member of a category.
 - $\text{BB9} \in \text{Basketballs}$
- A category is a subclass of another category.
 - $\text{Basketballs} \subset \text{Balls}$
- All members of a category have some properties.
 - $(x \in \text{Basketballs}) \Rightarrow \text{Spherical}(x)$

- Members of a category can be recognized by some properties.
 - $\text{Orange}(x) \wedge \text{Round}(x) \wedge \text{Diameter}(x)=9.5 \wedge x \in \text{Balls} \Rightarrow x \in \text{Basketballs}$
- A category as a whole has some properties.
 - $\text{Dogs} \in \text{DomesticatedSpecies}$
- We say that two or more categories are **disjoint** if they have no members in common.
 - $\text{Disjoint}(\{\text{Animals}, \text{Vegetables}\})$
 - $\text{ExhaustiveDecomposition}(\{\text{Americans}, \text{Canadians}, \text{Mexicans}\},$
 - $\text{NorthAmericans})$
 - $\text{Partition}(\{\text{Males}, \text{Females}\}, \text{Animals})$

Physical composition:

General **PartOf** relation to say that one thing is part of another. Objects can be grouped into partOf hierarchies, reminiscent of the Subset hierarchy:

- PartOf (Bucharest , Romania)
 - PartOf (Romania, EasternEurope)
 - PartOf (EasternEurope, Europe)
 - PartOf (Europe, Earth).
- The PartOf relation is transitive and reflexive; that is,
$$\text{PartOf}(x, y) \wedge \text{PartOf}(y, z) \Rightarrow \text{PartOf}(x, z) .$$

$$\text{PartOf}(x, x) .$$

Measurements:

- In both scientific and commonsense theories of the world, objects have height, mass, cost, and so on. The values that we assign for these MEASURE properties are called **measures**.

- If the line segment is called L1, we can write
$$\text{Length}(L1) = \text{Inches}(1.5) = \text{Centimeters}(3.81) .$$
- Conversion between units is done by equating multiples of one unit to another:
$$\text{Centimeters}(2.54 \times d) = \text{Inches}(d).$$
- Measures can be used to describe objects as follows:
 - $\text{Diameter}(\text{Basketball } 12) = \text{Inches}(9.5) .$
 - $\text{ListPrice}(\text{Basketball } 12) = \$ (19) .$
 - $d \in \text{Days} \Rightarrow \text{Duration}(d) = \text{Hours}(24) .$

EVENTS:

- To handle actions happening at the same time, we introduce an alternative formalism known as **event calculus**, which is based on points of time rather than on situations.

Event calculus reifies **fluents** and **events**.

- The **fluent** $\text{At}(\text{Shankar}, \text{Berkeley})$ is an object that refers to the fact of Shankar being in Berkeley, but does not by itself say anything about whether it is true.
- To assert that a fluent is actually true at some point in time we use the predicate T , as in $T(\text{At}(\text{Shankar}, \text{Berkeley}), t)$.

Events are described as instances of event categories.

- The event $E1$ of Shankar flying from San Francisco to Washington, D.C. is described as

$$E1 \in \text{Flyings} \wedge \text{Flyer}(E1, \text{Shankar}) \wedge \text{Origin}(E1, \text{SF}) \wedge \text{destination}(E1, \text{DC}) .$$

- If this is too verbose, alternative three-argument version of the category of flying events and say

$$E1 \in \text{Flyings}(\text{Shankar}, \text{SF}, \text{DC}) .$$

- $T(f, t)$ Fluent f is true at time t
- $Happens(e, i)$ Event e happens over the time interval i
- $Initiates(e, f, t)$ Event e causes fluent f to start to hold at time t
- $Terminates(e, f, t)$ Event e causes fluent f to cease to hold at time t
- $Clipped(f, i)$ Fluent f ceases to be true at some point during time interval i
- $Restored(f, i)$ Fluent f becomes true sometime during time interval i

Processes:

The events we have seen so far are what we call **discrete events**.

- Shankar's trip has a beginning, middle, and end. If interrupted halfway, the event would be something different.
- If we take a small interval of Shankar's flight, say, the third 20-minute segment, that event is still a member of Flyings. In fact, this is true for any subinterval.
- Categories of events with this property are called **process** categories or **liquid event** categories. Any process e that happens over an interval also happens over any subinterval:

$$(e \in \text{Processes}) \wedge Happens(e, (t_1, t_4)) \wedge (t_1 < t_2 < t_3 < t_4) \Rightarrow Happens(e, (t_2, t_3)) .$$

Following topics refer text book:

- **Mental objects and modal logic**
- **Reasoning systems for categories**

Acting under unCertanity

- Agents may need to handle uncertainty, whether due to partial observability, nondeterminism, or a combination of the two.
- An agent may never know for certain what state it's in or where it will end up after a sequence of actions.
- Suppose, for example, that an automated taxi! automated has the goal of delivering a passenger to the airport on time. The agent forms a plan, A90, that involves leaving home 90 minutes before the flight departs and driving at a reasonable speed.
- Even though the airport is only about 5 miles away, a logical taxi agent will not be able to conclude with certainty that "Plan A90 will get us to the airport in time."
- This is the **qualification problem** , for which we so far have seen no real solution

Summarizing uncertainty

Let's consider an example of uncertain reasoning: diagnosing a dental patient's toothache.

Let us try to write rules for dental diagnosis using propositional logic, so that we can see how the logical approach breaks down.

Consider the following simple rule:

$$\text{Toothache} \Rightarrow \text{Cavity}$$

The rule is wrong. Not all patients with toothaches have cavities; some of them have gum disease, an abscess, or one of several other problems:

$$\text{Toothache} \Rightarrow \text{Cavity} \vee \text{GumProblem} \vee \text{Abscess}$$

We could try turning the rule into a causal rule:

$$\text{Cavity} \Rightarrow \text{Toothache}$$

The rule is wrong. not all cavities cause pain. The only way to fix the rule is to make it logically exhaustive: to augment the left-hand side with all the qualifications required for a cavity to cause a toothache

Probabilistic reasoning:

Probabilistic reasoning is a way of knowledge representation where we apply the concept of probability to indicate the uncertainty in knowledge.

In probabilistic reasoning, we combine probability theory with logic to handle the uncertainty.

Need of probabilistic reasoning in AI:

- When there are unpredictable outcomes.
- When specifications or possibilities of predicates becomes too large to handle.
- When an unknown error occurs during an experiment.

In probabilistic reasoning, there are two ways to solve problems with uncertain knowledge:

- Bayes' rule
- Bayesian Statistics

BASIC PROBABILITY NOTATION:

- Probability can be defined as a chance that an uncertain event will occur. It is the numerical measure of the likelihood that an event will occur.
- The value of probability always remains between 0 and 1 that represent ideal uncertainties.
- $0 \leq P(A) \leq 1$, where $P(A)$ is the probability of an event A .
- $P(A) = 0$, indicates total uncertainty in an event A .
- $P(A) = 1$, indicates total certainty in an event A .

- We can find the probability of an uncertain event by using the below formula.

$$\text{Probability of occurrence} = \frac{\text{Number of desired outcomes}}{\text{Total number of outcomes}}$$

- Probabilistic reasoning in Artificial intelligence
- $P(\neg A)$ = probability of a not happening event.
- $P(\neg A) + P(A) = 1$.

Event: Each possible outcome of a variable is called an event.

Sample space: The collection of all possible events is called sample space.

Random variables: Random variables are used to represent the events and objects in the real world.

Prior probability: The prior probability of an event is probability computed before observing new information.

Posterior Probability: The probability that is calculated after all evidence or information has taken into account. It is a combination of prior probability and new information.

Conditional probability:

- Conditional probability is a probability of occurring an event when another event has already happened.
- Let's suppose, we want to calculate the event A when event B has already occurred, "the probability of A under the conditions of B", it can be written as:

$$P(A|B) = \frac{P(A \wedge B)}{P(B)}$$

- Probabilistic reasoning in Artificial intelligence
- Where $P(A \wedge B)$ = Joint probability of a and B
- $P(B)$ = Marginal probability of B.

Example:

In a class, there are 70% of the students who like English and 40% of the students who like English and mathematics, and then what is the percent of students those who like English also like mathematics?

Solution:

Let,

A is an event that a student likes Mathematics

B is an event that a student likes English.

$$P(A|B) = \frac{P(A \cap B)}{P(B)} = \frac{0.4}{0.7} = 57\%$$

Hence, 57% are the students who like English also like Mathematics.

INFERENCE USING FULL JOINT DISTRIBUTIONS:

In this section we describe a simple method for probabilistic inference—that is, the computation of posterior probabilities for query propositions given observed evidence.

We use the full joint distribution as the “knowledge base” from which answers to all questions may be derived.

	<i>toothache</i>		$\neg$ <i>toothache</i>	
	<i>catch</i>	$\neg$ <i>catch</i>	<i>catch</i>	$\neg$ <i>catch</i>
<i>cavity</i>	0.108	0.012	0.072	0.008
$\neg$ <i>cavity</i>	0.016	0.064	0.144	0.576
Figure 13.3 A full joint distribution for the <i>Toothache</i> , <i>Cavity</i> , <i>Catch</i> world.				

Bayes' theorem

- Bayes' theorem is also known as Bayes' rule, Bayes' law, or Bayesian reasoning, which determines the probability of an event with uncertain knowledge.
- In probability theory, it relates the conditional probability and marginal probabilities of two random events.
- Bayes' theorem was named after the British mathematician Thomas Bayes. The Bayesian inference is an application of Bayes' theorem, which is fundamental to Bayesian statistics.
- It is a way to calculate the value of $P(B|A)$ with the knowledge of $P(A|B)$.

To determine which test is more indicative of someone really carrying the virus, we can use **Bayes' Theorem** to calculate the probability that a person actually has the virus given that they tested positive.

Given:

- **Prevalence ($P(\text{Virus})$)** = 1% = 0.01 (the probability that someone has the virus).
- **$P(\text{Not Virus})$** = 99% = 0.99 (the probability that someone does not have the virus).
- **$P(\text{Test A Positive} \mid \text{Virus})$** = 95% = 0.95 (the probability that Test A detects the virus when the person has it).
- **$P(\text{Test A Positive} \mid \text{Not Virus})$** = 10% = 0.10 (the false positive rate for Test A).
- **$P(\text{Test B Positive} \mid \text{Virus})$** = 90% = 0.90 (the probability that Test B detects the virus when the person has it).
- **$P(\text{Test B Positive} \mid \text{Not Virus})$** = 5% = 0.05 (the false positive rate for Test B).

calculate the **posterior probability** that a person actually has the virus given that the test result is positive for each test. This can be done using Bayes' Theorem:

- $P(\text{Virus} \mid \text{Test Positive}) = P(\text{Test Positive} \mid \text{Virus})$

$$P(\text{Virus} \mid \text{Test Positive}) = \frac{P(\text{Test Positive} \mid \text{Virus}) \cdot P(\text{Virus})}{P(\text{Test Positive})}$$

- $P(\text{Test Positive})$ is the total probability of a positive test result, given by the law of total probability:

$$P(\text{Test Positive}) = P(\text{Test Positive} \mid \text{Virus}) \cdot P(\text{Virus}) + P(\text{Test Positive} \mid \text{Not Virus}) \cdot P(\text{Not Virus})$$

- **Step 1: Calculate for Test A**

- For Test A:

- $P(\text{Test Positive})_A = (0.95 \cdot 0.01) + (0.10 \cdot 0.99) = 0.0095 + 0.099 = 0.1085$

Now calculate

$$P(\text{Virus} \mid \text{Test Positive})_A$$

$$P(\text{Virus} \mid \text{Test Positive})_A = \frac{0.95 \cdot 0.01}{0.1085} = \frac{0.0095}{0.1085} \approx 0.0875$$

Step 2: Calculate for Test B

For Test B:

$$P(\text{Test Positive})_B = (0.90 \cdot 0.01) + (0.05 \cdot 0.99) = 0.009 + 0.0495 = 0.0585$$

Now calculate $P(\text{Virus} \mid \text{Test Positive})_B$:

$$P(\text{Virus} \mid \text{Test Positive})_B = \frac{0.90 \cdot 0.01}{0.0585} = \frac{0.009}{0.0585} \approx 0.1538$$

Conclusion:

- For Test A, $P(\text{Virus} \mid \text{Test Positive}) \approx 0.0875$.
- For Test B, $P(\text{Virus} \mid \text{Test Positive}) \approx 0.1538$.

Thus, **Test B** is more indicative of someone actually having the virus when the test result is positive because the probability of having the virus given a positive result is higher for Test B than for Test A.

UNIT V

Learning form Examples:

An agent is learning if it improves its performance on future tasks after making observations about the world.

Why would we want an agent to learn?

There are three main reasons:

- First, the designers cannot anticipate all possible situations that the agent might find itself in.

Ex: a robot designed to navigate mazes must learn the layout of each new maze it encounters.

- Second, the designers cannot anticipate all changes over time;

EX: a program designed to predict tomorrow's stock market prices must learn to adapt when conditions change from boom to bust.

- Third, sometimes human programmers have no idea how to program a solution themselves.

FORMS OF LEARNING:

Any component of an agent can be improved by learning from data. The improvements, and the techniques used to make them, depend on four major factors:

- Which component is to be improved.
- What prior knowledge the agent already has.
- What representation is used for the data and the component.
- What feedback is available to learn from.

Components to be learned

The components of these agents include:

1. A direct mapping from conditions on the current state to actions.
2. A means to infer relevant properties of the world from the percept sequence.
3. Information about the way the world evolves and about the results of possible actions the agent can take.
4. Utility information indicating the desirability of world states.
5. Action-value information indicating the desirability of actions.
6. Goals that describe classes of states whose achievement maximizes the agent's utility.

Example:

An agent training to become a taxi driver.

Every time the instructor shouts “Brake!” the agent might learn a condition–action rule for when to brake (component 1);

the agent also learns every time the instructor does not shout. By seeing many camera images that it is told contain buses, it can learn to recognize them (2).

By trying actions and observing the results—for example, braking hard on a wet road—it can learn the effects of its actions (3).

Then, when it receives no tip from passengers who have been thoroughly shaken up during the trip, it can learn a useful component of its overall utility function (4)

Feedback to learn from:

There are three types of feedback that determine the three main types of learning:

- In unsupervised learning the agent learns patterns in the input even though no explicit feedback is supplied. The most common unsupervised learning task is clustering:
- In reinforcement learning the agent learns from a series of reinforcements—rewards or punishments.
- In supervised learning the agent observes some example input–output pairs and learns a function that maps from input to output.

Refer the remaining topics from the textbook